Введение

Данный документ описывает требования к системе по раскладам Таро.

Назначение

Цель: создать сайт для работы с таро: просмотра описания карт, создания раскладов, требования толкования.

Задачи: создать сайт, подключить базу данных, сформировать базу карт Таро, сформировать алгоритмы раскладов, подключить нейросеть для толкования, настроить оплату толкования.

Область применения

Система будет использоваться пользователями для просмотра информации о картах, создании раскладов и получения толкований. Толкования будут платными.

Описание текущей ситуации

На данный момент клиенты заказывают расклады у специалистов, живых людей. Это может стоить больших денег, требует выделения времени на ожидания и/или записи в лист ожидания у специалиста. Кроме того, специалисты могут ошибаться или быть предвзятыми в силу своих человеческих качеств.

Проблемы:

- Стоимость расклада у специалиста;
- Ожидание приема или толкования;
- Предвзятость/ошибки специалиста;
- Трудности в поиске специалиста.

Описание системы

Акторы:

- Пользователь - заходит на сайт, заказывает расклад, расшифровку, смотрит возможности.
- Администратор - просматривает статистику и логи, настраивает новые расклады, создает колоды.

Краткое описание

При заходе на сайт пользователь видит главный экран с кратким описанием проекта и двумя основными кнопками:

- «Вытянуть случайную карту»;
- «Сделать расклад».

Дополнительные разделы — «О проекте» и «Просмотреть значение карт» — спрятаны в выпадающем меню в верхней части страницы (или в «гамбургер-меню» на мобильных устройствах). Регистрация и заполнение анкеты на старте не требуются — пользователь может сразу пользоваться основным функционалом.

«О проекте»
При нажатии на пункт меню «О проекте» пользователь переходит на отдельную страницу, где размещена информация о проекте, его возможностях, доступных колодах и раскладах, а также ссылка на документ о персональных данных (для случаев, когда пользователь добровольно оставляет какие-либо данные).

«Вытянуть случайную карту»
Пользователь нажимает кнопку, выбирает колоду (если их несколько), и система случайным образом выбирает карту из колоды, после чего отображает её название, краткое и полное описание. Опционально пользователь может предварительно выбрать масть.

«Сделать расклад»
Пользователь нажимает кнопку и выбирает один из двух путей:

- По колоде — система показывает список доступных колод; после выбора колоды отображаются расклады, доступные для неё, сгруппированные по первому тегу;
- По раскладу — система предлагает выбрать тег, а затем — расклад из соответствующей группы.

После выбора расклада система случайным образом выбирает карты в количестве и порядке, заданных структурой расклада, и отображает на странице:

- изображение расклада с пронумерованными позициями;
- список выпавших карт (название и краткое описание);
- предложение получить толкование с помощью нейросети.

При согласии пользователя система предлагает оплатить толкование (первое толкование — бесплатно). После успешной оплаты система формирует мастер-промпт на основе данных расклада, выпавших карт и (при наличии) добровольно указанных пользователем данных и направляет запрос в нейросеть. Полученный ответ отображается пользователю на той же странице.

«Просмотреть значение карт»
При выборе этого пункта в меню система предлагает выбрать колоду, затем — карту. При выборе карты отображается её название, краткое и полное описание.

Административная часть
Администратор через отдельную панель управления может:

- создавать новые колоды и наполнять их картами;

- создавать расклады с указанием колоды, количества карт, описания, значений позиций и тегов;
- просматривать логи и статистику;
- просматривать историю платежей.

У каждой карты есть свойства: название, автор, масть (необязательно), краткое описание, полное описание, колода.

У каждого расклада есть свойства: название, количество карт, описание расклада, значение позиций карт, дополнительная информация (необязательно), теги, список избранных колод (необязательно).

Администратор может просматривать логи и статистику. В логах фиксируются ошибки сайта, информация о вызовах нейросети с временем и датой и кодом ответа, время ожидания нейросети, стоимость в токенах. Логи патрицонируются по дате и хранятся 30 дней. Статистика отображает количество уникальных пользователей, количество вытягиваний карт, количество раскладов, количество запросов толкования, количество оплат, количество заработанных денег, количество потраченных токенов, количество потраченных денег, пиковую нагрузку за 1/3/7/30 дней.

UseCase диаграмму смотрите в приложении 1.

User Stories

US-01. Я, как пользователь, хочу посмотреть информацию о проекте, чтобы понимать, как пользоваться сайтом и какие у него есть возможности.

US-02. Я, как пользователь, хочу вытянуть случайную карту, чтобы быстро получить ответ на вопрос без сложного расклада.

US-03. Я, как пользователь, хочу сделать расклад, чтобы получить структурированное описание ситуации и прогноз.

US-04. Я, как пользователь, хочу посмотреть значения карт, чтобы самостоятельно интерпретировать расклад.

US-05. Я, как пользователь, хочу оплатить толкование, чтобы понять выпавшие карты.

US-06. Я, как пользователь, хочу пользоваться базовым функционалом сайта (просмотр значений карт, вытягивание случайной карты, создание расклада) без регистрации и сбора персональных данных, чтобы не тратить время на создание аккаунта. (Примечание: для запроса AI-толкования (бесплатного или платного) необходимо указать email для доставки результата и формирования фискального чека).

US-07. Я, как пользователь, хочу выбрать колоду и масть перед вытягиванием карты, чтобы получить ответ из интересующей меня категории.

US-08. Я, как пользователь, хочу получить первое толкование бесплатно, чтобы оценить качество сервиса перед оплатой.

US-09. Я, как пользователь, хочу видеть понятную инструкцию по оплате, чтобы без проблем завершить платеж.

US-10. Я, как администратор, хочу создавать колоды, чтобы увеличивать вариативность раскладов и удерживать пользователей.

US-11. Я, как администратор, хочу создавать расклады, чтобы добавлять новые виды раскладов и повышать вовлеченность.

US-12. Я, как администратор, хочу просматривать логи, чтобы оперативно выявлять и устранять ошибки.

US-13. Я, как администратор, хочу смотреть статистику, чтобы принимать решения по развитию проекта.

US-14. Я, как администратор, хочу редактировать и удалять колоды, чтобы поддерживать базу в актуальном состоянии.

US-15. Я, как администратор, хочу видеть в системе историю поступлений, общую сумму возвратов и чистую выручку, чтобы контролировать свои доходы.

US-16. Я, как администратор, хочу редактировать и удалять расклады, чтобы исправлять ошибки и обновлять контент.

US-17. Я, как администратор, хочу управлять ценами на толкования, чтобы гибко настраивать монетизацию.

US-18. Я, как администратор, хочу видеть историю платежей и их статусы, чтобы контролировать финансовые потоки.

US-19. Я, как администратор, хочу редактировать системный промпт (мастер-промпт) через админ-панель, чтобы улучшать качество ответов нейросети без участия разработчика.

US-20. Я, как администратор, хочу редактировать текст страницы «О проекте» и блок FAQ, чтобы держать информацию для пользователей в актуальном состоянии.

US-21. Я, как администратор, хочу просматривать и обрабатывать обращения из формы «Свяжитесь с нами», чтобы отвечать на вопросы и оформлять возвраты средств.

US-22. Я, как администратор, хочу видеть историю действий других администраторов (логи аудита), включая изменения цен, правки страницы "О проекте", обработку

обращений и восстановление удаленных сущностей, чтобы контролировать безопасность и предотвращать несанкционированные изменения.

US-23. Я, как администратор, хочу, чтобы система автоматически формировала чеки в «Мой Налог» при оплате, чтобы я соблюдала законодательство (422-ФЗ) без ручной работы.

US-24. Я, как пользователь, хочу получить копию оплаченного толкования на свой email, чтобы не потерять результат, если случайно закрою страницу или у меня пропадет интернет.

US-25. Я, как пользователь, хочу отправить сообщение через форму обратной связи на странице "Свяжитесь с нами", чтобы задать вопрос, сообщить о проблеме или запросить возврат средств.

US-26. Я, как администратор, хочу, чтобы система идентифицировала анонимных пользователей по cookies и браузерному отпечатку, чтобы ограничить их одним бесплатным толкованием и предотвратить злоупотребления с моей стороны.

US-27. Я, как администратор, хочу, чтобы система ограничивала частоту запросов (rate limiting) и блокировала ботов, чтобы защитить сервер от перегрузки, DDoS-атак и нецелевого расходования моего бюджета на токены нейросети.

US-28. Я, как администратор, хочу управлять картами в колоде (добавлять, редактировать, удалять и загружать массово через CSV), чтобы наполнять колоды контентом.

US-29. Я, как администратор, хочу видеть задачи, которые упали с ошибкой (DLQ), и иметь возможность перезапустить их, чтобы быстро восстанавливать генерацию без обращения к разработчику.

US-30. Я, как администратор, хочу видеть историю изменения цен (кто и когда менял), чтобы контролировать финансовую модель проекта и отслеживать динамику.

US-31: Я, как пользователь, хочу иметь возможность отозвать согласие на обработку моих персональных данных (email) через форму обратной связи, чтобы защитить свою приватность.

US-32: Я, как администратор, хочу видеть список запросов на удаление персональных данных и отмечать их как обработанные, чтобы соблюдать требования 152-ФЗ и не пропустить важные запросы.

US-33. Я, как администратор, хочу управлять текстами юридических документов (Публичная оферта, Правила возврата, Пользовательское соглашение), чтобы поддерживать их в актуальном состоянии и соблюдать требования законодательства.

US-34. Я, как пользователь, хочу иметь возможность ознакомиться с Публичной офертой и Правилами возврата перед оплатой, чтобы понимать условия оказания услуг и свои права.

US-35. Я, как администратор, хочу включить двухфакторную аутентификацию (2FA) и настроить сброс пароля по email, чтобы обеспечить дополнительную защиту административной панели.

US-36. Я, как администратор, хочу просматривать зависшие Outbox-события и перезапускать их отправку, чтобы быстро восстанавливать генерацию толкований после сбоев Redis без обращения к разработчику.

Примечание: Технические сценарии (health-check для мониторинга, настройка алертов WAL/Redis) не вынесены в отдельные User Stories, так как не несут прямой бизнес-ценности для конечного пользователя. Данные требования в полном объеме покрыты разделами «Надежность и отказоустойчивость» и «Мониторинг».

Функциональные требования

Общие требования

1. Система должна предоставлять доступ к просмотру карт, вытягиванию карт и созданию раскладов без регистрации, заполнения форм и сбора персональных данных. Для получения AI-толкования система запрашивает адрес электронной почты (обязательное поле), так как это необходимо для отправки копии результата и соблюдения требований 422-ФЗ (формирование фискальных чеков).
2. Система должна иметь административную панель для управления контентом и просмотра данных.
3. Система должна использовать базу данных для хранения колод, карт, раскладов, логов, статистики, платежей и информации об анонимных пользователях.
4. Система должна обеспечивать разграничение прав доступа между пользователем и администратором.

Админ-панель

Общие требования к административной панели

1. Административная панель должна быть доступна только пользователям с ролью «Администратор».
2. Панель должна располагаться на отдельном URL-адресе (например, /admin), отдельно от пользовательской части сайта.
3. Сессия администратора должна автоматически завершаться при неактивности в течение заданного времени (например, 30 минут).

4. Все действия, изменяющие данные (создание, редактирование, удаление колод, раскладов, промптов), должны логироваться с указанием того, какой администратор их совершил.
5. Административная панель должна иметь единое боковое меню (сайдбар) для навигации между разделами:
    a. Управление колодами;
    b. Управление раскладами;
    c. Управление системным промптом;
    d. Управление страницей “О проекте”;
    e. Управление юридическими документами (оферта, правила возврата, пользовательское соглашение);
    f. Управление запросами на удаление ПДн;
    g. Управление ценами;
    h. Платежи;
    i. Обращения;
    j. Статистика;
    k. Логи;
    l. Мониторинг задач (DLQ и Outbox-события).
6. В шапке панели должен отображаться текущий авторизованный администратор и кнопка «Выйти».
7. Все списки данных (колоды, расклады, платежи, логи) должны поддерживать пагинацию (разбиение на страницы) для удобства работы с большими объемами данных.
8. Все таблицы данных должны поддерживать сортировку по ключевым столбцам (дата, название, статус) и поиск по текстовым полям.
   Поля поиска по текстовым полям (в списках карт, колод, платежей) должны использовать задержку ввода (debounce) не менее 300 мс перед отправкой запроса на сервер, чтобы избежать избыточной нагрузки на API при наборе текста.
9. Все формы создания и редактирования должны валидировать введенные данные на стороне клиента (в браузере) и на стороне сервера.
10. При возникновении ошибки валидации система должна подсвечивать конкретное поле, в котором допущена ошибка, и выводить понятное сообщение.
11. Все действия, требующие необратимых изменений (удаление колоды, расклада), должны требовать двойного подтверждения (всплывающее окно с кнопкой «Вы уверены?»). Операция ручной очистки логов в интерфейсе администратора отсутствует.
12. После успешного сохранения данных система должна отображать уведомление об успехе и перенаправлять пользователя в список соответствующих записей (если применимо).
13. Административная панель должна быть адаптирована для работы на десктопных компьютерах и планшетах (ширина экрана от 1024px). Разработка мобильной версии для смартфонов не требуется.
14. Интерфейс панели не должен быть проиндексирован поисковыми системами (необходимо добавить директиву noindex или закрыть доступ через robots.txt).
15. Время в интерфейсе панели должно отображаться в фиксированном часовом поясе проекта UTC+5.

Колода: Добавление, редактирование и удаление

1. Администратор может создать новую колоду через веб-форму в административной панели.
2. При создании колоды администратор заполняет следующие параметры:
    a. Название колоды (обязательное поле, уникальное);
    b. Автор колоды (обязательное поле);
    c. Тип колоды (обязательное поле: Таро, Оракул, Игральные карты, Другое);
    d. Максимальное количество карт (обязательное поле, целое число, по умолчанию 78);
    e. allowReversed (обязательное поле, булево, по умолчанию false) — поддерживает ли колода перевёрнутые карты.
    f. Масти (необязательное поле, массив строк, может быть null);
    g. Описание колоды (обязательное поле, текст).
3. Формат данных колоды: json

```
{
  "name": "Таро Райдера-Уэйта",
  "author": "Артур Эдвард Уэйт",
  "type": "tarot",
  "max_cards": 78,
  "allow_reversed": true,
  "suits": ["Старшие арканы", "Жезлы", "Кубки", "Мечи", "Пентакли"],
  "description": "Классическая колода...",
  "is_deleted": false
}
```

4. Если в колоде нет мастей (например, оракул), поле suits заполняется значением null.
5. Система проверяет уникальность названия колоды среди активных записей (is_deleted = false). Если активная колода с таким названием уже существует, система возвращает ошибку 409 CONFLICT (код NAME_ALREADY_TAKEN) и предлагает изменить название.
6. При проверке уникальности названия колоды система игнорирует записи, у которых флаг is_deleted = true. Если администратор создает новую колоду с названием, которое уже есть у удаленной колоды, система разрешает создание. На уровне БД это обеспечивается частичным уникальным индексом.
7. Администратор может редактировать параметры уже созданной колоды (название, автор, тип, масти, описание).
   Важное ограничение при редактировании массива suits:
   Если администратор пытается удалить масть из массива suits, система проверяет наличие активных карт (is_deleted = false) с этой мастью в данной колоде. Если такие карты существуют, система возвращает ошибку 409 CONFLICT (код SUIT_IN_USE) с сообщением: «Невозможно удалить масть "Мечи", так как в колоде есть карты с этой мастью. Сначала переназначьте или удалите эти карты». Если карт с этой мастью нет — масть удаляется без ошибок.
8. При редактировании система выполняет ту же валидацию, что и при создании.

9. Изменение максимального количества карт не должно удалять уже добавленные карты, но должно предупреждать администратора, если текущее количество карт превышает новый лимит.
10. При редактировании колоды изменение allowReversed с true на false не удаляет уже заполненные перевёрнутые описания карт — они просто перестают использоваться.
11. При редактировании колоды изменение allowReversed с false на true делает поля reversed* доступными для заполнения у всех карт колоды.
12. Администратор может удалить колоду.
13. При удалении колоды администратором:
    a. система присваивает колоде флаг is_deleted = true;
    b. система должна скрыть колоду из пользовательской части сайта (не отображаться при выборе колоды);
    c. система должна удалить ID этой колоды из массивов favoriteDeckIds во всех раскладах, где она была отмечена как избранная.
14. В разделе управления колодами администратор должен иметь возможность переключать вид отображения: «Активные колоды» (по умолчанию) и «Архив удаленных колод».
15. Администратор может восстановить удаленную колоду (снять флаг is_deleted).
16. При нажатии кнопки «Восстановить» система проверяет, не занято ли имя удаленной колоды активной записью. Если активная колода с таким названием уже существует, система возвращает ошибку 409 CONFLICT (код NAME_ALREADY_TAKEN) с сообщением: «Невозможно восстановить колоду: название уже используется другой активной колодой. Переименуйте активную колоду или восстановите эту под новым именем». Если имя свободно — флаг is_deleted снимается.

Карты: Добавление, редактирование и удаление

1. После создания колоды администратор может наполнить её картами через веб-форму или загрузку файла.
2. При добавлении карты администратор заполняет:
    a. Название (обязательное поле);
    b. Масть (обязательное поле, если у колоды в настройках указаны масти; если у колоды мастей нет, поле скрыто или принимает значение null);
    c. Краткое описание (обязательное поле);
    d. Полное описание (обязательное поле);
    e. Номер карты (необязательное поле, целое число).
    f. reversedShortDescription (необязательное поле, текст) — краткое описание перевернутого значения. Отображается только если у колоды allowReversed = true.
    g. reversedFullDescription (необязательное поле, текст) — полное описание перевернутого значения. Отображается только если у колоды allowReversed = true.
3. При сохранении карты система должна проверять:
    a. Если у колоды задан массив suits (масти есть), то поле «Масть» у карты обязательно для заполнения. Если поле пустое, система выдает ошибку и не сохраняет карту.

b. Значение поля «Масть» должно строго соответствовать одному из элементов массива suits выбранной колоды. Если введенной масти нет в списке (например, опечатка или масть из другой колоды), система возвращает ошибку VALIDATION_ERROR (400) с сообщением: «Указанная масть отсутствует в текущей колоде. Доступные масти: [список]».
c. В интерфейсе административной панели поле «Масть» при создании/редактировании карты должно быть реализовано в виде выпадающего списка (dropdown), заполняемого из массива suits колоды, чтобы исключить возможность ввода неверного значения.

4. Формат данных карты:json

```
{
  "name": "Шут",
  "suit": "Старшие арканы",
  "short_description": "Начало нового пути, спонтанность, вера в лучшее.",
  "full_description": "Шут символизирует начало нового жизненного этапа...",
  "number": 0,
  "image_url_thumb": "string | null",
  "image_url_medium": "string | null",
  "image_url_original": "string | null",
  "reversed_short_description": "...",
  "reversed_full_description": "...",
  "is_deleted": false
}
```

5. Система должна поддерживать ввод карт следующими способами:
    a. По одной через веб-форму (с загрузкой изображения).
    b. Массово через загрузку ZIP-архива, содержащего CSV-файл со списком карт и папку с изображениями.
    c. Массово через загрузку JSON-файла (если изображения уже загружены в S3 и админ знает их URL).
6. При массовой загрузке система проверяет:
    a. соответствие формата файла;
    b. наличие всех обязательных полей;
    c. уникальность названий карт в пределах одной колоды (среди активных карт, is_deleted = false). Уникальность обеспечивается частичным уникальным индексом idx_cards_name_active;
    d. соответствие количества карт максимальному лимиту колоды;
    e. количество столбцов в каждой строке данных (должно совпадать с количеством заголовков);
    f. наличие обязательных полей (name, shortDescription, fullDescription);
    g. соответствие поля suit одной из мастей, указанных в настройках колоды (если масти заданы).
7. Если при загрузке обнаружены ошибки, система отображает отчет с указанием строк/карт, в которых есть проблемы, и не сохраняет данные до их исправления.
8. Максимальный размер загружаемого файла CSV или JSON не должен превышать 2 МБ.

9. При парсинге CSV-файла система должна проверять ячейки, начинающиеся с символов =, +, -, @. Такие значения должны быть экранированы (добавлен апостроф ' в начало), чтобы при возможном экспорте данных обратно в Excel они не выполнились как формулы.
10. Формат CSV:
    a. файл должен быть в кодировке UTF-8 строго с BOM (для корректного открытия в Excel). В качестве разделителя используется точка с запятой ;.
    b. Первая строка файла должна содержать заголовки столбцов. Обязательный порядок столбцов:

```
name; suit; shortDescription; fullDescription; number; reversedShortDescription; reversedFullDescription; image_filename
```

    c. Если поле необязательное (например, масть, номер или перевёрнутое описание), ячейка просто остается пустой (между разделителями ничего нет). Например:

```
Шут; Старшие арканы; Краткое описание; Полное описание; 0;;
```

    d. Если текст описания содержит точку с запятой (;) или кавычки, он должен быть обернут в двойные кавычки (стандартное правило CSV). Например:

```
Шут;Старшие арканы;"Начало пути; спонтанность";"Полное описание с "кавычками"";0;;
```

11. Формат массовой загрузки с изображениями (ZIP):
    a. Для одновременной загрузки карт и их изображений администратор загружает ZIP-архив (размером не более 50 МБ).
    b. Внутри ZIP-архива должна находиться следующая структура:
        i. Файл cards.csv (в кодировке UTF-8 с BOM, формат описан в п. 10).
        ii. Папка images/ с файлами изображений в форматах JPG или PNG.
    c. В столбце image_filename файла cards.csv указывается имя файла из папки images/ (например, fool.jpg). Если изображения для конкретной карты нет, ячейка остается пустой.
    d. Бэкенд должен автоматически распаковать архив, сопоставить значения из image_filename с файлами в папке и прогнать эти изображения через модуль оптимизации (сжатие, WebP, загрузка в S3).
12. Система должна поддерживать загрузку изображений для карт в форматах JPG и PNG размером не более 5 МБ.
    a. Безопасность загрузки изображений: Сервер должен проверять реальный MIME-тип файла по его сигнатуре (magic bytes), а не по расширению. Для проверки запрещено использовать только расширение файла. Библиотека обработки изображений должна проверять целостность файла до его сохранения (метод verify()).
    b. Защита от бомб декомпрессии: Система должна отклонять изображения с разрешением более 4000x4000 пикселей, даже если их вес менее 5 МБ, чтобы предотвратить исчерпание оперативной памяти сервера при их обработке.

13. При загрузке изображения серверная часть должна автоматически выполнять его обработку (сжатие и изменение размера) с использованием библиотеки обработки изображений (например, Pillow для Python).
14. Система должна автоматически генерировать три версии изображения в формате WebP, а также их копии в формате JPEG (для браузеров, не поддерживающих WebP). На фронтенде отдача изображений должна выполняться через HTML-тег <picture> с указанием источника WebP и fallback-источника JPEG:
    a. thumbnail (для сетки карт и списков): ширина 150px, вес до 15 КБ;
    b. medium (для отображения в раскладе и на экране карты): ширина 400px, вес до 60 КБ;
    c. original (для увеличения карты по клику): ширина 800px, вес до 200 КБ.
15. Оригинальный загруженный файл после обработки удаляется. Оптимизированные версии (thumbnail, medium, original) загружаются в объектное хранилище (S3). В базе данных хранятся только публичные URL-адреса этих файлов в S3 (image_url_thumb, image_url_medium, image_url_original).
16. Администратор может редактировать и удалять отдельные карты в колоде.
17. При удалении карты администратором
    a. система присваивает карте флаг is_deleted = true;
    b. система скрывает карты с флагом is_deleted = true из пользовательской части сайта. Они не отображаются в галерее карт (при просмотре значений) и не участвуют в генерации новых раскладов (алгоритм случайного выбора игнорирует их).
18. В списке карт колоды в административной панели по умолчанию отображаются только активные карты. Для просмотра удаленных должен быть доступен фильтр (или переключатель «Показать удаленные»).
19. Система должна отображать текущее количество карт в колоде и максимальный лимит.

Расклад: Добавление, редактирование и удаление

1. Администратор может создать новый расклад через веб-форму в административной панели.
2. При создании расклада администратор заполняет следующие параметры:
    a. Название (обязательное поле, уникальное);
    b. Описание (обязательное поле, текст — краткое описание расклада для пользователя);
    c. Количество карт (обязательное поле, целое число, автоматически проверяется на соответствие количеству позиций);
    d. Позиции расклада (обязательное поле, массив объектов — см. п. 3);
    e. Дополнительная информация (необязательное поле, текст — инструкция по чтению расклада, порядок интерпретации и т.п.);
    f. Теги (обязательное поле, массив строк, минимум один тег);
    g. Изображение расклада (обязательное поле, JPG или PNG, не более 5 МБ).
    h. Избранные колоды (необязательное поле, массив UUID) — список рекомендованных колод для этого расклада. Поле не ограничивает

использование расклада с другими колодами, а лишь выделяет эти колоды в интерфейсе пользователя.

3. Формат данных расклада: json

```
{
  "name": "Кельтский крест",
  "description": "Кельтский крест подходит для разбора ситуации...",
  "amount": 10,
  "positions": [ ... ],
  "info": "Линия времени: читай карты 4 → 1 → 6 → 10...",
  "tags": [
    {"normalized_name": "отношения", "display_name": "Отношения"},
    {"normalized_name": "ситуация", "display_name": "Ситуация"},
    {"normalized_name": "разбор", "display_name": "Разбор"}
  ],
  "image_url": "/images/spreads/celtic-cross.png",
  "favorite_deck_ids": ["uuid-1", "uuid-2"]
}
```

4. Каждая позиция расклада содержит:
    a. Номер позиции (обязательное поле, целое число, порядок вытягивания);
    b. Название позиции (обязательное поле, краткое — например, «Настоящее», «Вызов»);
    c. Значение позиции (обязательное поле, текст — что означает эта позиция в раскладе).
5. Система проверяет, что количество позиций в массиве positions совпадает со значением поля amount. Если есть расхождение — система отображает ошибку и не сохраняет расклад.
6. Система проверяет, что номера позиций идут последовательно (1, 2, 3… N) без пропусков и дублей.
7. Система проверяет уникальность названия расклада среди активных записей (is_deleted = false). Если активный расклад с таким названием уже существует, система возвращает ошибку 409 CONFLICT (код NAME_ALREADY_TAKEN) и предлагает изменить название. Уникальность обеспечивается частичным уникальным индексом в БД.
8. Система проверяет уникальность комбинации «количество карт + названия позиций». Если существует расклад с таким же набором позиций, система предупреждает администратора о возможном дубликате и предлагает изменить расклад или подтвердить создание.
9. Изображение расклада обязательно. Если администратор не загрузил изображение, система не сохраняет расклад и сообщает об ошибке.
10. Если администратор загружает изображение, система проверяет:
    a. формат (JPG или PNG);
    b. размер (не более 5 МБ).
11. Расклад не привязан к конкретной колоде. Один и тот же расклад может быть использован с любой колодой, в которой достаточно карт.
12. При выборе расклада пользователем система проверяет, что в выбранной колоде достаточно карт для этого расклада. Если карт меньше — система сообщает об ошибке и предлагает выбрать другую колоду или расклад.
13. Если расклад предполагает использование определённых мастей (например, только старшие арканы), это должно быть указано в дополнительной

информации. Система не проверяет соответствие мастей на уровне расклада — ответственность за это лежит на администраторе и пользователе.

14. Администратор может редактировать параметры уже созданного расклада (название, описание, количество карт, позиции, дополнительная информация, теги, изображение).
15. При редактировании система выполняет ту же валидацию, что и при создании.
16. Если расклад уже использовался пользователями, его редактирование не влияет на ранее сделанные расклады. История раскладов сохраняется в том виде, в котором они были созданы.
17. Администратор может удалить расклад.
18. При удалении расклада статистика по нему сохраняется. Расклад помечается как удаленный (мягкое удаление) и не отображается пользователям, но данные о его использовании остаются в базе.
19. Система должна отображать для каждого расклада: название, количество карт, теги, дату создания/обновления.
20. Теги представляют собой массив строк. Один расклад может иметь несколько тегов.
21. Пользователь выбирает расклад по тегам: система отображает разделы («Отношения», «Финансы», «Любовь», «Судьба» и т.п.), и расклад попадает во все разделы, теги которых у него указаны.
22. Система должна поддерживать группировку раскладов по тегам для отображения пользователю.
23. При сохранении тега система должна нормализовать его: удалить лишние пробелы по краям и сохранить в базе два значения: normalized_name (в нижнем регистре для поиска и автодополнения) и display_name (в том виде, в котором тег ввел администратор, для отображения пользователю).
24. Теги сохраняются в поле tags (JSONB) в виде массива объектов: [{"normalized_name": "отношения", "display_name": "Отношения"}, ...]. Это позволяет сохранить оригинальное написание и обеспечить быстрый поиск.
25. Система должна проверять, что тег не пустой и не содержит только пробелы.
26. При вводе тега в админ-панели система предлагает автодополнение на основе уже существующих в базе тегов (поиск по normalized_name, отображение display_name).
27. На старте теги хранятся в виде JSONB-массива объектов внутри сущности «Расклад». В будущем при масштабировании проекта они могут быть вынесены в отдельную таблицу-справочник.
28. Администратор может восстановить удаленный расклад (снять флаг is_deleted).
29. При восстановлении удаленного расклада система проверяет уникальность названия среди активных записей. При конфликте возвращается ошибка 409 CONFLICT (NAME_ALREADY_TAKEN).

Управление системным промптом

1. Администратор может просматривать текущий текст системной инструкции (мастер-промпта) в административной панели.
2. Администратор может редактировать текст системной инструкции.
3. При редактировании система должна валидировать, что текст не пустой и содержит базовые переменные (если они предусмотрены шаблоном).

4. Система должна сохранять историю изменений системной инструкции (кто, когда и что изменил).
5. Система должна предоставлять возможность откатить промпт к любой из сохраненных версий.
6. Изменения в системной инструкции должны вступать в силу без необходимости перезапуска сервера или внесения изменений в код (применяться к следующим запросам).

Управление страницей “О проекте”

1. Администратор должен иметь возможность редактировать основной текст страницы (философия, описание работы).
2. Администратор должен иметь возможность добавлять, редактировать и удалять вопросы и ответы в блоке FAQ.
3. Изменения, внесенные администратором, должны отображаться на странице «О проекте» для пользователей без необходимости перезапуска сервера.
4. При редактировании страницы «О проекте» и блока FAQ весь вводимый текст должен проходить серверную санитизацию для предотвращения XSS-атак.

Управление юридическими документами

1. В разделе управления юридическими документами администратор видит список из трех системных документов (фиксированные slug). Для каждого документа в списке отображается его название, текущая активная версия (например, v1.2) и дата последнего обновления:
    a. offer — Публичная оферта (договор на оказание услуг, условия оплаты);
    b. refund-policy — Правила возврата средств (условия и порядок возвратов);
    c. user-agreement — Пользовательское соглашение (правила использования сайта, дисклеймер ИИ, маркировка 16+).
2. Рядом с каждым документом должны быть доступны кнопки: «Редактировать» (создать новую версию) и «История версий».
3. Администратор может редактировать заголовок и текст любого документа.
4. Текст документов вводится в формате Markdown. В интерфейсе админ-панели должен быть предусмотрен предпросмотр (Preview) отрендеренного Markdown перед сохранением. При рендеринге Markdown на стороне сервера (перед сохранением в БД или перед отдачей в публичное API) система обязана проводить строгую санитизацию HTML. Запрещено выполнение любых тегов <script>, <iframe>, <object>, а также инлайновых обработчиков событий (например, onclick, onerror).
5. При сохранении изменений система автоматически:
    a. Находит текущую активную версию документа (is_active = true) и меняет её флаг на is_active = false.
    b. Создает новую запись с тем же slug, инкрементированной версией (например, с 1.0 на 1.1) и флагом is_active = true.
    c. Обновляет поле updated_at.
    d. Сбрасывает (инвалидирует) кэш в Redis для публичного эндпоинта GET /api/legal/{slug}, а также кэш списка документов в админ-панели;

    e. Записывает событие legal_doc.updated в лог аудита (с указанием ID админа, slug документа, старой и новой версии);
    f. Изменения становятся доступны пользователям немедленно;
    g. Возвращает администратору объект только что созданной активной версии документа.
6. Удаление юридических документов запрещено (кнопка «Удалить» отсутствует). Старые версии документов остаются в базе данных для юридической истории (их можно просмотреть через API).
7. В интерфейсе админ-панели должна быть доступна история версий документа с возможностью просмотра полного текста любой из них.
8. При нажатии на кнопку «История версий» открывается модальное окно (или отдельная страница) со списком всех версий документа, отсортированных по убыванию даты. Для каждой версии отображаются: номер версии, дата создания, статус (Активна / Архив) и кнопка «Просмотреть текст». Текст старых версий доступен только для чтения (редактирование архивных версий запрещено).
9. История изменений (версионирование текстов оферты) хранится в логах аудита. Если в будущем потребуется откат к предыдущей версии, она восстанавливается вручную администратором из логов.

Управление запросами на удаление ПДн

1. В разделе отображается список заявок из таблицы data_deletion_requests.
2. Для каждой заявки отображаются: дата, статус, хэш email (или пометка «Данные удалены», если статус COMPLETED), причина отказа (если REJECTED).
3. Администратор может открыть карточку заявки, чтобы увидеть исходный текст обращения пользователя (если он был создан через форму).
4. В карточке доступны две кнопки: «Подтвердить удаление» и «Отклонить».
5. При нажатии на «Подтвердить удаление» бэкенд запускает процесс анонимизации (согласно разделу «Доступ к документу о персональных данных», п. 8), меняет статус заявки и обнуляет поле email в самой заявке. Отменить это действие невозможно.

Управление ценами

1. Администратор может изменить цену одного AI-толкования при переходе в раздел “Управление ценами”.
2. Поле ввода должно принимать только положительные числа (целые или с двумя знаками после запятой).
3. При сохранении новой цены система должна валидировать, что значение больше нуля. Если введено некорректное значение (0, отрицательное число, текст), система выводит ошибку.
4. При сохранении новой цены система инвалидирует (сбрасывает) кэш GET /api/config/price в Redis, чтобы фронтенд получал актуальные данные при следующей загрузке страницы. Если пользователь уже находится на экране оплаты, защита от рассинхронизации осуществляется через проверку expected_amount (см. раздел «Создание платежа»).

5. Изменение цены не должно требовать перезапуска сервера или внесения изменений в код (цена должна храниться в базе данных или конфигурационном файле).
6. В разделе должна отображаться история изменения цен (кто и когда изменил цену) в режиме просмотра (без возможности редактирования или удаления прошлых записей), чтобы можно было отследить динамику.

Просмотр платежей

1. Администратор может просматривать историю платежей через административную панель.
2. Для каждого платежа отображаются:
    a. ID платежа в ЮKassa;
    b. сумма;
    c. статус (succeeded, canceled, refunded);
    d. статус снимка расклада (например, COMPLETED) — для понимания, была ли оказана услуга фактически (даже если деньги возвращены);
    e. дата и время создания;
    f. generated_spread_id — идентификатор конкретного сгенерированного расклада (снимка), за который была произведена оплата;
    g. visitor_id — идентификатор анонимного пользователя из payments.visitor_id;
    h. ссылка на расклад (если расклад ещё доступен в системе);
    i. статус фискального чека: created (создан), failed (ошибка создания), annulled (аннулирован при возврате), not_required (чек не требуется); .
    j. email пользователя (подтягивается напрямую из таблицы payments, а не через JOIN с generated_spreads);
    k. кнопка/ссылка «Сделать возврат» (которая ведет на страницу входа в ЛК ЮKassa), либо текст-подсказка: «Для оформления возврата перейдите в личный кабинет ЮKassa»;
    l. Статус снимка расклада (generated_spreads.generation_status) — для понимания, можно ли возобновить расклад или он уже отменен/завершен;
    m. Если payments.status = canceled, а generated_spreads.generation_status = PAYMENT_PENDING (пользователь отменил оплату, но может попробовать снова), выводить подсказку: «Ожидает повторной оплаты».
    n. Флаг дублирующей оплаты (possible_duplicate_payment). Если флаг равен true, в таблице платежей рядом со статусом выводится красная иконка (!) с подсказкой: «Дублирующая оплата. Требуется ручной возврат средств».
3. Если статус чека — failed, в таблице платежей он подсвечивается красным цветом. Над таблицей появляется баннер: «Внимание! Есть ошибки при создании/аннулировании чеков. Проверьте логи сервиса yookassa-to-mynalog».
4. Администратор может фильтровать платежи по:
    a. статусу;
    b. дате (диапазон);
    c. флагу possible_duplicate_payment (чекбокс «Только дублирующие оплаты»).

5. Администратор может искать платежи по:
    a. ID платежа в ЮKassa (yookassa_id) — точное совпадение.
    b. visitor_id — точное совпадение.
    c. generated_spread_id — точное совпадение.
    d. Email пользователя — частичное совпадение (поиск по подстроке).
    e. Сумма платежа — точное совпадение.

   Поиск должен игнорировать регистр (для email) и поддерживать частичное совпадение для текстовых полей.
6. Система должна отображать общую сумму успешных платежей за выбранный период, общую сумму возвратов (статус refunded) и чистую выручку (Net Revenue).
   Чистая выручка рассчитывается по формуле: Сумма успешных платежей (succeeded) - Сумма возвратов (refunded). (Комиссия эквайринга и налоги в рамках MVP не вычитаются, так как удерживаются на стороне ЮKassa и ФНС соответственно).
7. Администратор может выгрузить список платежей в формате CSV для дальнейшего анализа.
8. Система не инициирует выплаты (payouts) на банковские карты. Перечисление успешно принятых платежей на расчетный счет или карту администратора осуществляется автоматически на стороне платёжного сервиса ЮKassa в рамках заключённого договора эквайринга. Адмін-панель служит только для просмотра фактов поступления средств

Просмотр обращений

1. Система должна отображать список всех поступивших через форму обратной связи сообщений.
2. Для каждого обращения в списке отображаются:
    a. ID обращения;
    b. дата и время поступления;
    c. имя отправителя (если указано);
    d. контактные данные для связи (email или Telegram);
        i. telegram_username (если указан);
        ii. contact_channel ('email' или 'telegram');
    e. тема обращения (если указана, иначе выводить «Без темы»);
    f. статус обработки;
    g. краткий фрагмент текста сообщения (первые 100 символов).
3. По умолчанию список сортируется по дате поступления (сначала новые).
4. Каждое обращение имеет один из следующих статусов:
    a. «Новое» — обращение поступило, но ещё не просмотрено администратором (по умолчанию при создании).
    b. «В работе» — администратор просмотрел обращение и занимается его решением.
    c. «Решено» — вопрос закрыт (например, дан ответ, оформлен возврат средств).
5. Администратор может вручную менять статус обращения.
6. При клике на обращение открывается карточка с полным текстом сообщения и всеми контактными данными пользователя.

7. В карточке обращения администратор может оставить внутренний комментарий. Каждый новый комментарий добавляется в историю (таблица support_ticket_comments) с указанием автора и времени. История комментариев видна только администраторам.
8. Администратор может изменить статус обращения.
9. Администратор может удалить обращение (с обязательным подтверждением). Удаление является мягким (Soft Delete): система присваивает тикету флаг is_deleted = true, записывает deleted_at = now() и deleted_by = текущий_админ. Обращение скрывается из основного списка, но остается в базе данных для сохранения истории и аудита.
10. Для просмотра удаленных обращений в разделе «Обращения» должен быть доступен переключатель «Показать архив». Из архива обращение можно восстановить (снять флаг is_deleted).
11. Система должна поддерживать фильтрацию списка обращений по:
    a. статусу (Новое, В работе, Решено);
    b. дате поступления (диапазон дат).
12. Система должна поддерживать поиск по тексту сообщения, имени отправителя или контактным данным.
13. В шапке админ-панели рядом с иконкой профиля или в самом разделе «Обращения» должен отображаться счетчик новых обращений (например, красный бейдж с цифрой). Это позволит быстро замечать входящие запросы. Обработка запросов на отзыв персональных данных: Если текст обращения содержит запрос на отзыв согласия, администратор обязан создать заявку в разделе «Запросы на удаление ПДн» (если она не создалась автоматически) и нажать кнопку «Обработано». Система автоматически запустит фоновую задачу анонимизации данных (см. раздел «Доступ к документу о персональных данных», п. 8). Факт выполнения фиксируется во внутреннем комментарии к обращению. Важно: Ручное удаление email из таблиц БД администратором больше не требуется.
14. Если пользователь запрашивает возврат средств после того, как AI-толкование уже было успешно сгенерировано (статус задачи COMPLETED), администратор вправе отказать в возврате, так как услуга была оказана в полном объеме. Исключение составляют случаи, когда задача завершилась с ошибкой FAILED_VALIDATION (модерация 16+). В этом случае возврат обязателен по требованию пользователя. Решение о возврате принимается администратором в ручном режиме через ответ в обращении.

Просмотр статистики

1. Данные для раздела статистики должны браться из реляционной базы данных (PostgreSQL). Источником данных для событийных метрик является таблица analytics_events. Запрещается использовать Redis в качестве единственного источника для исторических данных или агрегированных метрик, так как Redis оптимизирован для оперативного кэширования, а не для долгосрочного хранения. Redis может использоваться только для получения оперативных данных (например, текущий размер очереди).
2. Данные в разделе обновляются в реальном времени или с минимальной задержкой (не более 5 минут).

3. Раздел поддерживает выбор периода отображения: за 1, 3, 7 или 30 дней (по умолчанию — 7 дней).
4. Для ключевых метрик отображается сравнение с предыдущим аналогичным периодом (например, «на этой неделе vs на прошлой»).
5. Все числовые метрики сопровождаются визуализацией (графики, диаграммы, карточки) для наглядности динамики.
6. Система отображает сводные карточки с ключевыми метриками за выбранный период:
    a. Количество уникальных посетителей (считается как COUNT(DISTINCT visitor_id) за выбранный период);
    b. Общее количество вытягиваний карт;
    c. Общее количество созданных раскладов;
    d. Количество запросов на AI-толкование (включая бесплатные и платные);
    e. Количество успешных оплат;
    f. Сумма заработанных средств (в рублях);
    g. Чистая выручка (Net Revenue) за период (Сумма успешных платежей минус сумма возвратов).
7. Система отображает воронку конверсии пользователей: посетители → вытягивание карты → создание расклада → запрос толкования → оплата.
8. Система отображает график активной аудитории (DAU / WAU / MAU) за выбранный период.
9. Система отображает конверсию из бесплатного толкования в платное (доля пользователей, которые после использования бесплатного запроса совершили хотя бы одну оплату).
10. Система отображает количество потраченных токенов (входных и выходных отдельно) за выбранный период.
11. Система отображает сумму затрат на токены YandexGPT (в рублях) за выбранный период.
12. Система рассчитывает маржинальность (разница между заработанными средствами и затратами на токены) за выбранный период.
13. Система отображает среднюю стоимость одного AI-толкования (затраты на токены / количество запросов).
14. Система отображает средний доход на одного платящего пользователя (ARPPU) за выбранный период.
15. Система отображает распределение раскладов по популярности (какие расклады выбирают чаще всего).
16. Система отображает распределение колод по популярности (какие колоды используются чаще всего).
17. Система отображает распределение раскладов по тегам (какие темы наиболее востребованы).
18. Система отображает количество ошибок при запросах к нейросети (таймауты, 4xx, 5xx) и их долю от общего числа запросов.
19. Система отображает среднее время генерации толкования (от отправки запроса до получения ответа от нейросети).
20. Система отображает пиковую нагрузку (максимальное количество запросов в минуту/час) за выбранный период.
21. Система отображает количество активных платежей в статусе pending (если такие есть).

22. Не все фильтры применимы ко всем метрикам. API должно корректно обрабатывать фильтры в зависимости от запрашиваемого блока:
    a. Фильтры deck_type и spread_id применяются только к метрикам контента (воронка, популярность раскладов/колод, количество вытягиваний и созданных раскладов).
    b. Фильтр payment_status применяется только к финансовым метрикам (сумма заработанных средств, количество оплат, ARPPU).
    c. Если фильтр не применим к конкретной метрике, он игнорируется (система не должна возвращать 0 или ошибку).
23. Система предоставляет возможность выгрузки статистики в формате CSV или XLSX для дальнейшего анализа.
24. Для всех расчетных метрик в системе должны использоваться следующие формулы (SQL-логика):
    a. Конверсия в оплату: (Количество уникальных visitor_id, совершивших успешную оплату / Общее количество уникальных visitor_id, запросивших AI-толкование) * 100%;
    b. Маржинальность: Сумма успешных платежей (amount) минус Общая сумма затрат на токены;
    c. Средняя стоимость одного AI-толкования: Общая сумма затрат на токены / Общее количество запросов к YandexGPT (включая COMPLETED, FAILED, FAILED_VALIDATION);
    d. ARPPU: Сумма успешных платежей / Количество уникальных visitor_id, совершивших хотя бы один успешный платеж;
    e. Пиковая нагрузка: Данная метрика не хранится в PostgreSQL. Бэкенд должен запрашивать данные из системы мониторинга (например, счетчики Nginx для RPS или Redis для длины очереди). API возвращает агрегированный результат.
    f. DAU (Daily Active Users): COUNT(DISTINCT visitor_id) из analytics_events, где event_type = 'visit' и created_at попадает в выбранный день.
    g. Воронка конверсии: Рассчитывается как последовательность уникальных visitor_id в analytics_events:
        - Шаг 1: visit
        - Шаг 2: card_draw OR spread_generate
        - Шаг 3: interpretation_request
        - Шаг 4: payment_success
    h. Популярность колод: COUNT(*) из analytics_events где event_type = 'card_draw', сгруппированный по deck_id.
    i. Популярность раскладов: COUNT(*) из analytics_events где event_type = 'spread_generate', сгруппированный по spread_id.
25. Для расчета агрегированных метрик (воронка конверсии, DAU/WAU/MAU, маржинальность) в PostgreSQL должны использоваться материализованные представления (Materialized Views). Обновление этих представлений (REFRESH MATERIALIZED VIEW) должно выполняться фоновой задачей Celery Beat не чаще одного раза в 5 минут, чтобы не блокировать основные таблицы.
26. Система должна отображать на дашборде блок «Требуют внимания» (Action Required), содержащий:
    a. Количество платежей с флагом possible_duplicate_payment = true (Дублирующие оплаты).

b. Количество обращений со статусом «Новое».
c. Количество задач в DLQ и зависших Outbox-событий.

При клике на счетчик дублирующих оплат система перенаправляет администратора в раздел «Платежи» с уже примененным фильтром ?possible_duplicate=true.

Просмотр логов

1. По умолчанию при открытии раздела отображаются логи за последние 24 часа.
2. Администратор может фильтровать логи по временному диапазону, уровню важности (INFO, WARN, ERROR), типу события (event_type), идентификатору актора (actor_id — ID администратора или ID анонимного посетителя). Дополнительно должен быть доступен фильтр «Только аудит» (is_audit=true), который скрывает технические логи и оставляет только события изменения данных (создание/редактирование/удаление, изменение цен, промптов, юридических документов и т.д.).
3. Администратор может искать логи по текстовому описанию (description) или по entity_id.
4. В таблице списка логов отображаются: timestamp, level, event_type, actor_id, success и description. При клике на запись открывается модальное окно с полной информацией, включая source_ip, entity_type, entity_id и содержимое поля payload (в виде отформатированного JSON).
5. При просмотре логов в административной панели (в модальном окне payload) администратор видит уже маскированный текст (например, [REDACTED]). Возможность просмотра «сырых» логов без маскировки в интерфейсе отсутствует.
6. Раздел «Логи» должен поддерживать пагинацию для удобной работы с большими объемами данных.

Мониторинг задач (DLQ)

1. В разделе отображается таблица задач, попавших в Dead Letter Queue. Для каждой задачи выводятся: task_id, тип задачи (например, generate_interpretation (очередь llm_tasks) или send_email (очередь email_tasks)), spread_id, дата и время попадания в DLQ, текст ошибки (кратко).
2. Рядом с разделом «Мониторинг задач (DLQ)» должен быть доступен раздел «Outbox-события». В нем отображается таблица событий со статусом FAILED. Для каждого события выводятся: ID, тип события (event_type), payload (в сжатом виде), дата создания, время следующей попытки (next_retry_at, если статус PENDING) и текст последней ошибки (last_error).
   При клике на событие открывается карточка с кнопкой «Переотправить». При нажатии на нее бэкенд переводит событие в статус PENDING и обнуляет retry_count.
3. В разделе «Мониторинг задач (DLQ)» или на дашборде статистики должен отображаться счетчик «Зависшие Outbox-события» (количество событий со статусом PENDING или FAILED, созданных более 10 минут назад). Если счетчик растет, это сигнализирует о проблемах с Redis или упавшем диспетчере.

4. При клике на задачу открывается карточка с полным текстом ошибки (stack trace), а также с исходными данными задачи (payload), которые передавались воркеру.
5. В карточке задачи должна быть кнопка «Повторить выполнение». При нажатии на нее задача удаляется из DLQ и повторно ставится в основную очередь. Если задача выполняется успешно, она исчезает из раздела.
6. Администратор может удалить задачу из DLQ без повторного выполнения (например, если понятно, что это дубликат или ошибка в данных пользователя).
7. Рядом с пунктом меню «Мониторинг задач» должен отображаться счетчик задач в DLQ (красный бейдж), чтобы администратор сразу видел, если что-то сломалось.
8. Рядом с разделом «Мониторинг задач» должен отображаться текущий размер активной очереди. Формат отображения: «Очередь: [Всего_в_Redis]/[MAX_HARD_QUEUE_LIMIT] (Бесплатные: [X]/[MAX_FREE_QUEUE_LENGTH], Платные: [Y]/[MAX_PAID_QUEUE_LENGTH], Резервы на оплате: [Z])».
   Значение Z (резервы) запрашивается из Redis (ключ queue_reservations:paid). Если общая сумма (Y + Z) приближается к MAX_PAID_QUEUE_LENGTH, индикатор платных задач подсвечивается желтым (например, 80% заполнения), а при 100% и больше — на красный, сигнализируя администратору о необходимости перехода на более мощную модель или увеличения квот Yandex Cloud.

Безопасность админ-панели

1. Доступ к панели должен осуществляться через страницу авторизации (логин и пароль).
2. Если у администратора включена 2FA, после ввода правильного логина и пароля система должна запрашивать 6-значный код из приложения. Доступ к админ-панели предоставляется только после успешной проверки второго фактора.
3. Пароли администраторов должны храниться в базе данных в виде хешей с использованием алгоритма Argon2id (с параметрами по умолчанию, рекомендованными OWASP). Хранение паролей в открытом виде запрещено.
4. При неудачных попытках входа (например, 5 неверных паролей подряд) система должна временно блокировать возможность входа с этого IP-адреса.
5. На странице авторизации администратора (входа в админ-панель) должна быть ссылка «Забыли пароль?».
6. При нажатии на ссылку «Забыли пароль?» система открывает форму, где администратор указывает свой email.
7. Независимо от того, существует ли указанный email в базе данных или нет, система должна возвращать одинаковый HTTP-ответ (200 OK) и отображать одинаковое сообщение: «Если аккаунт с таким email существует, мы отправили на него инструкцию по восстановлению».
    a. Если email существует: система генерирует уникальный одноразовый токен (UUID или криптографически стойкую строку), сохраняет его в базе данных и отправляет письмо через Unisender Go.

    b. Если email не существует: система выполняет «холостую» операцию (имитирует генерацию и хэширование токена для выравнивания времени ответа, чтобы предотвратить Timing Attacks), но не отправляет письмо и не сохраняет токен. В лог пишется событие WARN с типом auth.reset_password_failed_attempt (для мониторинга попыток подбора).
8. Система формирует и отправляет письмо на указанный email через сервис Unisender Go. Письмо должно содержать ссылку вида: https://ваш-сайт.ru/admin/reset-password?token={уникальный_токен}.
9. Для защиты от спама и подбора email на страницу запроса сброса пароля должен быть установлен двухуровневый rate limiting:
    a. По IP-адресу: не более 3 запросов с одного IP в час.
    b. По целевому email: не более 1 запроса на конкретный email в 5 минут.

   При срабатывании любого из лимитов система не отправляет письмо, но возвращает HTTP-ответ 200 OK с тем же текстом (для защиты от Email Enumeration). В лог пишется событие WARN с event_type: auth.reset_password_rate_limited.
10. При переходе по ссылке из письма система открывает страницу, где администратор вводит новый пароль (дважды для подтверждения). Пароль должен соответствовать единой парольной политике проекта (см. раздел "Аутентификация и модель ролей"): минимум 10 символов, наличие букв разного регистра и цифр. После успешной смены пароля токен аннулируется (удаляется из базы), а администратор перенаправляется на страницу входа.
11. В профиле администратора должна быть возможность включить двухфакторную аутентификацию (2FA) и запросить сброс пароля по email (см. US-35).
12. При активации 2FA система генерирует уникальный секретный ключ и отображает его в виде QR-кода для сканирования мобильным приложением-аутентификатором (TOTP).
13. Для завершения настройки администратор должен ввести 6-значный код из приложения. Если код верный, 2FA считается включенной.
14. Сразу после включения 2FA система должна сгенерировать список одноразовых резервных кодов (например, 8 штук) на случай потери телефона. Коды должны быть показаны администратору один раз с предложением сохранить их в надежном месте. В базе данных хранятся только их хеши.
15. Если администратор потерял телефон, он может использовать один из резервных кодов для входа. Использованный код сгорает. Если коды закончились — сброс 2FA возможен только через прямое вмешательство в базу данных (или через обращение к разработчику/техподдержке хостинга).
16. На ввод кода 2FA должен быть установлен лимит (например, не более 5 попыток ввода). При превышении лимита — временная блокировка входа на 15 минут.
17. Защита от CSRF (Cross-Site Request Forgery):
    a. Все операции, изменяющие состояние (POST/PUT/DELETE), должны быть защищены от CSRF с использованием токенов.
    b. Для административной панели (Synchronizer Token Pattern): При успешной аутентификации бэкенд генерирует уникальный csrf_token, привязанный к текущей сессии администратора, и передает его на фронтенд. Фронтенд обязан передавать этот токен в заголовке X-CSRF-

Token при каждом небезопасном запросе. Бэкенд-middleware проверяет валидность токена. При отсутствии или несовпадении токена возвращается ошибка 403 FORBIDDEN (код CSRF_TOKEN_INVALID).

c. Для публичной части (Double Submit Cookie): Бэкенд устанавливает не-HttpOnly cookie csrf_token (флаги Secure, SameSite=Strict). Кука должна устанавливаться при первом GET-запросе пользователя к публичному API, чтобы обеспечить работоспособность первого POST-запроса (например, /api/consent). Фронтенд считывает значение из cookie и дублирует его в заголовке X-CSRF-Token для всех POST-запросов. Бэкенд-middleware проверяет, что заголовок совпадает со значением cookie.
d. Cookie visitor_token и сессионные cookie администратора остаются строго HttpOnly и не используются в качестве CSRF-токенов.
e. Дополнительно (Defense in Depth) бэкенд проверяет заголовки Origin и Referer для всех небезопасных запросов. Если запрос приходит с недоверенного источника, он отклоняется с кодом 403 FORBIDDEN.
f. Для корректной работы механизма CSRF (Double Submit Cookie и Synchronizer Token) фронтенд обязан отправлять все запросы к API с флагом credentials: 'include' (или withCredentials: true в axios), чтобы браузер сохранял и передавал установленные cookie. Конфигурация CORS на бэкенде должна разрешать передачу учетных данных (Access-Control-Allow-Credentials: true) с доверенного домена.

Первый визит пользователя и идентификация

1. При первом заходе на сайт пользователь видит главный экран без обязательной регистрации или авторизации.
2. На главном экране отображаются:
    a. краткое описание проекта;
    b. две основные кнопки: «Вытянуть случайную карту» и «Сделать расклад»;
    c. выпадающее меню с пунктами «О проекте», «Просмотреть значение карт» и «Свяжитесь с нами».
3. Пользователь может сразу пользоваться основным функционалом сайта без заполнения каких-либо форм.
4. При самом первом заходе на сайт система должна отображать Cookie-баннер (всплывающее окно внизу экрана). Баннер содержит информацию о том, что сайт использует cookies и браузерный отпечаток (fingerprint) для идентификации анонимных пользователей и ограничения бесплатных толкований. Баннер должен содержать кнопки «Принять» и «Отклонить», а также ссылку на «Политику конфиденциальности». При загрузке страницы фронтенд должен считать не-HttpOnly cookie csrf_token (устанавливаемую бэкендом при начальном GET-запросе) для обеспечения работы кнопки «Принять».
5. Система использует два типа cookie:
    a. Строго необходимые (Essential): Сессионная cookie с подписанным visitor_token (HttpOnly, Secure, SameSite=Strict). Необходима для работы

сайта, сохранения состояния расклада и проведения оплаты. Устанавливается автоматически при первом заходе и не может быть отключена пользователем (иначе сайт не будет работать).
    b. Аналитические и Fingerprint: Используются для сбора браузерного отпечатка (fingerprint) и ограничения бесплатных толкований. Устанавливаются только после явного согласия.
    c. Отзыв согласия: Пользователь может изменить свое решение в любой момент, кликнув на ссылку «Настройки Cookie» в футере сайта. При отзыве согласия (выборе «Отклонить») система прекращает сбор данных fingerprint, а сохраненный ранее отпечаток аннулируется (обнуляется в БД через API). Строго необходимые cookie при этом остаются.
6. Сбор данных для fingerprint начинается на стороне клиента (Frontend). Только после нажатия пользователем кнопки «Принять» в Cookie-баннере, фронтенд запускает JS-скрипт сбора браузерного отпечатка (с использованием Canvas, WebGL, характеристик экрана и т.д.), вычисляет хэш (SHA-256) и отправляет его на эндпоинт POST /api/consent.
7. Идентификация при согласии («Принять»):
    a. Фронтенд отправляет POST /api/consent с телом { "consent": true, "fingerprint_hash": "<строка>" }.
    b. Бэкенд генерирует уникальный visitor_id (UUIDv4).
    c. Бэкенд создает JWT-токен (visitor_token), содержащий visitor_id, подписывает его секретным ключом и устанавливает в HttpOnly-cookie.
    d. Бэкенд берет fingerprint_hash из запроса, добавляет к нему серверную соль (secret salt), хэширует (например, через HMAC-SHA256) и сохраняет результат в поле fingerprint таблицы anonymous_visitors.
    e. Бэкенд устанавливает флаг cookie_consent_given = true.
    f. Пользователю доступно одно бесплатное AI-толкование.
8. Идентификация при отказе («Отклонить»):
    a. Фронтенд отправляет POST /api/consent с телом { "consent": false, "fingerprint_hash": null }.
    b. Бэкенд генерирует visitor_id и создает visitor_token (для работы корзины и оплаты).
    c. Fingerprint не собирается и не сохраняется (fingerprint = null в БД).
    d. Флаг cookie_consent_given устанавливается в false.
    e. Бесплатное AI-толкование не предоставляется (пользователь может получить только платную услугу). Если пользователь попытается запросить бесплатное толкование, система вернет ошибку CONSENT_REQUIRED с предложением дать согласие в модальном окне (см. раздел «Получение толкования расклада»).
9. Система сохраняет в базе данных запись об анонимном посетителе в таблице anonymous_visitors со следующими полями:
    a. visitor_id (строка, уникальный идентификатор);
    b. fingerprint (строка, хэш браузерного отпечатка);
    c. free_interpretation_used (булево, по умолчанию false);
    d. first_visit_at (дата и время);
    e. last_visit_at (дата и время);
    f. consent_at (время согласия);

    g. cookie_consent_given.
10. При каждом запросе бэкенд извлекает visitor_token из HttpOnly-cookie, проверяет его подпись и срок действия. Если токен валиден, бэкенд получает visitor_id и идентифицирует пользователя.
11. Если cookie удалены, система создает новую сессию (новый visitor_token). Если пользователь ранее давал согласие на fingerprint, фронтенд при новой загрузке страницы снова собирает хэш (если пользователь снова нажал «Принять») и отправляет его. Бэкенд сравнивает полученный хэш (с учетом соли) с записями в базе. При совпадении бэкенд привязывает новый visitor_token к старому visitor_id, предотвращая получение повторного бесплатного толкования.
12. Если пользователь очистил cookie, но его fingerprint совпал с уже существующим (и есть согласие), система не должна создавать новую запись anonymous_visitors. Флаг free_interpretation_used остается true, если он был установлен ранее.
13. Каждому анонимному пользователю предоставляется одно бесплатное AI-толкование.
14. При запросе толкования (вызов эндпоинта POST /api/spreads/{generated_spread_id}/interpretation) бэкенд выполняет проверку в одной транзакции с блокировкой строк generated_spreads и anonymous_visitors (SELECT ... FOR UPDATE в порядке: сначала расклад, затем посетитель):
    a. если interpretation_text уже есть — возвращается 200 { "status": "COMPLETED" } (идемпотентный повтор);
    b. если generation_status IN ('QUEUED','PROCESSING','RETRY_WAIT') — возвращается 200 { "status": "<current_status>" } (идемпотентный повтор для того же расклада);
    c. если free_interpretation_used = false и free_interpretation_pending = false — бэкенд выставляет free_interpretation_pending = true, free_interpretation_pending_since = now(), переводит расклад в QUEUED, ставит событие в outbox_events и возвращает 200 { "status": "QUEUED", "requires_payment": false };
    d. если free_interpretation_used = true — возвращается 402 с error.code = PAYMENT_REQUIRED;
    e. если free_interpretation_pending = true и активная бесплатная задача относится к другому generated_spread_id — возвращается 409 с error.code = FREE_INTERPRETATION_PENDING и details.active_free_generated_spread_id;
    f. если free_interpretation_pending = true, но активной бесплатной задачи нет — резерв считается «зависшим», бэкенд сбрасывает его (pending=false, pending_since=NULL), пишет WARN free_reservation.stale_reset и продолжает по ветке (c).

    Флаг free_interpretation_used не выставляется в момент постановки задачи. Он становится true только после успешного COMPLETED в воркере. Флаг free_interpretation_pending снимается (false) при COMPLETED, FAILED, FAILED_VALIDATION и CANCELED.
15. Система не гарантирует 100% защиту от повторного получения бесплатного толкования. Пользователь, использующий другое устройство или браузер с другим отпечатком, может получить еще одно бесплатное толкование. Это допустимое поведение.

16. При повторном визите пользователь видит тот же главный экран без изменений.
17. Если пользователь уже использовал бесплатное толкование, при попытке получить новое толкование система отображает уведомление:
    a. «Вы уже использовали бесплатное толкование. Чтобы получить новое, оплатите его»;
    b. кнопка «Оплатить толкование» (по клику отправляется запрос на /api/payments/create).
18. Если пользователь не использовал бесплатное толкование — система не отображает никаких дополнительных уведомлений до момента запроса толкования.
19. При повторном заходе на сайт (если cookie visitor_token сохранились или успешно восстановлены по fingerprint), фронтенд вызывает GET /api/spreads/latest-unviewed. Если система возвращает данные о раскладе (статус COMPLETED, is_viewed = false), на главном экране отображается баннер: «У вас есть готовое толкование от [дата]. Проверить статус».
20. При клике на баннер открывается модальное окно с просьбой ввести email, который был указан при оплате этого расклада (или сразу на страницу результата, если в текущей сессии есть валидный result_access_token). В окне отображается маска адреса (например, a***@mail.ru), чтобы пользователь понял, его ли это расклад».
21. Если введенный email совпадает с email в базе данных для этого generated_spread_id, система генерирует result_access_token (срок жизни 1 час для этого сценария) и возвращает его вместе с текстом толкования. Фронтенд открывает страницу с результатом, используя этот токен, и меняет флаг is_viewed = true. Если email не совпадает (или введен неверно 3 раза подряд), доступ блокируется, а система выводит сообщение: «Неверный email. Если вы не помните почту, напишите в техподдержку».

Доступ к документу о персональных данных

1. На странице «О проекте» размещается ссылка на документ о политике конфиденциальности и обработке данных.
2. Документ описывает:
    a. какие данные собирает система (анонимные идентификаторы, cookies, хэш браузерного отпечатка, вычисленный на стороне пользователя с использованием Canvas, шрифтов и характеристик устройства); Также пользователь может добровольно указать текст вопроса (user_context), который может содержать персональные данные. Этот текст передается в нейросеть для генерации ответа, хранится в базе в зашифрованном виде (в таблице generated_spreads) и не попадает в логи.
    b. с какой целью (ограничение бесплатных толкований);
    c. как пользователь может отказаться от сбора данных (отключить cookie). Указывается, что отозвать согласие можно через ссылку «Настройки Cookie» в футере сайта или через обращение в техподдержку.
3. Система собирает минимальный объем персональных данных (адрес электронной почты) исключительно для следующих целей:

    a. отправка копии AI-толкования;
    b. формирование и отправка фискального чека в ФНС (через сервис yookassa-to-mynalog);
    c. связь с пользователем в случае возникновения проблем с оплатой или обращений в поддержку.
4. Тексты сгенерированных ответов, сохраненные в логах для анализа ошибок (модерации), проходят обязательную процедуру деперсонализации (маскирования PII) перед сохранением.
5. Сбор email осуществляется на основании явного согласия пользователя, которое он дает путем установки чекбокса перед инициированием процесса получения толкования.
6. Система передает email следующим третьим лицам (обработчикам): сервису рассылки Unisender Go (для технической отправки письма) и сервису yookassa-to-mynalog (для формирования чека). Ссылки на результат (result_access_token) являются конфиденциальными и передаются только на email пользователя. Система обеспечивает защиту от случайной утечки токенов через заголовки браузера и логи веб-сервера.
7. Срок хранения email: В базе данных email хранится не более 12 месяцев с момента последнего взаимодействия пользователя с сервисом. При сохранении или обновлении email в таблицах generated_spreads и support_tickets бэкенд автоматически заполняет или перезаписывает поле email_delete_after (вычисляется как now() + interval '12 months').
   Пример: Если пользователь сделал расклад в январе 2026 года, а затем запросил толкование для него в декабре 2026 года, срок хранения email продлевается до декабря 2027 года.
8. Механизм автоматической анонимизации (по таймауту): Ежедневно в 03:00 фоновая задача app.tasks.anonymize_old_emails ищет записи в таблицах generated_spreads и support_tickets, где email_delete_after < now(), и заменяет реальный email на заглушку (deleted_user@anonymized.local). Данный механизм не затрагивает таблицы payments и receipt_jobs (422-ФЗ).
9. Механизм ручной анонимизации (по запросу пользователя): Если пользователь отозвал согласие через форму поддержки, администратор нажимает кнопку «Обработано» в заявке. Это запускает задачу anonymize_user_data, которая не только меняет email на заглушку в generated_spreads и support_tickets, но и обнуляет fingerprint в anonymous_visitors, а также выставляет cookie_consent_given = false и free_interpretation_used = true (защита от повторного получения бесплатного толкования).
10. Пользователь имеет право отозвать согласие, направив запрос через форму «Свяжитесь с нами». Бэкенд создает запись в таблице data_deletion_requests со статусом PENDING и сохраняет хэш email (email_hash).
    Алгоритм обработки запроса администратором:
    a. Администратор открывает карточку заявки в разделе «Запросы на удаление ПДн» и нажимает кнопку «Обработано».
    b. Бэкенд переводит статус заявки в COMPLETED, а поле email в самой таблице data_deletion_requests обнуляет (NULL), сохраняя только email_hash для аудита.
    c. Бэкенд запускает фоновую задачу анонимизации, которая:

i. В таблице generated_spreads заменяет реальный email на заглушку deleted_user@anonymized.local.
ii. В таблице support_tickets заменяет email на заглушку.
iii. В таблице anonymous_visitors обнуляет поле fingerprint (NULL), устанавливает cookie_consent_given = false и free_interpretation_used = true (защита от повторного получения бесплатного толкования).

d. Данные в таблицах payments и receipt_jobs НЕ анонимизируются, так как этого требует налоговое законодательство (422-ФЗ). В карточке заявки делается пометка об этом.
e. Если удаление невозможно из-за налоговых требований, администратор переводит заявку в статус REJECTED, указывает причину (например, «Требование 422-ФЗ») и также обнуляет поле email в заявке.

Вытягивание случайной карты

1. Пользователь нажимает кнопку «Вытянуть случайную карту» на главном экране.
2. Система всегда отображает экран выбора колоды — даже если колода одна.
3. Система отображает только те колоды, в которых есть хотя бы одна карта. Колоды без карт не показываются в списке выбора.
4. При формировании списка колод для выбора система должна отображать только колоды с флагом is_deleted = false.
5. Пользователь выбирает колоду. Если в выбранной колоде есть масти (suits не равно null), система отображает выбор:
    a. «Вытянуть из всей колоды»;
    b. список мастей.
6. Система исключает из списка масти, в которых нет карт. Если после исключения не осталось ни одной масти — система пропускает шаг выбора и вытягивает карту из всей колоды.
7. Если в колоде нет мастей (suits: null), система пропускает шаг выбора масти и сразу вытягивает карту из всей колоды.
8. Система случайным образом выбирает одну карту из выбранной области (вся колода или конкретная масть). Фронтенд отправляет запрос на POST /api/cards/draw, передавая выбранные параметры. Бэкенд возвращает объект карты.
9. Если у колоды включён флаг allowReversed, система случайным образом определяет положение карты (прямое или перевёрнутое) с вероятностью 50/50.
10. При случайном выборе карты из колоды (или из конкретной масти) алгоритм должен игнорировать карты с флагом is_deleted = true. Удаленные карты не могут выпасть пользователю.
11. Система отображает результат:
    a. изображение карты (при перевернутом положении — перевернутое на 180°);
    b. номер карты;
    c. масть (если есть);
    d. название карты;

    e. пометку «Перевёрнутая» (если применимо);
    f. краткое описание;
    g. полное описание.
12. Если у карты заполнены поля reversedShortDescription и reversedFullDescription, при перевёрнутом положении система показывает именно их. Если не заполнены — показывает базовые описания с пометкой «Перевёрнутая».
13. Если изображение карты отсутствует, система отображает заглушку с названием карты.
14. Под результатом система отображает две кнопки:
    a. «Вытянуть еще раз» — вытягивает новую карту с теми же параметрами (та же колода и та же масть);
    b. «Вернуться на главный экран» — возвращает пользователя на главную страницу.
15. Чтобы вытянуть карту из другой колоды или выбрать другую масть, пользователь должен вернуться на главный экран и заново пройти процесс выбора.
16. Толкование с помощью нейросети для одиночной карты не предусмотрено. AI-толкование доступно только для раскладов.
17. Количество вытягиваний карт не ограничено. Пользователь может вытягивать карты бесконечно.
18. История вытянутых карт не сохраняется и не отображается в пользовательском интерфейсе. Однако факт вытягивания карты асинхронно фиксируется в таблице analytics_events (event_type = 'card_draw') для последующего расчета статистики. В событии сохраняются visitor_id и deck_id.
19. Если при вытягивании произошла ошибка (например, проблема с базой данных), система отображает сообщение об ошибке и предлагает повторить попытку.
20. Все ошибки логируются в соответствии с разделом «Логирование и аудит».

Просмотр значения карт

1. При нажатии на пункт «Просмотреть значение карт» в выпадающем меню на главном экране система открывает страницу со списком доступных колод. Список колод загружается через GET /api/decks.
2. Система отображает только те колоды, в которых есть хотя бы одна карта. Колоды без карт скрываются из списка.
3. Список колод и список карт внутри колоды для просмотра значений должен фильтроваться по is_deleted = false.
4. Для каждой колоды в списке отображаются:
    a. название колоды;
    b. автор колоды;
    c. краткое описание колоды;
    d. обложка или изображение колоды.
5. Пользователь выбирает колоду. После этого система переходит к отображению списка карт выбранной колоды.
6. Система отображает список всех карт выбранной колоды в виде сетки (галереи) или списка.

7. Для каждой карты в списке отображаются:
    a. изображение карты (если загружено);
    b. название карты;
    c. масть (если у колоды есть масти);
    d. номер карты (если указан).
8. Если изображение карты отсутствует, система отображает заглушку с названием карты.
9. Система должна поддерживать поиск по названию карты (текстовое поле над списком).
10. Если в колоде есть масти, система должна предоставлять фильтр по мастям (например, выпадающий список или кнопки-табы).
11. Список карт должен быть отсортирован по номеру (по возрастанию). Если номер не указан, сортировка происходит по алфавиту.
12. При клике на карту из списка система открывает страницу (или модальное окно) с подробной информацией о ней.
13. На странице карты отображаются:
    a. увеличенное изображение карты;
    b. название карты;
    c. масть (если есть);
    d. номер карты (если есть);
    e. краткое описание (shortDescription);
    f. полное описание (fullDescription).
14. Если у колоды включён флаг allowReversed = true и у карты заполнены поля reversedShortDescription и reversedFullDescription, система должна отобразить оба значения (прямое и перевёрнутое). Для удобства их можно вывести в виде двух блоков («Прямое положение» и «Перевёрнутое положение») или сделать переключатель (табы).
15. Если поля reversed* не заполнены, система отображает только прямое значение и поясняет, что перевернутое значение для этой карты не задано.
16. На странице карты должна быть кнопка «Назад к списку карт» (возвращает к шагу 5) и кнопка «На главный экран».
17. Просмотр значений карт доступен без регистрации и без ограничений по количеству просмотров.
18. Просмотр значений карт не требует оплаты и не расходует бесплатное AI-толкование.
19. На странице просмотра карт не должно быть кнопок запроса AI-толкования (этот раздел предназначен исключительно для самостоятельного изучения карт).

Создание расклада

1. Пользователь нажимает кнопку «Сделать расклад» на главном экране.
2. Система предлагает два пути выбора:
    a. «По колоде» — сначала выбрать колоду, затем расклад;
    b. «По раскладу» — сначала выбрать тег, затем расклад.

    Список тегов загружается через GET /api/tags, а расклады — через GET /api/spreads с соответствующими фильтрами.

3. При выборе колоды (по пути «По колоде» или «По раскладу») система должна отображать только колоды с флагом is_deleted = false.
4. Если пользователь выбирает путь “По колоде”:
    a. Система отображает список всех колод, в которых есть хотя бы одна карта. Колоды без карт не показываются.
    b. Пользователь выбирает колоду. Система отображает все расклады, совместимые с выбранной колодой по количеству карт (количество карт в колоде ≥ количество карт в раскладе).
    c. Расклады, для которых у выбранной колоды установлен флаг «избранная», отмечаются звездочкой (★).
    d. Пользователь выбирает расклад.
5. Если пользователь выбирает путь “По раскладу”:
    a. Система отображает список всех доступных тегов (например, «Отношения», «Финансы», «Любовь», «Судьба»).
    b. Пользователь выбирает тег. Система отображает все расклады с этим тегом.
    c. Под списком раскладов отображается кнопка «Выбрать другой тег», возвращающая пользователя к предыдущему шагу.
    d. Пользователь выбирает расклад. Система отображает список всех колод, совместимых с выбранным раскладом по количеству карт.
    e. Колоды, отмеченные для этого расклада как избранные, показываются первыми и отмечаются звездочкой (★).
    f. Пользователь выбирает колоду.
6. После выбора колоды система отображает промежуточный экран подтверждения:
    a. текст: «Сконцентрируйтесь и коснитесь экрана, когда будете готовы»;
    b. кнопка «Я готов».
7. При нажатии на кнопку «Я готов» система:
    a. случайным образом выбирает карты из колоды в количестве и порядке, заданных раскладом;
    b. если у колоды включён allowReversed, для каждой карты случайным образом определяет положение (прямое или перевёрнутое) с вероятностью 50/50;
    c. отображает результат.
8. При случайном вытягивании карт для расклада алгоритм должен выбирать карты только из пула, где is_deleted = false.
9. Отображение результата включает:
    a. изображение расклада с пронумерованными позициями;
    b. список выпавших карт в порядке позиций (начиная с позиции 1), где для каждой карты указаны:
        i. номер позиции;
        ii. название карты;
        iii. масть (если есть);
        iv. краткое описание;
        v. пометка «Перевёрнутая» (если применимо).
10. Вытягивание карт сопровождается анимацией появления карт (по одной или последовательно).

11. В момент, когда система случайно выбирает карты для пользователя (после нажатия кнопки «Я готов»), она должна создать в базе данных запись о конкретном сгенерированном раскладе. Эта запись должна содержать снимок (snapshot) следующих данных на текущий момент:
    a. Название и описание расклада.
    b. Список позиций (номер, название, значение).
    c. Выпавшие карты (название, масть, положение, краткое описание).
    d. Дополнительная информация (если была).
12. Любые последующие изменения или удаление исходного шаблона расклада администратором не должны влиять на уже созданные снимки пользователей.
13. Под результатом расклада система отображает кнопку «Получить толкование».
14. Если в момент, пока пользователь находится на сайте, администратор удалил колоду или карту, при попытке сделать запрос пользователь должен получить корректную ошибку (например, «Выбранная колода больше недоступна, пожалуйста, обновите страницу»), а не системный сбой.
15. Если в процессе выбора колоды или карт пользователь нажимает кнопку "Сгенерировать", а администратор удалил (или изменил) эту колоду в этот же момент, бэкенд должен выполнить транзакционную проверку. Если сущность удалена (is_deleted = true), система возвращает HTTP-статус 409 Conflict с error.code = ENTITY_DELETED и сообщением: "Выбранная колода была удалена. Пожалуйста, обновите страницу и выберите другую".
16. При вытягивании карт для расклада бэкенд должен выполнять запрос к базе данных в рамках транзакции с блокировкой выбранных карт (SELECT FOR UPDATE). Это необходимо для предотвращения ситуации, когда администратор удаляет или редактирует карту в момент, пока система формирует для пользователя снимок (snapshot) расклада.

Получение толкования расклада

1. При нажатии на кнопку «Получить толкование» система открывает форму запроса, которая содержит:
    a. опциональное поле для ввода контекста или вопроса пользователя (с атрибутом maxlength="1500");
    b. обязательное поле для ввода адреса электронной почты (с текстом-подсказкой «Укажите email, чтобы мы продублировали толкование на почту. Это поможет, если вы случайно закроете страницу, а также потребуется для отправки чека об оплате»);
    c. чекбокс согласия на обработку персональных данных: «Я согласен(на) на обработку моих персональных данных (email) для получения толкования, отправки фискальных чеков и связи со мной в случае необходимости». Ссылка в тексте чекбокса должна вести на Политику конфиденциальности.
2. Система должна проверять введенный email на соответствие формату (наличие символа @, домена и доменной зоны верхнего уровня, отсутствие пробелов) согласно стандарту RFC 5322.
   Валидация выполняется как на стороне клиента (Frontend, до отправки запроса), так и на стороне сервера (Backend, с использованием Pydantic

EmailStr). Максимальная длина email — 254 символа. При несоответствии формата API возвращает ошибку 422 VALIDATION_ERROR с сообщением: «Пожалуйста, введите корректный адрес электронной почты (например, name@domain.ru)».

3. После заполнения формы фронтенд отправляет запрос на POST /api/spreads/{generated_spread_id}/interpretation.
4. Бэкенд проверяет, есть ли уже готовый текст толкования для этого снимка. Если есть — возвращает его сразу (без списания денег и запросов к ИИ).
5. Обработка отказа от сбора данных (Fingerprint):
   Если пользователь ранее отказался от сбора данных (cookie_consent_given = false), при попытке получить бесплатное толкование система не может предоставить ему эту возможность (так как механизм ограничения бесплатных запросов завязан на fingerprint). В этом случае бэкенд возвращает ошибку 403 CONSENT_REQUIRED с сообщением: «Для получения бесплатного толкования необходимо согласие на обработку данных браузера».
   Фронтенд в ответ на эту ошибку должен открыть модальное окно со следующим текстом: «Вы не можете получить бесплатное толкование, так как ранее отказались от сбора данных. Мы используем отпечаток браузера (fingerprint) исключительно для того, чтобы каждый пользователь мог получить одно бесплатное толкование. Без этого мы не можем отличить вас от новых пользователей. Вы можете согласиться на сбор данных или оплатить толкование.»
   Кнопки в модальном окне:
   a. «Согласиться и получить бесплатно» — при нажатии фронтенд собирает fingerprint, отправляет POST /api/consent (consent: true), дожидается успеха и автоматически повторяет запрос на POST /api/spreads/{id}/interpretation.
   b. «Оплатить толкование» — перенаправляет на экран оплаты (если пользователь не хочет давать согласие).
6. Бэкенд проверяет статус бесплатного толкования:
   a. Если бесплатное не использовано — ставит задачу в очередь, возвращает статус QUEUED, фронтенд переходит в режим ожидания.
   b. Если бесплатное уже использовано — возвращает PAYMENT_REQUIRED. Фронтенд открывает окно оплаты ЮKassa.
7. После успешной оплаты (или получения статуса QUEUED) страница переходит в режим ожидания, бэкенд формирует мастер-промпт на основе данных снимка, карт, контекста и ставит задачу воркеру.
8. Система не позволяет запрашивать повторную генерацию для одного и того же снимка.
9. Кнопка «Оплатить» / «Получить толкование» должна быть неактивна (заблокирована), пока пользователь не введет валидный email и не установит галочку согласия.
10. После перенаправления пользователя на страницу оплаты ЮKassa, на исходной вкладке сайта должен запускаться таймер ожидания (например, 15 минут). Этот таймер синхронизирован с временем жизни резерва слота в Redis. Если за это время статус платежа не изменился на succeeded (пользователь закрыл окно или оплата зависла), фронтенд должен прекратить опрос статуса и

показать сообщение: «Время ожидания оплаты истекло. Если вы уже оплатили, подождите немного или обновите страницу».

11. Когда пользователь возвращается на сайт по return_url после оплаты в ЮKassa, фронтенд должен немедленно отправить запрос на бэкенд для проверки актуального статуса платежа (без ожидания фоновой сверки). Если платеж уже succeeded, система должна вернуть статус QUEUED или PROCESSING для фронтенда. Если же фронтенд видит, что платеж в базе все еще pending, он вызывает внутренний API-метод /api/payments/verify, который инициирует однократную сверку с ЮKassa. Этот метод должен использовать ту же транзакционную блокировку, что и вебхук, чтобы избежать дублирования задач.
12. После успешной оплаты (или подтверждения бесплатного запроса) страница должна переходить в режим ожидания. На экране отображается анимация загрузки и текст: «Аметистовая жаба анализирует ваш расклад. Пожалуйста, не закрывайте страницу. Ожидаемое время: [X] секунд/минут (динамически обновляется на основе estimated_wait_seconds из API)». Если в ответе API поле estimated_wait_seconds равно null (задача уже в работе), фронтенд должен отображать текст «Ожидаемое время: менее минуты».
13. Пока на экране отображается индикатор загрузки («Аметистовая жаба анализирует ваш расклад...»), система должна перехватывать событие попытки закрытия или обновления вкладки браузера (beforeunload). При попытке пользователя покинуть страницу браузер должен показать системное диалоговое окно с предупреждением: «Вы уверены, что хотите уйти? Толкование еще не готово. Если вы закроете страницу, результат придет только на указанный email».
    Важно: Перехват события beforeunload активируется только в том случае, если пользователь уже отправил форму (ввел email и нажал кнопку запроса толкования) и задача находится в статусе QUEUED или PROCESSING. Если пользователь просто открыл страницу, окно не показывается.
14. Браузер пользователя должен автоматически отправлять GET-запросы на эндпоинт /api/spreads/{generated_spread_id}/status каждые 3–5 секунд, пока не получит статус COMPLETED (успех) или FAILED / FAILED_VALIDATION (ошибка).
15. Как только polling возвращает статус completed, фронтенд должен скрыть экран ожидания и выполнить GET-запрос на эндпоинт /api/spreads/{generated_spread_id}/view (передав cookie с visitor_token для авторизации). Получив текст, система отображает его под результатом расклада.
16. После успешной генерации текста толкования система должна сформировать уникальную безопасную ссылку на страницу с результатом. Ссылка должна содержать result_access_token — криптографически подписанный JWT-токен, привязанный к generated_spread_id и имеющий срок жизни не менее 30 дней.
17. На экране с готовым результатом система должна отображать кнопку «Скопировать ссылку на результат» и текст-подсказку: «Сохраните эту ссылку, чтобы иметь доступ к толкованию в любое время». Ссылка имеет формат: https://[домен]/result/{generated_spread_id}?token={result_access_token}.
18. Под текстом сгенерированного толкования на экране результата должен отображаться обязательный юридический дисклеймер: «Ответ сгенерирован

искусственным интеллектом и носит информационно-развлекательный характер. Может содержать неточности. Маркировка 16+.»

19. Доступ к результату расклада (как через ссылку из письма, так и через баннер на главной) должен осуществляться только при наличии валидного result_access_token. Простое знание UUID расклада не дает доступа к тексту. При переходе по ссылке с валидным токеном пользователь видит результат без необходимости повторной авторизации или ввода email. В самом тексте результата не должны отображаться персональные данные (email), чтобы минимизировать риски при пересылке ссылки.
20. Флаг is_viewed изменяется на true в базе данных только в момент, когда бэкенд успешно возвращает текст толкования пользователю. Это происходит в двух случаях:
    a. При успешном вызове эндпоинта /view (будь то после завершения polling или при переходе по Signed URL из письма).
    b. После успешной верификации email на сайте (для просмотра ранее сгенерированного расклада).

    Воркер (Celery), генерирующий текст в фоне, не изменяет этот флаг. Polling (эндпоинт /status) не изменяет этот флаг, он только сообщает фронтенду, что можно запросить текст через /view.

    Если polling возвращает статус FAILED или FAILED_VALIDATION, система должна вывести сообщение: «Не удалось сгенерировать толкование. Платеж сохранён. Попробуйте запросить толкование ещё раз» и показать кнопку «Повторить запрос». Если ошибка связана с модерацией (FAILED_VALIDATION), рядом с кнопкой «Повторить запрос» должна отображаться ссылка «Запросить возврат средств», ведущая на форму «Свяжитесь с нами» с предзаполненной темой «Возврат за неоказанную услугу».

    Важно: Если для данного расклада уже была успешная оплата, при нажатии на кнопку «Повторить запрос» фронтенд отправляет запрос на POST /api/spreads/{generated_spread_id}/interpretation. Бэкенд автоматически определит наличие оплаты и поставит задачу в очередь без повторного открытия окна ЮKassa.
21. Бесплатное толкование считается использованным только после успешного завершения генерации (generation_status = COMPLETED). До этого момента право удерживается резервом free_interpretation_pending = true.
    a. Если нейросеть выдала ошибку (FAILED или FAILED_VALIDATION), воркер выставляет free_interpretation_pending = false, free_interpretation_pending_since = NULL, а free_interpretation_used не меняет. Пользователь может запросить толкование повторно без оплаты.
    b. Если задача была переведена в CANCELED (например, по TTL или вручную), воркер/обработчик выполняет те же действия, что и в пункте (a).
    c. Если задача завершилась COMPLETED, воркер в одной транзакции выставляет free_interpretation_used = true, free_interpretation_pending = false, free_interpretation_pending_since = NULL.
22. Если пользователь сделал расклад, но не запросил толкование — бесплатное толкование не сгорает.

23. Все ошибки логируются в соответствии с разделом «Логирование и аудит».

Оплата толкования

1. Стоимость толкования

1. Актуальная цена должна браться из базы данных.

2. Инструкция по оплате

1. Перед инициацией платежа (на экране запроса толкования) система должна отображать блок с краткой инструкцией по оплате.
2. Инструкция должна содержать следующие обязательные элементы:
    a. Стоимость: четкое указание цены (например, «Стоимость толкования — 159 ₽»).
    b. Что оплачивается: пояснение, что оплата взимается за генерацию персонального AI-толкования для текущего расклада.
    c. Пошаговый процесс: краткое описание того, что произойдет после нажатия кнопки «Оплатить» (например: «1. Нажмите "Оплатить". 2. В новом окне введите данные карты. 3. Дождитесь подтверждения оплаты»).
    d. Блок юридического согласия: чекбокс с текстом: «Я ознакомлен(а) и согласен(на) с условиями [Публичной оферты] и [Правил возврата]». Ссылки должны вести на соответствующие страницы сайта.
    e. Важное предупреждение: текст, предупреждающий, что после оплаты страницу нельзя закрывать, так как на ней появится результат. Также необходимо указать: «Если вы закроете вкладку, результат будет автоматически отправлен на указанный вами email. Чек об оплате также придет на почту».
3. Инструкция должна содержать ссылку на страницу «Свяжитесь с нами» с пометкой: «Если у вас возникли проблемы с оплатой, напишите нам».
4. Текст инструкции должен быть написан простым языком, без использования технических терминов (избегать слов «вебхук», «транзакция», «эквайринг»).
5. На экране ожидания оплаты или в случае отмены платежа система должна дублировать краткую подсказку о том, как можно повторить попытку оплаты.

3. Сценарий оплаты

1. Оплата происходит в новом окне браузера, чтобы результат расклада оставался на экране.
2. Общий сценарий:
    a. Пользователь нажимает «Получить толкование» (если бесплатное толкование уже использовано).
    b. Система создаёт платёж через API ЮKassa и получает confirmation_url.
    c. Система открывает confirmation_url в новом окне.
    d. Результат расклада остается на текущей странице без изменений.

e. После нажатия на кнопку «Оплатить» и успешного создания платежа в ЮKassa (получения confirmation_url), кнопка должна блокироваться (становиться неактивной) и менять текст на «Ожидание оплаты...».
f. После успешной оплаты ЮKassa отправляет вебхук на сервер системы.
g. Система проверяет вебхук, обновляет статус платежа и запускает генерацию толкования.
h. После получения ответа от нейросети текст толкования добавляется под результатом расклада на исходной странице.
i. Если браузер пользователя блокирует открытие нового окна (popup blocker), фронтенд должен перехватить эту ошибку и вместо нового окна выполнить редирект на страницу оплаты ЮKassa в текущей вкладке. При этом результат расклада (снимок) уже сохранен в базе данных. После успешной оплаты пользователь вернется на сайт по return_url, где фронтенд восстановит экран результата и запустит polling статуса.

4. Создание платежа

1. При инициации платежа (создании запроса к API ЮKassa) бэкенд системы должен запрашивать актуальную цену из базы данных на текущий момент, а не использовать жестко зашитое в код значение.
2. Если администратор изменил цену в тот момент, пока пользователь находился на сайте, при нажатии кнопки «Оплатить» фронтенд передает в запросе ту сумму, которую видел пользователь (поле expected_amount). Бэкенд сверяет её с актуальной ценой в базе данных.
3. Если актуальная цена в БД не совпадает с expected_amount, платеж не создается. Бэкенд возвращает ошибку 409 CONFLICT с кодом PRICE_CHANGED и новой ценой в details.new_amount. Фронтенд отображает пользователю модальное окно: «Стоимость толкования изменилась и составляет [новая цена] ₽. Продолжить оплату?». При согласии пользователя фронтенд повторяет запрос с новой суммой. Если платеж был создан ранее, но не оплачен, система использует сумму, зафиксированную в объекте платежа при его создании (защита от рассинхронизации).
4. На странице оплаты и в инструкции цена должна подтягиваться динамически из настроек.
5. Перед инициацией создания платежа (при вызове POST /api/payments/create) система проверяет длину платной очереди с помощью механизма резервирования слотов.
    a. Бэкенд отправляет команду INCR queue_reservations:paid в Redis (счетчик активных резервов).
    b. Если полученное значение превышает MAX_PAID_QUEUE_LENGTH (20), система выполняет DECR (откат) и возвращает ошибку QUEUE_OVERFLOW (503). Текст на фронтенде: «Сейчас очень много желающих. Деньги не списаны. Попробуйте через 10-15 минут».
    c. Если резерв успешен, создается платеж в ЮKassa. Время жизни резерва — 15 минут.
    d. Важно: Если пользователь не завершил оплату за 15 минут (закрыл вкладку, отвлекся), резерв автоматически снимается фоновой задачей, а при попытке вернуться пользователь получит ошибку истечения сессии.

6. При инициации оплаты система отправляет POST-запрос на https://api.yookassa.ru/v3/payments.
7. Пример запроса:

```
json
{
  "amount": {
    "value": "159.00",
    "currency": "RUB"
  },
  "capture": true,
  "confirmation": {
    "type": "redirect",
    "return_url": "https://ваш-сайт.ru/payment/return?spread_id={generated_spread_id}"
  },
  "description": "Толкование расклада Таро",
  "metadata": {
    "generated_spread_id": "550e8400-e29b-41d4-a716-446655440000",
    "visitor_id": "123e4567-e89b-12d3-a456-426614174000"
  }
}
```

8. Ключевые параметры:
    a. amount — сумма платежа. Всегда дробное значение, разделитель — точка.
    b. capture: true — платеж автоматически переходит в статус succeeded после подтверждения.
    c. confirmation.type: redirect — пользователь перенаправляется на страницу оплаты ЮKassa. После оплаты ЮKassa вернёт пользователя на return_url.
    d. metadata — дополнительные данные для связки платежа с раскладом и пользователем.
9. Аутентификация: HTTP Basic Auth. В заголовке Authorization передается shopId (как имя пользователя) и секретный ключ (как пароль).
10. Ключ идемпотентности (Idempotence-Key): Обязательный заголовок.
    a. Фронтенд генерирует новую уникальную строку (UUIDv4) непосредственно перед каждым вызовом API создания платежа.
    b. Важно: При получении ошибки PRICE_CHANGED и повторной отправке запроса с новой суммой, фронтенд обязан сгенерировать новый Idempotence-Key. Использование старого ключа приведет к ошибке со стороны ЮKassa.
    c. ЮKassa обеспечивает идемпотентность в течение 24 часов (защита от случайных дублей при обрыве сети).

Ответ: ЮKassa возвращает объект платежа с id, status (начинается с pending) и confirmation_url. Система сохраняет payment.id в базе данных вместе с generated_spread_id, visitor_id и email (переданным пользователем для чека и доставки результата). visitor_id определяется бэкендом только из подписанного visitor_token (HttpOnly cookie). Перед созданием платежа бэкенд загружает generated_spreads по generated_spread_id и проверяет, что generated_spreads.visitor_id совпадает с visitor_id из токена. При несовпадении

возвращается 404 NOT_FOUND без раскрытия информации о существовании расклада.
В metadata платежа ЮKassa передаются generated_spread_id и visitor_id. Поле spread_id в платежах не используется.

11. Сумма платежа фиксируется в момент вызова API ЮKassa (POST /api/payments/create). Если администратор изменил цену в базе данных после того, как пользователь открыл экран оплаты, но до нажатия кнопки "Оплатить", система спишет ту сумму, которая была актуальна на момент нажатия кнопки. В случае изменения цены на странице оплаты, фронтенд должен запросить актуальную цену перед вызовом API.

5. Статусы платежа

Платеж последовательно переходит между статусами:

Статус	Описание
pending	Платеж создан, ожидает подтверждения от пользователя
waiting_for_capture	Платеж подтвержден, но еще не завершен (для платежей с предавторизацией)
succeeded	Платеж успешно завершен. Финальный статус
canceled	Платеж отменен или не прошёл из-за ошибки

Примечание по статусу waiting_for_capture: В проекте используется одностадийная оплата (capture: true), поэтому платежи сразу переходят из pending в succeeded (или canceled). Статус waiting_for_capture в локальной базе данных (payments.status) не используется. Если в вебхуке приходит событие со статусом waiting_for_capture, бэкенд должен зафиксировать его в логах как WARN event_type: payment.unexpected_status и проигнорировать изменение локального статуса, оставив его в pending.

6. Настройка вебхуков

1. Вебхуки (HTTP-уведомления) позволяют автоматически отслеживать изменение статуса платежа. Настройка производится в личном кабинете ЮKassa в разделе «Интеграция — HTTP-уведомления».
2. Требования к URL:
    a. Протокол HTTPS (обязательно).
    b. Поддерживается TLS 1.2 или выше.
    c. URL — это адрес на вашем сервере. Пример для нашего проекта: https://your-site.ru/api/webhooks/yookassa.
3. События для подписки:
    a. payment.succeeded — платеж прошёл успешно.
    b. payment.canceled — платеж отменен или не прошёл.

    c. refund.succeeded — возврат средств успешно проведен (если возврат инициируется через API, либо если вы включили уведомления о возвратах в личном кабинете).
4. В личном кабинете ЮKassa в настройках HTTP-уведомлений должна быть включена опция проверки подписи (MD5-хэш). Секретное слово, заданное в личном кабинете для формирования подписи, должно быть передано разработчику и сохранено в переменных окружения (.env) для проверки запросов на стороне сервера.

7. Обработка вебхука на сервере

При получении вебхука система должна выполнить следующие шаги:

1. Обязательно проверить подлинности запроса. Система должна отклонять запросы (возвращать HTTP 403), если они не прошли две проверки:
    a. Проверка IP-адреса: IP-адрес отправителя должен входить в актуальный allowlist официальных IP-адресов ЮKassa (список запрашивается у менеджера и хранится в конфигурации).
    b. Проверка подписи (MD5): Система должна проверить параметр md5 в теле запроса. Значение md5 формируется как MD5-хэш от строки, собранной из значений параметров запроса вместе с секретным словом (указывается в настройках личного кабинета ЮKassa). Если хэш не совпадает с расчетным, запрос отклоняется.

   Примечание: Проверка MD5-подписи является официальным требованием ЮKassa для HTTP-уведомлений. Несмотря на то, что MD5 считается устаревшим алгоритмом, его использование обязательно в данном контексте. Основным рубежом защиты является строгая проверка IP-адресов (allowlist).
2. Записать событие в журнал. Даже если платеж уже обработан, каждое входящее событие должно быть зафиксировано. Если проверка IP-адреса или MD5-подписи (из п. 1) не пройдена, система должна записать в логи событие уровня WARN с event_type: payment.webhook_security_failed и указанием source_ip отправителя. После логирования обработка вебхука немедленно прерывается, а сервер возвращает HTTP-код 403 (Forbidden).
3. Проверить идемпотентность. Вебхук может прийти повторно. Если платёж с таким payment.id уже обработан — не выполнять действия повторно.
4. Проверить статус платежа. Извлечь payment.id из тела вебхука, найти соответствующий платёж в базе данных и сравнить статусы. Если локальный статус уже succeeded — ничего не делать.
5. Валидировать данные. Проверить сумму, валюту и metadata (например, generated_spread_id и visitor_id) на соответствие ожидаемым значениям.
6. Обязательная перепроверить через API. После успешной проверки подписи и IP-адреса система должна отправить GET-запрос к API ЮKassa (/v3/payments/{payment_id}), чтобы получить актуальный статус платежа. Источником истины для обновления статуса в БД является ответ API, а не тело вебхука. Это защищает от подделки данных внутри самого запроса.
7. Обновить локальный статус. Если платеж подтвержден (succeeded) — обновить запись в базе данных.
   Дополнительно: если по generated_spread_id уже есть активная бесплатная задача (is_free_request = true и generation_status IN

('QUEUED','PROCESSING','RETRY_WAIT')) или уже сохранён interpretation_text, дубль-задача не создаётся. В payments.possible_duplicate_payment выставляется true, в лог пишется событие уровня WARN с event_type = payment.possible_duplicate_free_race. Администратор увидит флаг в списке платежей и сможет оформить возврат через ЛК ЮKassa вручную.

8. При успешном обновлении статуса платежа на succeeded система должна выполнить DECR queue_reservations:paid в Redis, так как задача переходит из статуса «резерва» в статус реальной задачи в очереди Celery.
   Примечание: Если оплата пришла после истечения 15-минутного резерва (фоновый клинер уже вернул счетчик в исходное состояние), система все равно принимает платеж и ставит задачу в очередь (так как деньги списаны), но в лог пишется событие CRITICAL с event_type = queue.overshoot_payment для оповещения администратора о превышении лимита.
9. Защита от дублирования задач (Race Condition): Перед созданием события в outbox_events система обязана проверить текущий статус снимка generated_spreads (с блокировкой SELECT ... FOR UPDATE).
    a. Если interpretation_text уже заполнен (текст сгенерирован) ИЛИ generation_status IN ('QUEUED', 'PROCESSING', 'RETRY_WAIT', 'COMPLETED', 'FAILED_VALIDATION'):
        i. Система обновляет статус платежа в таблице payments на succeeded (так как деньги реально списаны).
        ii. Система выставляет флаг possible_duplicate_payment = true в таблице payments.
        iii. Система НЕ создает новую запись в outbox_events и не ставит задачу воркеру.
        iv. В лог пишется событие уровня WARN с event_type: payment.duplicate_detected и указанием payment_id и generated_spread_id.
    b. Во всех остальных случаях (статус AWAITING_PAYMENT или PAYMENT_PENDING): логика стандартная — обновление статуса, запись в outbox_events.
10. Логика обновления статуса платежа при вызове эндпоинта /api/payments/verify должна быть вынесена в общую сервисную функцию (или метод), которая также используется обработчиком вебхуков. Это гарантирует, что при одновременном приходе вебхука и запроса от фронтенда (гонка), транзакционная блокировка SELECT FOR UPDATE пропустит только один запрос, а второй просто вернет текущий статус, не создавая дубль задачи в очереди.
11. Обеспечение согласованности (Transactional Outbox). Обновление статуса платежа и постановка задачи на генерацию должны выполняться через паттерн Transactional Outbox. В рамках одной транзакции PostgreSQL система должна:
    a. Заблокировать строку платежа (SELECT FOR UPDATE) и обновить его статус на succeeded.
    b. Обновить статус в таблице generated_spreads на QUEUED.
    c. Вставить новую запись в таблицу outbox_events с типом события generate_interpretation и payload, содержащим generated_spread_id.
    d. Подтвердить транзакцию (COMMIT).

Примечание: Проверка лимитов очереди (MAX_PAID_QUEUE_LENGTH) на этом шаге не выполняется. Если очередь достигла значения MAX_HARD_QUEUE_LIMIT, задача все равно попадает в outbox, а в лог пишется событие CRITICAL для оповещения администратора.

12. После успешного коммита транзакции диспетчер Outbox (фоновая задача app.tasks.process_outbox, запускаемая каждые 5 секунд) забирает события из таблицы outbox_events и идемпотентно публикует их в очередь llm_tasks в Redis. Алгоритм работы диспетчера (гарантия At-least-once):
    a. Диспетчер выбирает пачку событий со статусом PENDING (например, 50 штук), у которых next_retry_at <= now() (или NULL), с блокировкой SELECT ... FOR UPDATE SKIP LOCKED, чтобы несколько параллельных процессов не взяли одно и то же событие.
    b. Для каждого события вызывается метод Celery (apply_async).
    c. Если Celery возвращает task_id (успешная запись в Redis), диспетчер обновляет статус события на PROCESSED, записывает processed_at = now() и сохраняет celery_task_id.
    d. Если при отправке возникает ошибка (Redis недоступен), диспетчер увеличивает retry_count на 1, записывает ошибку в last_error и вычисляет next_retry_at с использованием экспоненциальной задержки (1 мин, 5 мин, 15 мин, 1 час). Статус остается PENDING.
    e. Если retry_count превышает 15 (или если с момента создания события прошло более 24 часов), событие переводится в статус FAILED для ручного разбора администратором.
    f. Восстановление FAILED событий: Администратор через админ-панель может сбросить статус события на PENDING и обнулить retry_count, чтобы диспетчер Outbox повторил попытку отправки после восстановления Redis. Автоматически события из статуса FAILED не восстанавливаются, чтобы не создавать бесконечный цикл ошибок при долгом сбое инфраструктуры.
    g. Прямая отправка задач в Redis из обработчика вебхука или API запрещена. Отправкой занимается только диспетчер Outbox.
13. Вебхук не выполняет запросы к нейросети и не сохраняет текст толкования. Его единственная задача — зафиксировать факт оплаты, обновить статусы в БД (через outbox_events) и вернуть HTTP-ответ ЮKassa.
14. Подтвердить получение вебхука. Вернуть HTTP-статус 200 OK. Если сервер не ответит в течение 5 секунд, ЮKassa повторит попытку.
15. Если от ЮKassa приходит вебхук payment.canceled или пользователь возвращается на сайт по кнопке «Отмена», кнопка оплаты должна снова стать активной.
16. При получении от ЮKassa уведомления payment.canceled система должна найти платёж по payment_id и перевести его локальный статус в canceled. Также система должна выполнить DECR queue_reservations:paid в Redis, освобождая зарезервированный слот. При этом статус снимка generated_spreads.generation_status остается PAYMENT_PENDING, чтобы пользователь мог инициировать новую попытку оплаты для этого же расклада (создав новый объект платежа).

Если локальный статус платежа уже был изменён на canceled (например, при повторной доставке вебхука), система должна проигнорировать повторное событие (идемпотентность) и вернуть HTTP 200.

17. При переходе платежа в статус canceled (как по вебхуку, так и при возврате пользователя по return_url) фронтенд должен разблокировать кнопку «Оплатить», вернуть ее в исходное состояние и отобразить сообщение: «Оплата не была завершена. Вы можете попробовать снова». При нажатии на кнопку «Оплатить» фронтенд инициирует новый запрос на POST /api/payments/create (с новым Idempotence-Key), создавая новый объект платежа для того же снимка расклада.
    Если у платежа был выставлен флаг possible_duplicate_payment = true, система должна сбросить его в false (так как возврат средств уже произведен, и инцидент исчерпан), чтобы платеж перестал отображаться в списке «Требуют внимания».
    Важно: Обновление статуса платежа на refunded НЕ влияет на поле generated_spreads.generation_status. Снимок расклада остается в том статусе, в котором он находился (например, COMPLETED, если текст уже был получен, или FAILED_VALIDATION, если услуга не была оказана). Возврат средств является исключительно финансовой операцией.
18. При получении вебхука refund.succeeded система должна найти платеж по payment_id, обновить его локальный статус на refunded и записать событие в лог (event_type = payment.refunded). Бэкенд не инициирует задачи на аннулирование чеков — этим занимается отдельный сервис yookassa-to-mynalog.

8. Повторные платежи

1. Если пользователь оплатил толкование, но оно не сгенерировалось из-за ошибки нейросети — платёж сохраняется, и пользователь может повторить запрос без повторной оплаты.
2. Реализация:
    a. В таблице payments хранится запись со статусом succeeded, привязанная к generated_spread_id и visitor_id.
    b. При повторном запросе толкования система проверяет: есть ли успешный платеж для этого расклада и пользователя.
    c. Если есть — запрос к нейросети отправляется без нового платежа.
    d. Если нет — открывается страница оплаты.
3. Если пользователь нажимает кнопку «Повторить запрос» для задачи, находящейся в статусе FAILED или FAILED_VALIDATION, система должна перевести ее статус в QUEUED и отправить задачу в очередь Redis (без создания нового платежа). Фронтенд не вызывает метод /api/payments/create, а сразу обращается к /interpretation.
4. При нажатии пользователем кнопки «Повторить запрос» для задачи, находящейся в статусе FAILED или FAILED_VALIDATION, система должна перевести ее статус в QUEUED и отправить задачу в очередь Redis (без создания нового платежа). При этом статус самого платежа в БД остается succeeded.

9. Возвраты

1. В MVP автоматический возврат средств через API нашего сайта не предусмотрен. Все возвраты осуществляются вручную администратором через Личный кабинет ЮKassa. Это обеспечивает финансовую безопасность и требует осознанного подтверждения человеком.
2. Правила возврата для цифровой услуги:
    a. Если AI-толкование уже было успешно сгенерировано (статус COMPLETED и текст доступен пользователю), услуга считается оказанной в полном объеме. В этом случае возврат средств, как правило, не производится (за исключением случаев технического сбоя или двойного списания).
    b. Если услуга не была оказана (статус задачи FAILED или FAILED_VALIDATION), и пользователь не может получить результат, администратор обязан оформить возврат средств по запросу пользователя.
    c. При случайном двойном списании (например, пользователь оплатил один и тот же расклад дважды) администратор обязан вернуть средства за дублирующий платеж. Факт дублирования автоматически фиксируется системой (флаг possible_duplicate_payment) и отображается в админ-панели.
    d. Если задача завершилась с ошибкой FAILED_VALIDATION по причине срабатывания модерации контента (нарушение маркировки 16+ или стоп-слова), администратор обязан оформить полный возврат средств по запросу пользователя, так как система не смогла предоставить услугу надлежащего качества. Альтернативно, администратор может предложить пользователю бесплатно изменить контекст запроса и повторить генерацию.
3. Процесс возврата:
    a. Пользователь обращается в техподдержку с запросом на возврат (через форму «Свяжитесь с нами»).
    b. Администратор проверяет обоснованность запроса. Если AI-толкование уже было успешно сгенерировано (статус COMPLETED), администратор вправе отказать в возврате (см. раздел «Просмотр обращений»).
    c. При одобрении администратор заходит в ЛК ЮKassa и инициирует возврат средств по конкретному платежу;
    d. Для платежей с флагом possible_duplicate_payment = true (дублирующие оплаты):
        i. Администратор переходит в раздел «Платежи», ставит фильтр «Только дублирующие оплаты».
        ii. Открывает карточку платежа, нажимает ссылку «Перейти в ЛК ЮKassa» и оформляет возврат вручную.
        iii. После успешного возврата ЮKassa пришлет вебхук refund.succeeded, и система автоматически снимет флаг possible_duplicate_payment (согласно п. 16.1).
4. Синхронизация статусов:
    a. После проведения возврата в ЮKassa, ЮKassa отправляет вебхук refund.succeeded на наш сервер.

b. Наш бэкенд переводит локальный статус платежа в payments.status = refunded и пишет событие в лог (payment.refunded).
    i. Если у платежа был выставлен флаг possible_duplicate_payment = true, система сбрасывает его в false (инцидент с дублирующей оплатой исчерпан).
    ii. Статус самого снимка расклада (generated_spreads.generation_status) при этом НЕ меняется. Он остается в том состоянии, в котором находился до возврата (например, COMPLETED, если текст был сгенерирован, или FAILED_VALIDATION). Возврат средств — это исключительно финансовая операция, которая не отменяет факт генерации контента.
c. Сервис yookassa-to-mynalog (работающий независимо) получает этот же вебхук (или находит возврат при синхронизации) и автоматически аннулирует фискальный чек в «Мой Налог».
d. Наш бэкенд не отправляет прямых запросов на аннулирование чеков, полагаясь на yookassa-to-mynalog.
e. Если автоматическое аннулирование чека завершилось ошибкой (сервис недоступен, ФНС вернула ошибку), администратор увидит статус ошибки в карточке платежа и может воспользоваться аварийным эндпоинтом повторного аннулирования (см. Спецификацию API).

10. Автоматическая генерация чеков для самозанятых

1. Система должна автоматически формировать чеки для самозанятого при каждом успешном платеже через ЮKassa.
2. При автоматическом формировании чека через yookassa-to-mynalog система должна обеспечивать наличие следующих обязательных реквизитов, установленных ст. 14 422-ФЗ: дата и время расчета; ФИО и ИНН самозанятого; наименование оказанной услуги; сумма расчета; указание на применение специального налогового режима «Налог на профессиональный доход»; уникальный идентификационный номер чека.
3. В поле «Наименование услуги» должна передаваться строка «Толкование расклада Таро». Использование общих формулировок (например, просто «Услуга» или «Консультация») не допускается.
4. Если при регистрации чека в «Мой Налог» возникает ошибка (например, недоступность сервиса, неверные реквизиты), система должна:
    a. Логировать событие event_type: tax.receipt_failed с указанием кода ошибки;
    b. Отображать уведомление администратору в панели управления (бейдж с ошибкой);
    c. Автоматически повторять попытку отправки чека с экспоненциальной задержкой (до 5 попыток).
5. Сервис yookassa-to-mynalog при создании чека использует email пользователя, указанный при запросе толкования. Так как поле email является обязательным для инициализации платежа, чек всегда автоматически уходит клиенту на почту (требование 54-ФЗ/422-ФЗ).

6. Для автоматизации используется open-source решение yookassa-to-mynalog — сервис синхронизации платежей из ЮKassa в «Мой Налог».
7. Сервис yookassa-to-mynalog разворачивается как отдельный Docker-контейнер и работает независимо от основного бэкенда сайта.
8. Сервис должен поддерживать два способа авторизации в «Мой Налог»:
    a. логин + пароль;
    b. refresh token (для входа через Госуслуги).
9. Обеспечение сохранности данных (SQLite):
    a. В docker-compose.yml для контейнера yookassa-to-mynalog должен быть настроен именованный Docker Volume (например, yookassa_data), который монтируется в директорию с базой данных SQLite. Это предотвращает потерю состояния сервиса при перезапуске контейнера.
    b. Раз в сутки (через системный cron или Celery Beat) выполняется резервное копирование файла базы данных SQLite в S3-совместимое хранилище (в отдельную директорию /backups/yookassa/).
10. Идемпотентность: Сервис yookassa-to-mynalog использует payment_id (или yookassa_id) в качестве уникального ключа. Повторная обработка одного и того же платежа не приводит к созданию дублирующего чека.
11. Сервис должен автоматически:
    a. обнаруживать новые успешные платежи в ЮKassa;
    b. создавать чеки в «Мой Налог» с суммой и наименованием услуги;
    c. регистрировать чеки в ФНС;
    d. аннулировать чеки при возвратах (для этого в конфигурации сервиса REFUNDS_ENABLED должен быть установлен в true, а в вебхуках ЮKassa — включено событие refund.succeeded);
    e. защищать от дублирования чеков;
    f. сохранять состояние синхронизации в SQLite (с бэкапом согласно п. 9).
12. Сервис должен выполнять валидацию конфигурации при старте и автоматические повторы при сбоях сети.
13. Учетные данные для доступа к ЮKassa и «Мой Налог» должны храниться в переменных окружения (.env) и не попадать в код или систему контроля версий. Обязательные переменные окружения для yookassa-to-mynalog:
    a. YOOKASSA_SHOP_ID, YOOKASSA_SECRET_KEY (авторизация в ЮKassa);
    b. TAX_LOGIN, TAX_PASSWORD или TAX_REFRESH_TOKEN (авторизация в «Мой Налог»);
    c. TAXPAYER_NAME (ФИО самозанятого, например: «Иванов Иван Иванович»);
    d. TAXPAYER_INN (ИНН самозанятого, 12 цифр);
    e. TAX_SERVICE_NAME (наименование услуги, по умолчанию: «Толкование расклада Таро»).
14. Сервис yookassa-to-mynalog должен выполнять валидацию конфигурации при старте. Если хотя бы одна из обязательных переменных (ФИО, ИНН, ключи доступа) отсутствует или пуста, контейнер должен аварийно завершить работу (fail-fast) с записью в лог ошибки уровня CRITICAL.
15. Для развёртывания сервиса требуются:
    a. Linux-сервер;
    b. Docker и Docker Compose;

    c. учётные данные ЮKassa (shopId + секретный ключ);
    d. учётные данные «Мой Налог» (логин + пароль или refresh token).
16. Администратор должен иметь доступ к логам сервиса yookassa-to-mynalog для диагностики проблем с созданием чеков.
17. Статус not_required выставляется в случаях, когда чек не требуется по законодательству (например, при нулевой сумме платежа, возврате средств до момента фискализации или при тестовых платежах).

11. Сверка платежей

1. Система должна запускать фоновую задачу app.tasks.reconcile_payments (например, через Celery Beat) в очереди payment_tasks не реже одного раза в 15 минут. Задача должна находить в локальной базе данных все платежи со статусом pending, которые были созданы более 10 минут назад, и для каждого из них отправлять GET-запрос к API ЮKassa (/v3/payments/{payment_id}) для получения актуального статуса.
2. Если API ЮKassa возвращает статус succeeded, а локальный статус платежа — pending, система должна обработать этот платеж как успешный: обновить локальный статус, записать задачу в outbox_events и инициировать отправку email. Если API возвращает canceled, система должна обновить локальный статус на canceled и разблокировать интерфейс (если пользователь еще на сайте).
3. Фоновая задача сверки должна использовать ту же транзакционную логику с блокировкой строки, что и обработчик вебхука. Если сверка обнаружила, что статус в ЮKassa стал succeeded, а локальный все еще pending, она обновляет его и ставит задачу в очередь. Если локальный статус уже succeeded (вебхук успел раньше), обновление игнорируется.

Интеграция с нейросетью (YandexGPT)

1. Параметры интеграции и авторизация

1. Система должна использовать API Yandex Cloud Foundation Models (YandexGPT) для генерации текстов толкований.
2. В качестве основной модели на старте должна использоваться YandexGPT Lite.
3. Система должна поддерживать возможность смены модели (например, на YandexGPT Pro) через изменение переменных окружения без внесения правок в исходный код.
4. Аутентификация запросов к API должна осуществляться с использованием API-ключа сервисного аккаунта Yandex Cloud.
5. API-ключ и идентификатор каталога (folder_id) должны храниться в переменных окружения (.env) и не должны попадать в репозиторий с кодом.
6. В каждом запросе к API система должна передавать заголовки авторизации (Authorization: Api-Key <ключ>) и идентификатор каталога (x-folder-id).

2. Формирование запроса к нейросети

1. Система должна отправлять POST-запрос на эндпоинт YandexGPT API в формате JSON для генерации толкования.
2. В теле запроса должен передаваться объект completionOptions со следующими параметрами: stream: false (синхронный режим), temperature: 0.3 (для снижения вариативности и галлюцинаций), maxTokens: 2000 (максимальная длина ответа).
3. В массиве messages система должна передавать два сообщения: сообщение с ролью "system", содержащее мастер-промпт (системную инструкцию), и сообщение с ролью "user", содержащее данные конкретного расклада (текст расклада, выпавшие карты, контекст пользователя).
4. Система должна брать данные о раскладе (название, позиции, описания) из сохраненного снимка, а не из актуального шаблона расклада в админке. Это гарантирует, что нейросеть получит именно те данные, которые видел пользователь.
5. Перед отправкой запроса система должна валидировать общую длину промпта (промпт + контекст). Если длина превышает лимит контекстного окна модели, система должна обрезать избыточный контекст или вернуть ошибку администратору/пользователю.
6. Система не должна передавать в нейросеть персональные данные пользователя (ФИО, телефоны), если они не были явно указаны пользователем в поле «Контекст/Вопрос».
7. Бюджетирование токенов (Token Budgeting). Для предотвращения переполнения контекстного окна модели (лимит YandexGPT Lite — 8 000 токенов, из которых 2 000 зарезервированы под ответ, лимит промпта — 6 000 токенов), воркер перед отправкой запроса обязан подсчитать длину промпта и применить следующий алгоритм (Fallback):
    a. Шаг 1: Если суммарная длина (системная инструкция + снимок + контекст пользователя) превышает 6 000 токенов, система принудительно обрезает поле user_context (контекст/вопрос пользователя), оставляя только первые 1 500 символов.
    b. Шаг 2: Если после обрезки контекста длина всё ещё превышает 6 000 токенов (например, из-за объемного снимка расклада), система модифицирует cards_snapshot перед подстановкой в промпт: удаляет поля full_description у всех карт, оставляя только name, suit, position и short_description.
    c. Шаг 3: Если после Шага 2 длина всё ещё превышает 6 000 токенов, воркер прерывает выполнение задачи, не отправляя запрос в YandexGPT. Задача переводится в статус FAILED_VALIDATION, в лог пишется событие уровня ERROR с event_type = llm.context_overflow, а пользователю на фронтенде выводится сообщение: «Не удалось обработать запрос из-за слишком большого объема данных. Пожалуйста, сократите текст вопроса».

3. Обработка нагрузки и очередей

1. Учитывая ограничение YandexGPT Lite в 1 одновременный поток, система должна использовать очередь задач (например, на базе Redis/Celery или асинхронный подход) для обработки запросов.
2. Если в момент запроса нейросеть занята обработкой предыдущего запроса, новый запрос должен ставиться в очередь, а пользователю на фронтенде должен отображаться индикатор загрузки. Если очередь пуста, текст гласит: «Аметистовая жаба анализирует ваш расклад, это займет около 30–60 секунд». Если в очереди есть задачи, текст меняется на: «Сейчас высокая нагрузка. Ожидаемое время генерации — до X минут. Пожалуйста, не закрывайте страницу».
3. Бэкенд рассчитывает ожидаемое время (ETA) по формуле: ETA = Количество задач в очереди * Среднее время генерации (30 сек).
4. Приоритизация задач:
    a. Задачи с высоким приоритетом (priority = 9): запросы, которые уже оплачены, или бесплатные запросы, если очередь пуста.
    b. Задачи с низким приоритетом (priority = 1): бесплатные запросы, если в очереди уже есть платные задачи.
    c. Воркер (concurrency=1) всегда забирает задачу с наивысшим приоритетом.
5. Система должна устанавливать таймаут ожидания ответа от нейросети (например, 60 секунд). Если ответ не получен, система должна прервать ожидание и обработать это как ошибку.
6. Система должна иметь настраиваемые параметры лимитов очереди (через переменные окружения):
    a. MAX_FREE_QUEUE_LENGTH — максимальное количество бесплатных задач в очереди (по умолчанию 10);
    b. MAX_PAID_QUEUE_LENGTH — максимальное количество платных задач в очереди (по умолчанию 20);
    c. MAX_HARD_QUEUE_LIMIT — абсолютный предел длины очереди (сумма бесплатных и платных), защищающий Redis и воркеры от переполнения (по умолчанию 100).
    (Примечание: При достижении этого предела время ожидания для последнего пользователя может достигать 50 минут. Это осознанное бизнес-решение для максимизации выручки в пиковые моменты).
7. Логика для бесплатных запросов: При вызове эндпоинта /interpretation (для бесплатного запроса) бэкенд проверяет текущую длину бесплатных задач в очереди llm_tasks. Если она превышает MAX_FREE_QUEUE_LENGTH, эндпоинт возвращает ошибку QUEUE_OVERFLOW (503) с сообщением: «Сейчас очень много желающих получить толкование. Пожалуйста, попробуйте через 10–15 минут. Деньги не списаны».
8. Логика для платных запросов (до оплаты): Перед созданием платежа в ЮKassa (эндпоинт /api/payments/create) бэкенд проверяет текущую длину платных задач в очереди. Если она превышает MAX_PAID_QUEUE_LENGTH, система не создает платеж и возвращает ошибку QUEUE_OVERFLOW (503) с сообщением: «Сейчас очень много желающих получить толкование. Пожалуйста, попробуйте через 10–15 минут. Деньги не списаны».

9. Логика для платных запросов (после оплаты): Платные запросы (после успешного вебхука) всегда принимаются в очередь, даже если MAX_PAID_QUEUE_LENGTH переполнен, так как деньги уже списаны. Единственное исключение — достижение абсолютного предела MAX_HARD_QUEUE_LIMIT. В этом случае задача все равно ставится в очередь (обязательство перед клиентом), но система пишет в лог событие уровня CRITICAL (event_type: queue.hard_limit_reached) и отправляет уведомление администратору о необходимости срочного масштабирования мощностей.
10. Бэкенд должен иметь возможность в реальном времени запрашивать текущую длину очереди llm_tasks из Redis (например, через команду LLEN с учетом приоритетов). При расчете ETA для платных запросов бэкенд должен учитывать не только реальные задачи в очереди, но и активные резервы (счетчик queue_reservations:paid), так как они гарантированно превратятся в задачи в ближайшие 15 минут.
11. Бэкенд рассчитывает ожидаемое время (ETA) для конкретного пользователя. Так как воркер обрабатывает строго одну задачу за раз (concurrency=1), время ожидания зависит от общего количества задач в очереди, стоящих перед пользователем, независимо от их приоритета.
    Формула: ETA = (Количество задач в очереди передо мной) * 30 сек.
    Пример 1: Если пользователь делает платный запрос, а в очереди уже 5 платных и 10 бесплатных задач, ETA = (5 + 10) * 30 = 450 секунд (7.5 минут).
    Пример 2 (Аварийный режим): Если очередь достигает абсолютного предела MAX_HARD_QUEUE_LIMIT (100 задач), максимальное время ожидания для последнего в очереди составит 100 * 30 = 3000 секунд (50 минут). При достижении этого предела новые запросы и платежи блокируются. При 80% заполнения очереди (80 задач) в лог пишется событие WARN, при 100% — CRITICAL.
12. Фронтенд, находящийся в режиме ожидания результата (polling), при получении информации о высокой нагрузке на сервер (флаг is_high_load: true) должен скорректировать текст экрана ожидания. Вместо «это займет до 60 секунд» система выводит: «Из-за аномально высокой нагрузки ожидание может занять до 50 минут. Пожалуйста, не закрывайте страницу. Результат также придет на указанный email».
    (Если estimated_wait_seconds превышает 600 секунд, рекомендуется дополнительно выводить совет: «Вы можете закрыть эту вкладку, результат придет на почту»).

4. Работа воркера

1. Воркер должен быть настроен на обработку строго одной задачи одновременно (concurrency=1), чтобы не превышать лимит YandexGPT Lite.
2. Политика повторных попыток (Retry) должна быть строго зафиксирована в конфигурации приложения (переменные окружения) и применяться в зависимости от типа ошибки:
    a. Ошибки 429 (Too Many Requests) и 503 (Service Unavailable): До 5 попыток с экспоненциальной задержкой. Если API передает заголовок Retry-After, пауза берется из него.

    b. Таймаут (превышение 60 секунд): 1 повторная попытка (итого 2 попытки).
    c. Ошибки 400, 401, 403 (Bad Request, Auth): 0 попыток. Задача сразу помечается как FAILED.
    d. Ошибки валидации контента (бред нейросети): До 2 повторных генераций (итого 3 генерации).
    e. Ошибка переполнения контекста (LLM_CONTEXT_OVERFLOW): 0 попыток. Задача сразу помечается как FAILED_VALIDATION, так как повторный запрос с теми же данными приведет к тому же результату.
3. Воркер является единственным компонентом системы, который имеет право записывать текст сгенерированного толкования в поле interpretation_text таблицы generated_spreads. Ни обработчик вебхуков, ни эндпоинты API не выполняют запись этого поля.
4. После успешного получения ответа воркер обновляет статус задачи на COMPLETED, сохраняет текст толкования и инициирует отправку письма. Если у задачи флаг is_free_request = true, воркер в той же транзакции обновляет anonymous_visitors для соответствующего visitor_id:
    a. free_interpretation_used = true;
    b. free_interpretation_pending = false;
    c. free_interpretation_pending_since = NULL.

   При переходе задачи в FAILED или FAILED_VALIDATION воркер в отдельной транзакции выставляет free_interpretation_pending = false, free_interpretation_pending_since = NULL, а free_interpretation_used оставляет без изменений.
5. Фоновый воркер не должен зависеть от статуса активности браузера пользователя. Если пользователь закрыл вкладку или потерял соединение, воркер обязан завершить обработку задачи, получить ответ от YandexGPT и сохранить готовый текст толкования в базе данных, привязав его к visitor_id и generated_spread_id.
6. Воркер должен быть спроектирован так, чтобы повторное выполнение одной и той же задачи с теми же аргументами не приводило к дублированию данных или повторным действиям.
7. Перед вызовом нейросети и перед сохранением результата толкования в базу данных воркер должен проверять, не был ли этот результат уже сохранён для данного generated_spread_id (поле interpretation_text не пустое). Если результат уже существует, задача считается выполненной (COMPLETED), а повторный запрос к YandexGPT не выполняется. Это защищает от дублей, которые могут возникнуть из-за механизма At-least-once в Outbox.
8. При постановке задачи в очередь (перед вызовом task.delay() или task.apply_async()) бэкенд должен использовать механизм распределённой блокировки на базе Redis (например, через SET NX). Ключ блокировки должен формироваться на основе уникального идентификатора расклада (generated_spread_id).
9. В настройках Celery необходимо включить task_acks_late = True и task_reject_on_worker_lost = True. Это гарантирует, что сообщение о задаче не будет удалено из очереди до тех пор, пока воркер не завершит её выполнение. Если воркер упадёт, задача будет возвращена в очередь и передана другому воркеру.

10. При любом обновлении email пользователя (включая повторный запрос толкования для существующего generated_spread_id) система обязана обновить поле email_delete_after на now() + interval '12 months', чтобы продлить срок хранения в соответствии с 152-ФЗ.

5. Обработка ответа

1. При успешном ответе API система должна извлечь сгенерированный текст из поля result.alternatives[0].message.text.
2. После получения ответа от YandexGPT и до сохранения его в базу данных, бэкенд должен провести программную валидацию текста по следующим критериям (при провале любого из них — инкремент счетчика попыток):
    a. Длина: Текст должен содержать не менее 300 и не более 4000 символов.
    b. Язык: Текст должен быть написан на русском языке (проверка: доля кириллических символов должна составлять не менее 70% от всех буквенных символов).
    c. Стоп-слова (защита от утечки промпта и галлюцинаций): Текст не должен содержать фраз: «Как языковая модель», «Я не могу», «Вот ваш промпт», «Системная инструкция», «Я — искусственный интеллект».
    d. Семантическая релевантность: Текст ответа должен содержать названия (или синонимы) хотя бы 2 карт из cards_snapshot текущего расклада. Если нейросеть сгенерировала текст, не упомянув ни одной выпавшей карты, это считается галлюцинацией.
    e. Модерация 16+: Текст не должен содержать слов и выражений, относящихся к запрещенным темам (сцены насилия, суицид, наркотики, экстремизм). Список стоп-слов хранится в конфигурационном файле.
3. Если ответ не прошел валидацию (по любому из пунктов 2a–2e), задача не помечается как COMPLETED.
    a. Система автоматически отправляет повторный запрос к YandexGPT (с тем же промптом). Количество повторных попыток — не более 2 (итого максимум 3 генерации).
    b. Если после 3 попыток нейросеть продолжает выдавать невалидный текст, задача помечается как FAILED_VALIDATION.
    c. Пользователю выводится сообщение: «Не удалось сгенерировать корректное толкование (текст не прошел проверку безопасности 16+). Платеж сохранён. Вы можете изменить контекст вопроса и попробовать снова или запросить возврат средств через форму обратной связи.»
    d. Если задача упала с кодом llm.validation_failed_prohibited (нарушение 16+) или llm.validation_failed_stop_words, оплата за этот запрос считается неоказанной услугой. Администратор обязан оформить возврат средств по первому требованию пользователя (см. раздел «Возвраты»).
    e. Администратору в логи пишется событие уровня ERROR с указанием конкретной причины (llm.validation_failed_short, llm.validation_failed_language, llm.validation_failed_stop_words, llm.validation_failed_relevance, llm.validation_failed_prohibited) и сохранением «плохого» ответа нейросети для анализа. Важно: перед записью «плохого» ответа нейросети в лог, текст должен пройти обязательную процедуру маскирования персональных данных (PII)

согласно разделу «Безопасность и защита данных в логах». В логи не должны попадать имена, телефоны и email, которые нейросеть могла сгенерировать или скопировать из промпта.

4. Система должна сохранить полученный текст в базе данных, привязав его к идентификатору платежа и идентификатору расклада.
5. Система должна отобразить полученный текст толкования пользователю на странице с результатом расклада.
6. Система должна сохранить в базе данных информацию о количестве затраченных токенов (входных и выходных) для ведения статистики.

6. Отправка email

1. Система должна использовать внешний сервис для отправки транзакционных писем (например, Unisender Go или аналог) через SMTP или API.
2. Письмо с текстом толкования должно отправляться автоматически сразу после того, как воркер успешно сохранил результат в базе данных (статус completed), через задачу app.tasks.send_email в очередь email_tasks.
3. Письмо должно содержать:
    a. Название расклада;
    b. Список выпавших карт (название, положение);
    c. Текст сгенерированного AI-толкования;
    d. Уникальную безопасную ссылку на страницу с результатом (Signed URL): https://[домен]/result/{generated_spread_id}?token={result_access_token}. Ссылка генерируется воркером в момент успешного завершения задачи (COMPLETED) и действует не менее 30 дней.
4. Защита от спама. Форма ввода email должна быть защищена от автоматических ботов (honeypot-поле или капча), чтобы злоумышленники не могли использовать твой сервер для рассылки спама через твой почтовый сервис.
5. При получении от почтового сервиса Unisender Go ошибки доставки (например, невалидный адрес, переполненный ящик) воркер должен выполнить до 3 повторных попыток отправки с экспоненциальной задержкой. Если все попытки неудачны, система логирует событие event_type: email.delivery_failed, устанавливает в БД флаг email_delivered = false для данной задачи. При этом общий статус генерации толкования (generation_status) не меняется и остается COMPLETED, так как текст успешно сгенерирован и доступен пользователю на сайте.
   Перед отправкой письма воркер обязан проверить флаг email_delivered. Если он равен true, повторная отправка не производится (защита от дублей при повторных вызовах).

7. Обработка ошибок и сбоев

1. Если API возвращает ошибку (4xx, 5xx, превышение квот), система должна зафиксировать код ошибки и тело ответа в логах, отобразить пользователю сообщение («Не удалось сгенерировать толкование. Пожалуйста, попробуйте позже») и сохранить статус успешного платежа. При повторном нажатии

пользователем кнопки «Получить толкование» система должна отправить новый запрос к нейросети без повторного списания денег.

2. Если API возвращает ошибку 400 Bad Request из-за превышения длины промпта, либо если сработал механизм бюджетирования токенов (Шаг 3), система фиксирует код ошибки в логах и сохраняет статус FAILED_VALIDATION. Пользователю выводится сообщение: «Не удалось обработать запрос (превышен объем контекста). Оплата не будет списана (или будет возвращена)».
3. Если ответ от нейросети был получен, но не содержит ожидаемого поля с текстом (некорректный формат ответа), система должна обработать это как ошибку и залогировать некорректный ответ.
4. Если почтовый сервис возвращает ошибку (например, неверный email или сбой сервера), система должна логировать эту ошибку, но не должна менять статус выполнения задачи на failed (так как пользователь уже увидел результат на сайте).
5. Все ошибки интеграции должны логироваться в соответствии с разделом «Логирование и аудит» с указанием времени запроса, времени ответа и статуса.

8. Формирование мастер-промпта

1. Системная инструкция (роль и правила) должна храниться в базе данных или конфигурационном файле (не в коде) с возможностью редактирования администратором без перезапуска приложения.
2. Системная инструкция должна содержать следующие обязательные правила:
    a. роль («Ты — опытный таролог и психолог»);
    b. требование анализировать каждую карту отдельно и в связке;
    c. требование учитывать контекст вопроса, указанный пользователем (если он есть);
    d. запрет на вопросы и приглашения к диалогу в конце ответа;
    e. запрет на упоминание того, что ответ сгенерирован нейросетью;
    f. ограничение по объёму ответа (строго не более 4000 символов, чтобы уложиться в лимит токенов).
3. Данные расклада (пользовательская часть) должны формироваться динамически на основе:
    a. названия и описания расклада;
    b. списка выпавших карт с указанием позиций, названий, мастей, положения (прямое/перевёрнутое) и кратких описаний;
    c. контекста/вопроса пользователя (если он был указан).
4. Система должна подставлять динамические данные в шаблон пользовательской части промпта, сохраняя форматирование (нумерованный список позиций).
5. Пример итогового промпта, который система отправляет в нейросеть:

Системная инструкция	Ты — опытный таролог и психолог. Твоя задача — дать подробное, глубокое и человекочитаемое толкование расклада Таро. Ты не общаешься с пользователем напрямую, а пишешь текст, который бот или сайт передаст

	ему. Правила: 1. Анализируй каждую карту отдельно, учитывая её положение в раскладе и значение позиции. 2. Обязательно рассматривай карты в связке... 5. В конце ответа НЕ задавай вопросов... 7. Не упоминай, что ты нейросеть или ИИ. Говори от лица таролога.
Данные расклада (пользовательская часть):	Колода: Таро Райдера-Уэйта. Расклад: Кельтский крест. Описание расклада: Кельтский крест подходит для разбора ситуации... Дополнительная информация от пользователя: Пользователь спрашивает про отношения с партнером, они переживают затяжной кризис, и пользователь не знает, стоит ли продолжать эти отношения. Позиции и выпавшие карты (в порядке расклада): Настоящее: Иерофант (прямая) — Традиции, обучение, наставничество, конформизм. Вызов: Восьмерка Мечей (перевёрнутая) — Освобождение от ограничений, выход из тупика, обретение ясности. ... (остальные карты) Дай подробное толкование этого расклада, анализируя как отдельные карты, так и их сочетания. Учитывай описание расклада и контекст, указанный пользователем. Заверши толкование без вопросов и предложений продолжить диалог.

6. Система должна обеспечивать возможность администратору просматривать и редактировать текст системной инструкции через административную панель (для тонкой настройки качества ответов без участия разработчика).
7. Изменения в системной инструкции должны логироваться в соответствии с разделом «Логирование и аудит» (кто, когда и что изменил).

Раздел «Обратная связь»

1. При нажатии на пункт «Свяжитесь с нами» в выпадающем меню пользователь переходит на страницу с контактной информацией.
2. На странице отображаются:
    a. email техподдержки;
    b. форма обратной связи.
3. Форма обратной связи должна содержать следующие поля:
    a. Имя (опционально).
    b. Email для ответа (обязательно).
    c. Тема обращения (опционально).
    d. Текст сообщения (обязательно).
    e. Поле для указания Telegram-ника (опционально, если пользователь хочет связаться через мессенджер).

4. Страница «Свяжитесь с нами» также используется для запросов на возврат средств.
5. При отправке формы обратной связи система:
    a. сохраняет обращение в базе данных;
    b. логирует факт обращения;
    c. отображает пользователю подтверждение об отправке.
6. Администратор может просматривать обращения через административную панель.
7. При отправке формы обратной связи действует ограничение частоты запросов (Rate Limiting): не более 3 обращений за 10 минут с одного IP. При превышении лимита система возвращает ошибку 429 RATE_LIMITED, а фронтенд отображает сообщение: «Слишком много попыток отправки. Пожалуйста, подождите [X] минут и попробуйте снова».

Раздел “О проекте”

1. Страница «О проекте» должна быть доступна из выпадающего меню на главном экране.
2. Страница должна содержать текстовые блоки с информацией о концепции проекта, принципах работы, используемых технологиях (нейросеть) и правилах оплаты.
3. На странице должна быть размещена ссылка на документ «Политика конфиденциальности».
4. Контент страницы хранится в БД и редактируется через админ-панель; при отсутствии записей используются значения по умолчанию из шаблона.

Отображение изображений (общие правила для всех страниц)

1. При отображении любых списков или галерей карт (в просмотре значений, в результатах расклада, в админ-панели) система должна использовать «ленивую загрузку» (lazy loading). Картинки должны подгружаться только тогда, когда пользователь доскроллит до них (атрибут loading="lazy").
2. При отображении карты на экране (в раскладе или при одиночном вытягивании) фронтенд должен использовать версию medium.
3. Полноразмерная версия (original) должна загружаться только по клику на карту (если предусмотрен просмотр в модальном окне).

Логирование и аудит

1. Общие требования к системе логирования

1. Система должна использовать единый централизованный механизм логирования для всего сайта (серверной части), а не несколько разрозненных решений. Это необходимо для того, чтобы все события фиксировались в одном месте и по ним можно было проводить сквозной поиск.

2. Запись логов должна выполняться на стороне сервера, а не в браузере пользователя, чтобы исключить подмену данных.
3. Логи должны храниться в базе данных в виде реляционной таблицы (с отдельными колонками). При передаче логов во внешние системы или при просмотре через API данные сериализуются в формат JSON.
4. Система должна логировать как успешные, так и неуспешные события, если они имеют значение для безопасности или диагностики.

2. Категории событий, подлежащих логированию

1. Аутентификация, доступ и безопасность (Auth & Security)
    a. Успешный вход администратора.
    b. Неудачные попытки входа (с указанием IP-адреса).
    c. Выход из системы.
    d. Автоматическое завершение сессии по таймауту.
    e. Запросы на сброс пароля, успешные смены пароля и истечение срока действия токенов (с указанием IP-адреса).
    f. Неудачные попытки сброса пароля для несуществующих email (событие уровня WARN, event_type = auth.reset_password_failed_attempt).
    g. Срабатывание лимитов на сброс пароля (по IP или по email). Событие уровня WARN, event_type = auth.reset_password_rate_limited.
    h. События включения/выключения 2FA, успешные и неуспешные попытки ввода кода 2FA, а также использование резервных кодов.
    i. Отклонение запросов вебхуков ЮKassa с неверной подписью или IP-адресом (event_type = payment.webhook_security_failed).
    j. Отсутствие обязательного согласия на обработку персональных данных при запросе услуги (event_type = security.pd_consent_missing).
2. Аудит действий администратора (Data Changes)
    a. Создание, редактирование и удаление колод, карт, раскладов.
    b. Изменение текста системной инструкции (мастер-промпта).
    c. Изменение текста юридических документов (event_type = legal_doc.updated с указанием документа, старой и новой версии).
    d. Изменение цен на толкования.
    e. Редактирование страницы «О проекте» и FAQ.
    f. Восстановление удаленных сущностей (колод, карт, раскладов).
    g. Ручная отмена «зависшего» снимка расклада администратором (event_type = generated_spread.manual_cancel, с указанием ID админа и cancel_reason = ADMIN_MANUAL).
    h. Изменение статуса обращений, добавление внутренних комментариев.
    i. Удаление обращения (event_type = support_ticket.deleted) и восстановление из архива (event_type = support_ticket.restored).
3. Платежи и финансовые операции
    a. Создание платежа (event_type = payment.created).
    b. Успешное завершение платежа (вебхук succeeded).
    c. Отмена или ошибка платежа (canceled) — с указанием payment_id, visitor_id и generated_spread_id. Важно: в логе явно указывается, что снимок расклада остался в статусе PAYMENT_PENDING (готов к повторной оплате).

d. Повторная доставка вебхука (фиксируется как проигнорированный дубликат).
e. Ручной возврат средств через техподдержку (если применимо).
f. Аннулирование чека (tax.receipt_annulled) и ошибки при аннулировании (tax.receipt_annul_failed).
g. Обнаружение дублирующей оплаты (event_type = payment.duplicate_detected) с указанием payment_id, generated_spread_id и visitor_id. Флаг possible_duplicate_payment выставляется в true.
h. Попытки повторного запроса толкования для того же расклада (возврат уже сгенерированного текста) (event_type = llm.cached_response_returned).

4. Интеграция с нейросетью (YandexGPT)
   a. Отправка запроса к YandexGPT.
   b. Успешное получение ответа (с указанием затраченных входных и выходных токенов).
   c. Ошибки и таймауты (с указанием кода ошибки).
   d. Некорректный формат ответа от API.
   e. Срабатывания валидации (event_type = llm.validation_failed) с указанием причины:
      i. too_short / too_long (длина);
      ii. wrong_language (язык);
      iii. stop_words_found (утечка промпта);
      iv. irrelevant (нет упоминания карт);
      v. prohibited_16plus (запрещенный контент).
   f. Частые срабатывания модерации контента за короткий промежуток времени (более 5 событий LLM_PROHIBITED_CONTENT за 1 час). Система отправляет алерт администратору для проверки мастер-промпта. Текст алерта (email) должен содержать только метаданные (ID расклада, причину блокировки, время). Запрещено передавать в теле email сам сгенерированный текст или текст промпта пользователя во избежание утечки PII.
5. Очереди, задачи и Outbox (Celery / Redis)
   a. Смена статусов задач (pending → processing → completed / failed).
   b. События, связанные с повторными попытками (retry) и перемещением задач в DLQ (с указанием task_id, generated_spread_id и причины сбоя).
   c. Принудительное завершение «зависших» задач (зомби-задач) фоновой задачей cleanup_stale_free_reservations (event_type = task.stale_killed, с указанием error_code = TIMEOUT_STALE).
   d. Сброс зависших бесплатных резервов (event_type = free_reservation.stale_reset).
   e. Исчерпание попыток автоматической отправки Outbox-события (перевод в FAILED). Событие уровня CRITICAL (event_type = outbox.max_retries_exceeded).
   f. Успешная публикация Outbox-события в Redis (event_type = outbox.processed).
   g. Истечение времени резервирования слота в очереди (event_type = queue.reservation_expired).

    h. Приход оплаты после истечения резерва слота (event_type = queue.overshoot_payment, уровень CRITICAL).
6. Приватность и согласия (152-ФЗ)
    a. Восстановление сессии по fingerprint (event_type = visitor.restored с указанием visitor_id).
    b. Анонимизация персональных данных пользователя по запросу (event_type = privacy.user_anonymized с указанием visitor_id).
    c. Ежедневная автоматическая анонимизация email по истечении 12 месяцев (event_type = privacy.email_auto_anonymized).
7. Системные ошибки и Email
    a. Ошибки валидации входящих данных.
    b. Ошибки базы данных.
    c. Успешная отправка email (с указанием замаскированного адреса).
    d. Ошибки отправки email (с указанием кода ошибки почтового сервиса).
    e. В логах генерации толкования (llm.request) обязательно сохраняется generated_spread_id (ID снимка).

3. Структура записи лога

1. Каждая запись лога должна содержать следующие обязательные поля:
    a. id — уникальный идентификатор записи (UUID);
    b. timestamp — дата и время события (в базе данных хранится в UTC; при выводе в интерфейс административной панели конвертируется в часовой пояс проекта UTC+5);
    c. level — уровень важности (INFO, WARN, ERROR);
    d. event_type — тип события (например, deck.created, payment.succeeded);
    e. actor_id — идентификатор пользователя (ID админа или visitor_id);
    f. source_ip — IP-адрес запроса;
    g. success — булево значение (успешно событие или нет);
    h. entity_type — тип сущности (например, deck, spread, payment);
    i. entity_id — UUID затронутой сущности (если применимо);
    j. description — текстовое описание события (без чувствительных данных);
    k. payload — объект JSONB с техническими деталями (стек-трейс, тело запроса/ответа API). Персональные данные, текст user_context и секреты маскируются в обязательном порядке перед записью в БД с использованием библиотек или middleware.
2. Для событий, связанных с сущностями (колода, расклад, карта), в лог должно добавляться поле entity_id — идентификатор затронутой сущности.
3. Для событий, инициированных анонимным пользователем, в поле actor_id записывается visitor_id. Для событий администратора — ID администратора. Внешний ключ на anonymous_visitors для logs.actor_id не создаётся, так как таблица логов партиционируется и должна оставаться неизменяемой.
4. Все события сверки (запуск задачи, обнаружение расхождений, успешное обновление статуса) должны логироваться с указанием payment_id, старого и нового статуса. Ошибки при запросах к API ЮKassa во время сверки должны логироваться как WARN и повторяться с экспоненциальной задержкой.

4. Безопасность и защита данных в логах

1. Система не должна записывать в логи чувствительные данные в открытом виде: пароли и их хеши, секретные ключи и API-токены (ЮKassa, YandexGPT), полные номера банковских карт, а также персональные данные пользователей (ФИО, телефоны, полные email), если они не были явно предоставлены для конкретной цели. Также строго запрещено сохранять в логах полный текст поля user_context (вопрос пользователя), так как он может содержать PII.
2. Если в лог необходимо записать значение, которое может содержать чувствительные данные (например, тело вебхука от ЮKassa), система должна маскировать такие поля. Бэкенд должен использовать регулярные выражения (regex) для автоматического поиска и замены следующих паттернов на [REDACTED]:
    a. Номера телефонов (например, +7XXXXXXXXXX, 8XXXXXXXXXX).
    b. Адреса электронной почты (в любых текстовых полях, кроме специально предназначенных для этого).
    c. Номера банковских карт (16-19 цифр подряд).
    d. Если в логе нужно зафиксировать сам факт наличия user_context (например, при ошибке), в лог пишется только его длина (например, user_context_length: 150) или первые 10 символов с пометкой ...
3. Если в лог необходимо записать email-адрес (например, при ошибке отправки письма или для подтверждения успешной доставки), система должна маскировать его по следующему правилу: первый символ адреса + три звездочки + символ @ + домен. Пример: для адреса anna.tarot@yandex.ru в лог должна попасть строка a***@yandex.ru
4. Доступ к просмотру логов должен иметь только администратор. Логи защищены от несанкционированного изменения и удаления (в том числе через интерфейс админ-панели). Удаление старых логов происходит исключительно автоматически путем сброса устаревших партиций (retention policy 30 дней).
5. Данные из ненадёжных источников (например, пользовательский ввод в поле «Контекст») должны экранироваться при выводе в интерфейсе админ-панели, чтобы исключить внедрение вредоносного кода (log injection).
6. При логировании события llm.request (отправка запроса в YandexGPT) в поле payload запрещено сохранять полный текст промпта и user_context. Разрешается сохранять только generated_spread_id, длину промпта в токенах и модель (YandexGPT Lite).

5. Хранение и ротация логов

1. Срок хранения логов — 30 дней с момента записи.
2. Таблица логов в базе данных должна быть партиционирована по дате (разбита на отдельные секции по дням или неделям).
3. Система должна автоматически создавать будущие секции (партиции) заранее (например, на 2–3 месяца вперед), чтобы избежать сбоев при записи новых логов.

Нефункциональные требования

Производительность и масштабируемость

1. Система должна выдерживать не менее 100 активных сессий для MVP (архитектура должна позволять масштабирование при росте).
2. Пропускная способность (RPS):
    a. Чтение (GET): не менее 50 RPS.
    b. Запись (POST/PUT/DELETE): не менее 5 RPS.
3. Время загрузки любой обычной страницы сайта (кроме страницы с ожиданием ответа от нейросети) не должно превышать:
    a. p95 (95% запросов) — не более 2,5 секунд;
    b. p99 (99% запросов) — не более 5 секунд.
4. Понятие «обычная страница»: Время загрузки p95 — 2,5 сек, p99 — 5 сек относится к страницам без тяжелой графики (Главная, «О проекте», страница карты). Для страниц с галереями (список карт колоды, таблицы в админ-панели) допустимое время p95 — 4 секунды. Время генерации ответа нейросети не входит в эти метрики.
5. Время ответа сервера на запросы к базе данных (например, при выборе колоды или получении списка карт) не должно превышать 500 миллисекунд.
6. Метрики нейросети:
    a. Время обработки запроса воркером (от взятия задачи до получения ответа): p95 — 30 сек, p99 — 60 сек.
    b. Время ожидания в очереди (от оплаты до начала обработки воркером): p95 — 1 минута (при очереди до 2-х человек). При номинальной полной очереди (10 бесплатных + 20 платных задач = 30 задач) максимальное время ожидания составляет около 15 минут. При достижении аварийного предела (100 задач) максимальное время ожидания составляет до 50 минут. При превышении этих лимитов новые запросы (или платежи) блокируются.
7. Учитывая ограничение YandexGPT Lite в 1 одновременный поток, система должна использовать очередь запросов. При пиковой нагрузке пользователь должен видеть индикатор загрузки, а не ошибку таймаута.
8. Архитектура должна позволять горизонтальное масштабирование, но развертывание Kubernetes или сложных кластеров в MVP не требуется.
9. Кэширование данных (Redis): Для снижения нагрузки на PostgreSQL тяжелые и часто запрашиваемые GET-запросы должны кэшироваться в Redis.
    a. Списки активных колод, карт и тегов кэшируются на 15 минут. При изменении данных в админ-панели соответствующий ключ в кэше инвалидируется (сбрасывается).
    b. Кэширование должно применяться только к публичной части (не применять к админке и платежам, где нужны актуальные данные).
10. Заголовки кэширования (Frontend): Для статических ресурсов (изображения карт, CSS, JS) Nginx должен отдавать заголовки Cache-Control: public, max-age=31536000 (1 год), чтобы браузеры пользователей кэшировали их локально.

Надежность и отказоустойчивость

1. SLA. Доступность сайта должна составлять не менее 99,5% в месяц (допускается около 3,5 часов простоя в месяц для плановых работ или сбоев хостинга).
2. RTO (Recovery Time Objective): Время восстановления работоспособности сайта не должно превышать 4 часа для некритичных сбоев и 1 час для критичных (блокирующих оплату или генерацию). Для Redis (кэш и брокер) RTO должен составлять не более 15 минут, чтобы минимизировать время нахождения сайта в режиме «только чтение». Для обеспечения RTO в 1 час инфраструктура должна поддерживать быстрое развертывание из снапшотов или иметь настроенный резервный сервер (standby). Для процедуры восстановления должен быть составлен пошаговый Runbook (инструкция).
3. Для процедуры восстановления должен быть составлен и поддерживаться в актуальном состоянии пошаговый Runbook (инструкция), содержащий команды для развертывания БД, восстановления WAL-логов и проверки работоспособности сервисов.
4. RPO (Recovery Point Objective):
    a. Целевое RPO: не более 15 минут. Для этого в PostgreSQL настраивается непрерывное архивирование WAL-журналов (PITR) с параметром archive_timeout = 5m (принудительная отправка сегмента раз в 5 минут) и archive_command в S3-хранилище.
    b. Аварийное RPO: В случае катастрофического отказа или повреждения WAL-архивов, система откатывается к последнему ежедневному полному бэкапу (pg_basebackup). В этом случае допустимая потеря данных (RPO) составляет до 24 часов. При переходе на этот сценарий администратору отправляется критическое уведомление.
5. При недоступности YandexGPT (ошибка 5xx или таймаут) система не должна «падать». Пользователь должен увидеть понятное сообщение об ошибке, а его успешный платёж — сохраниться в базе данных для возможности повторного запроса без оплаты.
6. При недоступности ЮKassa система должна корректно обрабатывать ошибки создания платежа и предлагать пользователю повторить попытку позже.
7. Резервное копирование и тестирование восстановления:
    a. Ежедневный полный бэкап (pg_dump / pg_basebackup): выполняется не реже одного раза в сутки. Срок хранения — не менее 7 дней.
    b. Непрерывное архивирование WAL: журналы транзакций отправляются в отдельное S3-совместимое хранилище. Параметр archive_timeout устанавливается в значение 300 (5 минут). Срок хранения — не менее 7 дней.
    c. Тестирование восстановления (Restore Tests): Администратор или DevOps-инженер обязан не реже одного раза в месяц выполнять тестовое восстановление базы данных из бэкапа на тестовом окружении. Цель теста — подтвердить, что фактическое время восстановления (RTO) укладывается в заявленные 1 час для критичных сбоев и 4 часа для некритичных, а потеря данных (RPO) не превышает 15 минут. Факт успешного теста, замеренное время восстановления и выявленные проблемы фиксируются в журнале регламентных работ.

8. На этапе MVP допускается использование однонодовых (single-instance) конфигураций PostgreSQL, Redis и воркера Celery. Кластеризация и развертывание отказоустойчивых конфигураций (Patroni, Redis Sentinel) переносятся на этап масштабирования.
9. Если Redis недоступен, система не должна создавать новые платежи и новые расклады. Пользователю выводится ошибка: "Сервис временно недоступен. Попробуйте позже". Если Redis недоступен для проверки rate limiting, система должна временно блокировать все новые запросы (fail-closed), чтобы предотвратить DDoS.
10. При падении воркера Celery Docker автоматически перезапускает его. Благодаря настройке task_acks_late = True, задача не будет потеряна: она вернется в очередь Redis и будет подхвачена новым запущенным воркером.

Нагрузочные характеристики

1. Система должна выдерживать не менее 100 одновременных пользователей (сессий) без деградации времени отклика для обычных страниц.
2. Учитывая ограничение YandexGPT Lite (1 одновременная генерация), система должна использовать очередь запросов. При пиковой нагрузке пользователь видит индикатор загрузки, а не ошибку.

Сценарии деградации при сбоях инфраструктуры

1. Недоступность PostgreSQL (База данных):
    a. Бэкенд должен быть настроен на автоматическое переподключение к БД (connection pool с pool_pre_ping).
    b. Если БД недоступна, API должен возвращать HTTP 503 Service Unavailable с кодом ошибки DATABASE_UNAVAILABLE.
    c. Пользовательский интерфейс при получении этой ошибки должен отображать полноэкранную заглушку: «Сервис временно недоступен. Пожалуйста, попробуйте обновить страницу через пару минут», а не показывать «белый экран» или бесконечный спиннер. Сайт переходит в режим полной недоступности, так как PostgreSQL является единственным источником истины.
2. Недоступность S3-хранилища (Картинки):
    a. Если объектное хранилище (S3/MinIO) временно недоступно, бэкенд продолжает отдавать текстовые данные (названия карт, описания) и корректно проводить оплаты.
    b. На фронтенде для всех тегов <img> должен быть настроен обработчик onerror. Если картинка не загружается, вместо нее отображается локальная заглушка (например, рубашка карты или фон с названием карты), чтобы не ломать верстку.
3. Недоступность Nginx (Точка входа):
    a. Для контейнера Nginx в docker-compose.yml должна быть настроена политика restart: always.
    b. Для контроля доступности всего сайта должен быть настроен внешний мониторинг (например, UptimeRobot или аналог), который раз в 1–2

минуты опрашивает публичный эндпоинт здоровья GET /api/health и отправляет уведомление администратору (в Telegram или на email) при получении ошибки или таймаута.

4. Недоступность Redis (Кэш, брокер сообщений, сессии, rate limiting):
    a. Операции чтения (GET): Просмотр колод, карт, страницы «О проекте» и других публичных данных продолжает работать. Бэкенд должен корректно обрабатывать ошибку подключения к Redis: игнорировать кэш и выполнять запросы напрямую к PostgreSQL. При этом время ответа может увеличиться, но сайт не должен возвращать ошибку 500/503.
    b. Операции записи и финансовые операции (POST/PUT/DELETE): Создание раскладов, запросы на AI-толкование, создание платежей (включая попытку резервирования слота очереди) и любые другие POST-запросы блокируются. Пользователю возвращается ошибка SERVICE_UNAVAILABLE (503) с сообщением: «Сервис временно перегружен, попробуйте позже. Деньги не списаны».
    c. Rate Limiting: Если Redis недоступен, проверка лимитов невозможна. В этом случае все POST-запросы блокируются (fail-closed), чтобы предотвратить DDoS-атаки, но GET-запросы пропускаются без ограничений.
    d. Режим Read-Only на фронтенде: При получении ошибки 503 от POST-эндпоинтов фронтенд должен перевести интерфейс в режим «только чтение»: заблокировать кнопки «Вытянуть карту», «Сделать расклад», «Оплатить», отобразить глобальный баннер «Сервис временно работает в режиме просмотра. Создание раскладов и оплата недоступны» и разблокировать их при следующем успешном запросе.
5. Общие правила:
    a. Никакая ошибка внешней инфраструктуры не должна приводить к потере уже оплаченных данных. Если PostgreSQL временно недоступна в момент прихода вебхука ЮKassa, вебхук должен вернуть ошибку (ЮKassa повторит попытку позже), либо бэкенд должен записать сырые данные вебхука в локальный лог/Redis для последующей обработки (если это возможно).
6. Классификация сбоев для RTO:
    a. Критичные сбои (RTO ≤ 1 час): Недоступность PostgreSQL (потеря данных, повреждение файлов), недоступность Nginx (полная недоступность сайта).
    b. Некритичные сбои (RTO ≤ 4 часа): Недоступность YandexGPT (генерация приостановлена, но сайт работает, платежи сохраняются), недоступность S3 (картинки не грузятся, отображаются заглушки), недоступность Redis (блокируются POST-запросы, но чтение работает).

Обработка ошибок внешних API (YandexGPT)

1. Политика обработки ошибок и повторных попыток (retry) при взаимодействии с YandexGPT строго описана в разделе "Работа воркера" (см. выше)

Безопасность

1. Весь трафик между пользователем и сервером должен передаваться по защищенному протоколу HTTPS с использованием TLS 1.2 или выше.
2. Система должна быть защищена от основных веб-уязвимостей (OWASP Top 10), включая SQL-инъекции, XSS (межсайтовый скриптинг) и CSRF (межсайтовую подделку запросов). Для защиты от Stored XSS весь пользовательский и административный контент, содержащий HTML/Markdown (юридические документы, страница «О проекте», тексты карт), должен проходить обязательную серверную санитизацию с использованием библиотек, отсекающих опасные теги и атрибуты (например, bleach для Python).
3. Формы на сайте (включая поле «Контекст/Вопрос») должны быть защищены от автоматического спама (например, с помощью капчи или honeypot-полей).
4. Для API-эндпоинтов должен быть настроен rate limiting (ограничение частоты запросов) для защиты от DDoS-атак и брутфорса.
5. Все секретные ключи (ЮKassa, YandexGPT, доступы к базе данных) должны храниться в защищенных переменных окружения (.env) и не должны попадать в репозиторий.
6. Платежные данные пользователей не должны храниться на сервере. Обработка карт происходит на стороне ЮKassa (PCI DSS-совместимость обеспечивается платёжным сервисом).
7. Content Security Policy (CSP): Заголовок Content-Security-Policy должен ограничивать источники загрузки ресурсов для предотвращения XSS-атак. Базовая конфигурация должна включать:
    a. default-src 'self';
    b. script-src 'self' https://yookassa.ru https://*.yookassa.ru https://*.sentry.io; (домен Sentry добавляется, если используется система мониторинга ошибок).
    c. style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
    d. font-src 'self' https://fonts.gstatic.com;
    e. img-src 'self' data: https://*.yandexcloud.net; (если изображения карт загружаются напрямую из S3-хранилища. Если используется проксирование через Nginx, допускается оставить только 'self').
    f. connect-src 'self' https://api.ametistovaya-zhaba.ru https://*.sentry.io; (домен API, к которому обращается фронтенд, и домен Sentry для отправки логов).
    g. frame-ancestors 'none'; (защита от Clickjacking).

    Конкретный список доменов должен быть задокументирован в конфигурации Nginx и обновляться при подключении новых сторонних сервисов (аналитика, мониторинг).
    Заголовок CSP также должен содержать директиву frame-ancestors 'none'; для предотвращения атак типа Clickjacking.
8. Веб-сервер (Nginx) должен отдавать заголовок Strict-Transport-Security: max-age=31536000; includeSubDomains для принудительного использования HTTPS.
9. Шифрование данных (Encryption at rest): База данных и ежедневные резервные копии должны храниться на диске с шифрованием (например, LUKS на уровне VPS или встроенное шифрование облачного провайдера).
10. Конкретные цифры Rate Limiting:

a. Для страницы авторизации: не более 5 попыток в минуту с одного IP.
b. Для эндпоинта сброса пароля (/api/admin/auth/reset-password): не более 3 запросов в час с одного IP, а также не более 1 запроса в 5 минут на конкретный email.
c. Для публичных GET-запросов (просмотр раскладов, карт, тегов): не более 120 запросов в минуту с одного IP. Для публичных POST-запросов (генерация расклада, вытягивание карты): не более 30 запросов в минуту с одного IP.
d. Для вебхуков ЮKassa: разрешать запросы только с официальных IP-адресов ЮKassa (список запрашивается у техподдержки).
e. Для эндпоинта /api/payments/verify: не более 5 запросов в минуту с одного visitor_id или IP-адреса, чтобы предотвратить DDoS-атаку на API ЮKassa через наш сервер.
f. Для формы обратной связи (POST /api/support/tickets): не более 3 запросов за 10 минут с одного IP-адреса и/или visitor_id.
g. f. Для эндпоинта подтверждения email для просмотра результата (POST /api/spreads/{id}/verify-email): не более 5 попыток на один generated_spread_id в час и не более 10 попыток с одного IP-адреса в час.

11. Защита от DDoS: На уровне хостинга должен быть подключен базовый сервис защиты от DDoS-атак (предоставляется провайдером VPS, например, Timeweb или Selectel).
12. Сканирование уязвимостей: В CI/CD (GitHub Actions) должен быть настроен бот (например, Dependabot), который автоматически проверяет используемые библиотеки на наличие известных уязвимостей (CVE).
13. Пентест: Для MVP достаточно прогона автоматических сканеров (например, OWASP ZAP) перед релизом. Полноценный пентест — в бэклоге.
14. Все загруженные пользователем или администратором файлы (изображения карт) должны сохраняться в объектное хранилище (S3) или в директорию, недоступную для прямого выполнения скриптов. Оригинальные имена файлов не сохраняются: файлы переименовываются в UUID перед сохранением на диск.
15. Все входящие данные (JSON, form-data) должны проходить строгую серверную валидацию с использованием схем (например, Pydantic в FastAPI). В схемах должен быть установлен запрет на передачу лишних полей (mass assignment), чтобы злоумышленник не мог перезаписать скрытые поля (например, is_admin, is_deleted, role).
16. Для защиты от DoS-атак на уровне веб-сервера (Nginx) должен быть установлен лимит на максимальный размер тела запроса (client_max_body_size). Для API-эндпоинтов (кроме загрузки файлов) лимит должен составлять не более 1 МБ. Для эндпоинта массовой загрузки карт (/api/admin/cards/upload) лимит должен быть увеличен до 50 МБ.
17. Если система выполняет исходящие HTTP-запросы по URL, полученным от пользователя (например, при загрузке изображений по ссылке), необходимо блокировать запросы к локальным IP-адресам (localhost, 127.0.0.1, 169.254.169.254) и внутренним подсетям.
18. Все административные эндпоинты (/api/admin/*) должны быть защищены middleware (или Depends в FastAPI), который проверяет наличие валидной

сессии и роль Admin. Проверка должна происходить на уровне маршрутизатора, а не внутри каждой отдельной бизнес-функции.

19. Должна быть предусмотрена процедура плановой ротации API-ключей (ЮKassa, YandexGPT, Unisender) и паролей к базе данных. Ключи должны иметь возможность обновляться в переменных окружения без простоя сервиса (graceful reload).
20. При распаковке ZIP-архивов сервер должен проверять пути файлов внутри архива. Запрещается распаковка файлов, пути которых содержат ../ (выход за пределы целевой директории). Все файлы должны распаковываться во временную директорию, недоступную извне.
21. Защита чувствительных данных в URL (result_access_token): Поскольку токен доступа к результату (result_access_token) передается в query-параметре URL, система должна минимизировать риски его утечки:
    a. Веб-сервер Nginx должен быть настроен таким образом, чтобы параметр token не попадал в открытом виде в логи доступа (access.log). Он должен заменяться на [REDACTED].
    b. Для всех страниц, отображающих результат расклада, сервер должен отдавать HTTP-заголовок Referrer-Policy: no-referrer (или strict-origin-when-cross-origin), чтобы токен не передавался на сторонние ресурсы при клике по внешним ссылкам.

Совместимость и юзабилити (UX/UI)

1. Сайт должен быть адаптивным: корректно отображаться и быть удобным для использования на экранах смартфонов (ширина от 320px), планшетов и десктопов.
2. Административная панель проектируется исключительно для десктопных компьютеров и планшетов (ширина экрана от 1024px). Разработка мобильной версии для админ-панели не требуется.
3. Сайт должен корректно работать в последних версиях браузеров: Chrome, Safari, Firefox, Edge, а также в мобильных браузерах (Chrome Mobile, Safari Mobile).
4. Интерфейс должен соответствовать стандарту доступности WCAG 2.1 уровня AA, что включает в себя: контрастность текста не менее 4.5:1, наличие альтернативного текста (alt) для всех изображений карт, возможность навигации по формам (включая оплату) с помощью клавиатуры. Минимальный размер интерактивных элементов (кнопок) на мобильных устройствах — 44x44 пикселя
5. Язык интерфейса на старте — русский. Все тексты должны быть грамотными и не содержать опечаток.
6. Все формы, содержащие поля для ввода персональных данных (email в формах толкования и оплаты), должны проходить обязательную клиентскую валидацию до отправки запроса на сервер. Кнопки «Оплатить» и «Получить толкование» должны быть заблокированы, пока введенный email не будет соответствовать формату RFC 5322, а чекбоксы согласия не будут установлены.

7. Обработка ошибок инфраструктуры (503 Service Unavailable): Фронтенд должен иметь глобальный перехватчик ошибок. При получении статуса 503 от любых POST-запросов (связанных с генерацией или оплатой), интерфейс должен блокировать интерактивные элементы (кнопки действий), предотвращая повторные клики, и выводить понятное пользователю уведомление о временной недоступности функций записи (согласно разделу «Сценарии деградации»).
8. В подвале (футере) сайта должна быть размещена ссылка «Настройки Cookie», открывающая модальное окно с текущим статусом согласия и возможностью изменить его (переключатели «Принять» / «Отклонить»). Это обеспечивает соблюдение требований 152-ФЗ о возможности отзыва согласия на обработку данных.
9. На странице отображения результата расклада (по ссылке из письма) в HTML-коде должен присутствовать мета-тег <meta name="referrer" content="no-referrer">, чтобы браузер пользователя не передавал URL с токеном доступа при переходе по внешним ссылкам.
10. Безопасность токена доступа (Frontend):
    a. После успешного извлечения токена из URL (?token=...) и загрузки данных на странице результата, фронтенд обязан очистить адресную строку браузера (удалить query-параметр token), используя метод history.replaceState(). Это предотвратит утечку токена при копировании ссылки или через историю браузера.
    b. Токен доступа (result_access_token) запрещено сохранять в localStorage или sessionStorage. Он должен храниться только в оперативной памяти (в состоянии компонента) на время текущей сессии.

Сопровождаемость и развертывание

1. Система должна разворачиваться с использованием Docker (и Docker Compose) для обеспечения одинакового окружения на локальном компьютере разработчика и на продакшн-сервере.
2. Все изменения в схеме базы данных должны выполняться через систему миграций (например, Alembic для Python или Prisma для Node.js), чтобы обновлять базу без потери данных.
3. Конфигурация приложения (например, выбор модели YandexGPT Lite или Pro, цена толкования) должна изменяться через переменные окружения без необходимости пересборки Docker-образа.
4. Код проекта должен быть покрыт документацией (README с инструкцией по запуску) и следовать единому стилю кодирования (линтеры).

Юридические и регуляторные требования

1. На сайте должны быть размещены следующие юридические документы:
    a. Публичная оферта (Договор возмездного оказания услуг) — регламентирует порядок оплаты, порядок оказания услуг, статус исполнителя (самозанятый), а также порядок акцепта оферты. Не содержит правил использования сайта и возрастных ограничений.

    b. Правила возврата средств — четко определяют условия возврата (см. раздел «Возвраты»), включая случаи неоказания услуги (FAILED_VALIDATION), двойного списания и порядок обращения в поддержку.
    c. Пользовательское соглашение — правила использования сайта, ограничение ответственности за использование AI-текстов (дисклеймер о том, что услуга носит информационно-развлекательный характер и не является медицинской, психологической или финансовой консультацией), а также маркировка «16+».
2. Система должна автоматически передавать данные об успешных оплатах в сервис «Мой Налог» для формирования чеков (через yookassa-to-mynalog в соответствии с 422-ФЗ для самозанятых).
3. Сайт и весь генерируемый контент должны быть маркированы знаком информационной продукции «16+».
4. Контент сайта (описания карт, AI-толкования) не содержит сцен насилия, жестокости, употребления наркотиков или материалов, запрещенных для детей (согласно ст. 5-10 436-ФЗ). Тематика Таро и эзотерики в данном контексте носит ознакомительный и развлекательный характер и соответствует возрастной категории «16+» (для детей, достигших шестнадцати лет).
5. Знак «16+» должен быть размещен в подвале (футере) сайта, а также на странице «О проекте».
6. При редактировании системного промпта (мастер-промпта) администратор должен избегать формулировок, которые могут привести к генерации нейросетью контента, не соответствующего маркировке «16+».
7. Техническая защита: Помимо инструкций в промпте, система применяет обязательную пост-модерацию сгенерированных текстов на стороне воркера (проверка на стоп-слова и запрещенные темы). Тексты, не прошедшие модерацию 16+, не сохраняются и не отображаются пользователю.
8. В подвале (футере) сайта должны быть указаны реквизиты исполнителя в соответствии с 422-ФЗ:
    a. ФИО;
    b. ИНН;
    c. Статус «Плательщик налога на профессиональный доход».

   Эти данные должны подтягиваться из переменных окружения основного бэкенда (TAXPAYER_NAME, TAXPAYER_INN) и быть видимыми на всех страницах сайта.
9. Для доступа к платному функционалу пользователь обязан явно подтвердить согласие с условиями Публичной оферты и Правилами возврата (путем установки чекбокса на странице оплаты). Без этого согласия создание платежа невозможно.

Локализация и часовые пояса

1. Все даты и время в базе данных (PostgreSQL) должны храниться в формате UTC (тип timestamp with time zone). Все даты и время в пользовательском интерфейсе, административной панели и на странице просмотра логов должны

отображаться в часовом поясе проекта UTC+5. Конвертация из UTC в UTC+5 выполняется на уровне бэкенда (API) или фронтенда перед отображением.

2. Суммы в интерфейсе и базе данных должны отображаться в рублях (RUB) с двумя знаками после запятой.

Обработка ошибок

Все ответы API с кодом ошибки (4xx, 5xx) должны возвращать JSON строго определенного формата:

```
{
  "error": {
    "code": "STRING_CODE",
    "message": "Понятное сообщение для пользователя на русском языке",
    "details": null
  }
}
```

Поле code используется фронтендом для программной обработки (например, INSUFFICIENT_FUNDS, VALIDATION_ERROR, LLM_TIMEOUT). Поле message выводится пользователю. Технические детали (stack trace) в ответе API не передаются».

YandexGPT

HTTP-код / Тип ошибки	**Причина**	**Действие системы (Backend)**	**Retry / Стратегия**	**Логирование (Уровень и данные)**
429 Too Many Requests	Превышение лимита запросов (квоты)	Задача возвращается в очередь (Redis) с задержкой. Пользователь видит статус «В очереди».	Да. Экспоненциальная задержка. Учесть заголовок Retry-After. Максимум 3-5 попыток.	WARN event_type: llm.rate_limit http_status: 429 spread_id, retry_count

5xx (500, 502, 503)	Внутренняя ошибка или недоступность сервиса Yandex Cloud	Задача возвращается в очередь. Пользователь видит сообщение «Сервис временно недоступен, ожидайте».	Да. Экспоненциальная задержка. Максимум 3-5 попыток.	ERROR event_type: llm.server_error http_status, error_body (sanitized) spread_id, retry_count
401 / 403	Неверный API-ключ или недостаточно прав	Немедленная остановка. Логирование критической ошибки. Уведомление администратора.	Нет. Требуется вмешательство человека.	ERROR (Critical) event_type: llm.auth_error http_status, error_body
400 Bad Request	Некорректный формат запроса (например, превышена длина промпта)	Логирование. Пользователю: «Не удалось обработать запрос». Платеж сохраняется.	Нет. Требуется исправление логики формирования промпта.	ERROR event_type: llm.bad_request http_status, error_body, spread_id

Timeout (>60 сек)	Превышено время ожидания ответа от нейросети	Прерывание запроса. Статус задачи failed. Пользователь видит кнопку «Повторить запрос».	Да (1 повторная попытка).	ERROR event_type: llm.timeout timeout_duration, spread_id

ЮKassa

HTTP-код / Тип ошибки	**Причина**	**Действие системы (Backend)**	**Retry / Стратегия**	**Логирование (Уровень и данные)**
400 Bad Request	Синтаксическая ошибка в запросе, неверные параметры	Логирование ошибки. Пользователю: «Ошибка при создании платежа».	Нет. Требуется исправление кода.	ERROR event_type: payment.bad_request http_status, error_body, payment_id, visitor_id
401 Unauthorized	Неверный shopId или секретный ключ	Немедленная остановка. Логирование критической ошибки. Уведомление	Нет. Требуется проверка учетных данных.	ERROR (Critical)

		администратора.		event_type: payment.auth_error http_status, error_body
403 Forbidden	Не хватает прав на операцию	Логирование. Проверка настроек магазина в ЮKassa.	Нет. Требуется вмешательство человека.	ERROR event_type: payment.forbidden http_status, error_body
429 Too Many Requests	Превышен лимит запросов к API ЮKassa	Задержка перед повторной попыткой.	Да. Экспоненциальная задержка.	WARN event_type: payment.rate_limit http_status, retry_count
500 Internal Server Error	Технические проблемы на стороне ЮKassa	Логирование. Пользователю: «Платёжный сервис недоступен».	Да. 2-3 попытки с задержкой.	ERROR event_type: payment.server_error http_status, error_body, payment_id

Логические ошибки (например, 410)	Заказ не оплачен, возврат невозможен	Логирование. Информирование администратора для ручной проверки.	Нет. Требуется ручной разбор.	ERROR event_type: payment.logic_error http_status, error_body, payment_id

Unisender Go

HTTP-код / Тип ошибки	**Причина**	**Действие системы (Backend)**	**Retry / Стратегия**	**Логирование (Уровень и данные)**
400 Bad Request	Неверный формат запроса, ошибка в параметрах	Логирование. Задача отправки email помечается как failed (уходит в DLQ). Общий статус generation_status остается COMPLETED (пользователь уже получил результат на сайте).	Нет. Требуется исправление кода.	ERROR event_type: email.bad_request http_status, error_body, visitor_id

429 Too Many Requests	Превышен лимит вызовов (60 в минуту)	Задача на отправку email возвращается в очередь.	Да. Экспоненциальная задержка. Максимум 3 попытки.	WARN event_type: email.rate_limit http_status, retry_count
5xx (500, 502, 503)	Внутренняя ошибка серверов Unisender Go	Задача возвращается в очередь для повторной отправки.	Да. Экспоненциальная задержка.	ERROR event_type: email.server_error http_status, error_body, retry_count
Ошибки валидации email	Неверный формат email получателя	Письмо не отправляется. email_delivered = false. Общий статус generation_status остается COMPLETED.	Нет.	WARN event_type: email.validation_error error_body (без указания самого email)
Ошибки аутентификации	Неверный API-ключ Unisender	Немедленная остановка.	Нет. Требуется проверка ключа.	ERROR (Critical)

		Уведомление администратора.		event_type: email.auth_error http_status, error_body

Технические условия

1. Серверная платформа:
    a. Процессор (vCPU): 2–4 ядра.
    b. Оперативная память (RAM): минимум 4 ГБ.
    c. Диск: от 40 ГБ, рекомендуется использовать NVMe SSD для обеспечения высокой скорости записи WAL-журналов и стабильной работы PostgreSQL.
    d. Операционная система: Ubuntu 22.04 или 24.04.
2. Язык разработки: Python (версия 3.11 или выше). Для хеширования паролей администраторов должна использоваться библиотека argon2-cffi.
3. Веб-фреймворк: FastAPI (для бэкенда и API). Выбор обусловлен высокой производительностью, нативной поддержкой асинхронности (что критично для работы с очередями и внешними API) и автоматической генерацией документации (Swagger/OpenAPI). В качестве альтернативы допустим Django (с Django REST Framework), если потребуется более "тяжёлая" админ-панель "из коробки".
4. Сервер приложений: Для production-развёртывания FastAPI (ASGI) должен использоваться Gunicorn в качестве менеджера процессов, который управляет несколькими воркерами Uvicorn.
5. Для обработки «ядовитых» сообщений (задач, которые стабильно падают с ошибкой при выполнении) используется отдельная очередь Celery dlq (Dead Letter Queue). Если задача превысила максимальное количество повторных попыток (согласно матрице retry), она автоматически маршрутизируется в очередь dlq для последующего анализа и ручного перезапуска администратором через админ-панель.
6. Основная СУБД: PostgreSQL (версия 16 или выше). Для обеспечения RPO ≤ 15 минут для финансовых данных должна быть включена настройка wal_level = replica (или logical), а также настроен параметр archive_mode = on и archive_command (или использован инструмент вроде pgBackRest / WAL-G) для непрерывной отправки WAL-сегментов в хранилище резервных копий. PostgreSQL является единственным источником истины (source of truth) для всей исторической статистики, логов, платежей и сгенерированных раскладов. Все агрегированные метрики (количество оплат, конверсия, ARPPU) рассчитываются на основе данных из этой базы.
7. Обязательное использование объектного хранилища. В production-среде для хранения изображений карт и раскладов должно использоваться S3-

совместимое объектное хранилище (например, Yandex Object Storage, Selectel или MinIO, развернутый в отдельном контейнере). Локальное хранение файлов на диске с бэкендом допускается только для локальной разработки (dev-environment). Раздача изображений пользователям должна осуществляться через Nginx (проксирование) или CDN для снижения нагрузки на основной бэкенд.

8. Брокер сообщений и кэш: Redis. Используется как:
    a. Брокер для очереди фоновых задач (Celery);
    b. Хранилище для сессий и временных данных;
    c. Инструмент для rate limiting (ограничения частоты запросов);
    d. Кэш для публичных GET-запросов (списки колод, карт, тегов).

    Для предотвращения потери задач и данных при перезапуске контейнера Redis должен быть настроен с включенным механизмом персистентности AOF (Append Only File) с политикой appendfsync everysec.
9. Система очередей: Celery. Используется для выноса длительных операций (запросы к YandexGPT, отправка email) из основного потока веб-сервера, чтобы не блокировать обработку запросов других пользователей. Для реализации паттерна Transactional Outbox должен быть настроен отдельный фоновый процесс-диспетчер (например, задача Celery Beat, запускаемая каждые 5 секунд). Диспетчер читает записи из таблицы outbox_events со статусом PENDING, публикует их в соответствующие очереди Redis и меняет статус на PROCESSED. Публикация должна быть идемпотентной (по id события), чтобы избежать дублирования задач при сбоях.
10. Мониторинг очередей: Для отслеживания состояния задач в реальном времени (сколько в очереди, сколько выполнено, есть ли ошибки) должен использоваться веб-интерфейс Flower (или его аналоги).
11. Контейнеризация: Весь проект (бэкенд, воркер, база данных, Redis, Nginx) должен быть упакован в Docker и запускаться через Docker Compose. Это обеспечит идентичность окружений на локальной машине разработчика и на production-сервере.

    Для сервиса PostgreSQL должен быть настроен отдельный Docker volume (или S3-совместимое хранилище) для сохранения WAL-архивов, чтобы они не терялись при перезапуске контейнера. В файле docker-compose.yml должна быть предусмотрена конфигурация для работы с S3-хранилищем:
    a. Если выбрано локальное S3-хранилище (MinIO), в docker-compose.yml должен быть добавлен отдельный сервис minio с настроенным постоянным томом (volume) для хранения данных.
    b. Если используется облачное хранилище (Yandex Object Storage), в переменные окружения бэкенда должны быть добавлены ключи доступа (S3_ACCESS_KEY, S3_SECRET_KEY), имя бакета (S3_BUCKET_NAME) и эндпоинт (S3_ENDPOINT).

    Для всех сервисов в файле docker-compose.yml (включая backend, worker, postgres, redis, nginx и minio) должна быть настроена политика автоматического перезапуска restart: always (или unless-stopped). Для сервисов postgres, redis и minio должны быть настроены healthcheck (проверки работоспособности), чтобы Docker мог автоматически перезапустить контейнер, если сервис внутри него завис, и не запускал бэкенд до полной готовности БД.

Для сервиса yookassa-to-mynalog в docker-compose.yml должна быть предусмотрена следующая конфигурация:

```
yaml

services:
  yookassa-to-mynalog:
    image: <образ_сервиса>
    restart: always
    volumes:
      - yookassa_data:/app/data # Монтирование тома для SQLite
    environment:
      - REFUNDS_ENABLED=true
      # ... остальные переменные

volumes:
  yookassa_data:
```

12. Веб-сервер (Reverse Proxy): Nginx. Используется как точка входа для приема HTTPS-трафика, отдачи статики и проксирования запросов к Gunicorn. Конфигурация Nginx (nginx.conf) должна быть настроена для раздачи изображений из объектного хранилища. Если используется MinIO, Nginx должен проксировать запросы к контейнеру minio (например, по пути /images/). Если используется облачное хранилище (Yandex Object Storage), Nginx должен либо проксировать запросы к нему (для обхода CORS-ограничений), либо раздавать статику напрямую из бакета через CDN, если это настроено на стороне облака. В конфигурации Nginx должна быть настроена директива client_max_body_size 5M; (глобально для загрузки картинок) и отдельно для API-эндпоинтов (например, location /api/ { client_max_body_size 1M; }), чтобы ограничить размер передаваемых JSON-данных. Также в конфигурации nginx.conf должна быть настроена кастомная директива log_format, которая маскирует query-параметр token (например, с использованием регулярных выражений), чтобы result_access_token не попадал в логи в открытом виде.
    Также для локации, обслуживающей фронтенд (страницы /result/*), Nginx должен принудительно отдавать HTTP-заголовок Referrer-Policy: no-referrer (в дополнение к мета-тегу в HTML).
13. Хостинг: Рекомендуется использовать российские облачные платформы (например, Yandex Cloud, Selectel, Timeweb или Servercore), предоставляющие VPS/VDS с возможностью установки Docker и обеспечивающие необходимую доступность (Uptime).
14. Мониторинг: Для MVP достаточно использовать Sentry (для отлова ошибок бэкенда и фронтенда), Flower (для визуального контроля очередей Celery) и скрипт-мониторинг успешности архивации WAL (алерт при ошибках команды archive_command). Развертывание тяжелого стека Prometheus + Grafana + Loki откладывается до этапа масштабирования.
    a. Требования к Sentry и CSP: При подключении Sentry (или аналогов) необходимо убедиться, что домены для сбора ошибок (например, *.sentry.io) добавлены в директивы script-src и connect-src заголовка

Content-Security-Policy (см. раздел «Безопасность», п. 7), иначе браузеры пользователей заблокируют отправку отчетов об ошибках.

b. Каналы оповещения (Алертинг): В качестве канала для отправки критических алертов должен использоваться только email (через сервис Unisender Go). Использование Telegram для алертов не предусмотрено, так как мессенджер работает в РФ нестабильно.
c. Инфраструктурные алерты (WAL и БД): Критически важно: Настроить алерт при любых ошибках выполнения archive_command (например, если WAL-файл не удалось отправить в S3). Это предотвратит ситуацию, когда система работает без точек восстановления.
   Также должен быть настроен алерт при отставании репликации (если используется standby-сервер) или при сбое отправки WAL-сегментов, чтобы дежурный инженер мог начать процедуру восстановления по Runbook до наступления полного отказа.
d. Сервисные алерты (Бизнес-логика и Очереди): Система должна отправлять уведомления администратору на email при наступлении следующих событий:
   i. Отсутствие успешных ответов от сервиса yookassa-to-mynalog более 15 минут (при наличии новых чеков).
   ii. Наличие в таблице receipt_jobs записей со статусом failed или created старше 30 минут (проблемы с ФНС).
   iii. Превышение количества событий в Outbox со статусом FAILED порога в 5 штук (сбой Redis или диспетчера).
   iv. Появление любых задач в очереди DLQ (Celery).
   v. Достижение 80% и 100% заполнения очереди llm_tasks (пороги MAX_HARD_QUEUE_LIMIT).

15. Логирование: Приложение должно писать структурированные логи (в формате JSON) в stdout. Для production-среды рекомендуется сбор логов в централизованное хранилище (например, Grafana Loki или Elasticsearch) для удобного поиска и анализа.
    При работе с сервисом yookassa-to-mynalog основной бэкенд должен регулярно проверять его доступность (healthcheck) и логировать любые сетевые ошибки взаимодействия с ним (например, если сервис не отвечает по HTTP).
16. Управление секретами: Все API-ключи, пароли и токены (от YandexGPT, ЮKassa, базы данных) должны храниться в переменных окружения (.env), которые не попадают в систему контроля версий. В переменных окружения должен храниться JWT_SECRET_KEY для генерации и проверки подписанных ссылок на результаты раскладов. Дополнительно, для отображения в футере сайта и в юридических документах, в .env основного бэкенда должны быть добавлены публичные реквизиты самозанятого:
    a. TAXPAYER_NAME (ФИО);
    b. TAXPAYER_INN (ИНН);
    c. ADMIN_ALERT_EMAIL (email администратора, на который отправляются критические алерты).
17. Процесс развёртывания (CI/CD): На этапе MVP достаточно настроить автоматический запуск линтеров и тестов при пуше в репозиторий (GitHub Actions). Автоматический деплой на сервер можно делать вручную по SSH.

18. Для обработки и санитизации Markdown на бэкенде должна использоваться связка библиотек markdown-it-py (парсинг) и bleach (очистка опасного HTML).

Технические спецификации и архитектура проекта

Архитектура проекта

Проект должен быть построен по клиент-серверной архитектуре с вынесением тяжелых задач в отдельные фоновые процессы. Система состоит из следующих независимых блоков:

1. Клиент (Frontend): Отвечает за отображение интерфейса, анимации и отправку запросов к API.
2. Веб-сервер (Nginx): Принимает запросы, отдает статику (картинки карт, CSS) и перенаправляет запросы к бэкенду.
3. Бэкенд (FastAPI): Обрабатывает логику выбора карт, общается с базой данных и платёжной системой.
4. База данных (PostgreSQL): Хранит колоды, расклады, тексты промптов, логи, данные о платежах и результаты толкований.
5. Брокер сообщений (Redis): Выступает в роли «очереди». Сюда бэкенд кладёт задачи, которые нельзя сделать мгновенно.
6. Воркеры (Celery): Берут задачи из Redis, ходят в нейросеть и отправляют письма. Работают независимо от основного сайта.
7. Внешние API:
    a. YandexGPT (генерация текста);
    b. ЮKassa (приём платежей);
    c. Unisender Go (отправка email).

Диаграмма последовательности для генерации расклада отражена в приложении 2.

Процесс получения толкования разделен на три этапа:
1. Этап 1 (Схема Б.1) — инициация. Отражен в приложении 3.
2. Этап 2 (Схема Б.2) — фоновая обработка. Отражен в приложении 4.
3. Этап 3 (Схема Б.3) — ожидание результата. Отражен в приложении 5.

Воркеры (асинхронная обработка задач)

1. В Celery должны быть настроены четыре независимые очереди:
    a. llm_tasks — для задач генерации AI-толкований (высокая нагрузка, долгое выполнение).
    b. email_tasks — для отправки транзакционных писем через Unisender Go.
    c. payment_tasks — для фоновой сверки платежей (Celery Beat) и обработки критических финансовых операций.

    d. dlq (Dead Letter Queue) — для «ядовитых» задач, которые стабильно падают с ошибкой.
2. В конфигурации Celery должны быть заданы следующие правила маршрутизации (полные имена задач -> очередь):
    a. app.tasks.generate_interpretation -> llm_tasks;
    b. app.tasks.send_email -> email_tasks;
    c. app.tasks.reconcile_payments -> payment_tasks;
    d. app.tasks.process_outbox -> payment_tasks (диспетчер Outbox-событий);
    e. app.tasks.anonymize_old_emails -> email_tasks (ежедневная очистка устаревших email);
    f. app.tasks.send_alert_email -> email_tasks (отправка критических системных уведомлений администратору);
    g. app.tasks.cleanup_expired_reservations -> payment_tasks (очистка просроченных резервов слотов в Redis);
    h. app.tasks.anonymize_user_data -> payment_tasks (анонимизация данных по запросу пользователя).

    В очередь payment_tasks помимо app.tasks.reconcile_payments и app.tasks.process_outbox маршрутизировать также app.tasks.cleanup_stale_free_reservations.
3. Для каждой очереди должен быть запущен отдельный пул воркеров со следующими настройками:
    a. Воркеры llm_tasks: Настраиваются с concurrency=1 и worker_prefetch_multiplier=1. Это строго необходимо для соблюдения лимита YandexGPT Lite (1 одновременный поток) и предотвращения блокировки быстрых задач долгими.
    b. Воркеры email_tasks: Могут работать с concurrency=2 или 4 (в зависимости от лимитов Unisender Go).
    c. Воркеры payment_tasks: Имеют наивысший приоритет в случае нехватки ресурсов сервера.
    d. Очередь dlq: Не обрабатывается воркерами автоматически. Задачи попадают сюда после исчерпания попыток ретрая и ожидают ручного вмешательства администратора через админ-панель.
4. Для очереди llm_tasks должны быть настроены приоритеты сообщений (через x-max-priority в Redis). Платные задачи публикуются с приоритетом 9, бесплатные — с приоритетом 1. Воркеры настроены забирать задачи в порядке приоритета, чтобы оплатившие пользователи не ждали в очереди за бесплатными.
5. Анонимизация устаревших email. Задача app.tasks.anonymize_old_emails запускается Celery Beat ежедневно в 03:00 в очереди email_tasks. Задача находит записи в generated_spreads и support_tickets, у которых поле email_delete_after меньше текущего времени (now()), и заменяет значение поля email на заглушку deleted_user@anonymized.local.
   Задача не затрагивает таблицы payments и receipt_jobs, так как хранение персональных данных там требуется по 422-ФЗ. Также она игнорирует записи, у которых уже стоит заглушка (идемпотентность).
6. Cleanup зависших бесплатных резервов. Задача app.tasks.cleanup_stale_free_reservations запускается Celery Beat каждые 15 минут в очереди payment_tasks.

Алгоритм работы:

a. Задача находит записи anonymous_visitors с free_interpretation_pending = true и free_interpretation_pending_since < now() - interval '2 hours'.
b. Для каждой такой записи она ищет активные бесплатные задачи (is_free_request = true и generation_status IN ('QUEUED','PROCESSING','RETRY_WAIT')).
c. Если активная задача найдена, но она «зависла» (статус QUEUED не менялся более 60 минут, или статус PROCESSING не менялся более 10 минут), задача принудительно переводится в FAILED с error_code = 'TIMEOUT_STALE', а в лог пишется событие уровня ERROR с event_type = task.stale_killed.
d. После этого (или если активной задачи изначально не было) система выставляет free_interpretation_pending = false, free_interpretation_pending_since = NULL.
e. Записывается событие уровня WARN с event_type = free_reservation.stale_reset, указав visitor_id и pending_since.

7. Очистка зависших платежей. Задача app.tasks.cancel_stale_payments запускается Celery Beat каждый час в очереди payment_tasks. Задача находит записи в generated_spreads со статусом AWAITING_PAYMENT или PAYMENT_PENDING, созданные более 24 часов назад, и переводит их в статус CANCELED. При этом заполняется поле cancel_reason = 'TTL_EXPIRED'. В лог пишется событие INFO с event_type = spread.payment_ttl_expired. Это предотвращает бесконечное накопление «брошенных» раскладов.
8. Механизм сбора аналитики. Бэкенд должен фиксировать аналитические события без блокировки основного потока обработки запроса. При возникновении события (вытягивание карты, создание расклада и т.д.) бэкенд отправляет легковесное сообщение в очередь analytics_tasks (или использует паттерн Transactional Outbox с быстрой вставкой в таблицу analytics_events без ожидания коммита, если СУБД справляется).
   Строго запрещено выполнять сложные агрегационные SQL-запросы к таблице analytics_events в момент записи события. Для отображения статистики в админ-панели используются Materialized Views, которые обновляются фоновой задачей Celery Beat (не чаще раза в 5 минут).
9. Мониторинг генерации чеков. Задача app.tasks.reconcile_receipts запускается Celery Beat каждые 15 минут в очереди payment_tasks. Задача проверяет таблицу receipt_jobs. Если находит записи со статусом created (или failed), созданные более 15 минут назад, она формирует уведомление (алерт) администратору. Это позволяет оперативно выявить ситуацию, когда сервис yookassa-to-mynalog упал, потерял связь с ФНС или не смог создать чек.
10. Очистка просроченных резервов очереди. Задача app.tasks.cleanup_expired_reservations запускается Celery Beat каждую минуту в очереди payment_tasks. Задача ищет в Redis записи о резервах (хранятся в виде Sorted Set с временем истечения) старше 15 минут, удаляет их и выполняет DECR queue_reservations:paid, освобождая место для новых платежей. В лог пишется событие INFO с event_type = queue.reservation_expired.
11. Анонимизация данных по запросу пользователя. Задача app.tasks.anonymize_user_data запускается в очереди payment_tasks при

нажатии админом кнопки «Обработано» в заявке на удаление ПДн. Задача принимает visitor_id и email, и в рамках одной транзакции заменяет email на заглушку в generated_spreads и support_tickets, обнуляет fingerprint в anonymous_visitors. В лог пишется событие INFO с event_type = privacy.user_anonymized.

Модель данных (база данных)

В качестве СУБД используется PostgreSQL 16+. Все первичные ключи (PK) — UUIDv4. Все таблицы содержат системные поля created_at и updated_at (timestamp with time zone, UTC).

Ограничения уникальности (Soft Delete): Поскольку в системе используется мягкое удаление (is_deleted = true), стандартные ограничения UNIQUE не применяются. Вместо них на уровне PostgreSQL создаются частичные уникальные индексы (Partial Unique Indexes) с условием WHERE is_deleted = false. Это позволяет создавать новые сущности с названиями, которые уже принадлежат удаленным записям, и защищает от Race Condition на уровне базы данных.

Примечание по резервированию слотов: Счетчик queue_reservations:paid и метаданные резервов (время истечения) хранятся в Redis (например, в виде Sorted Set). В PostgreSQL эта информация не сохраняется, так как является оперативной и не требует исторической персистентности.

Основные сущности (Таблицы):

1. admin_users (Администраторы):
    a. id (PK, UUID),
    b. email (unique),
    c. password_hash,
    d. totp_secret (nullable),
    e. is_active (bool).
2. admin_sessions (Сессии админов):
    a. id (PK, UUID);
    b. admin_id (FK);
    c. token_hash (string);
    d. csrf_token (string) — токен для защиты от CSRF (Synchronizer Token), генерируется при создании сессии;
    e. expires_at (timestamptz);
    f. ip_address (string);
    g. user_agent (string).
3. password_reset_tokens (Сброс паролей):
    a. id (PK, UUID),
    b. admin_id (FK),
    c. token_hash,
    d. expires_at,
    e. is_used (bool).
4. 2fa_backup_codes (Резервные коды 2FA):

    a. id (PK, UUID),
    b. admin_id (FK),
    c. code_hash,
    d. is_used (bool).
5. anonymous_visitors (Анонимные пользователи):
    a. visitor_id (PK, UUIDv4);
    b. fingerprint (string, nullable) — хэш браузерного отпечатка, прошедший серверное хэширование с солью (хранится в виде строки, например, SHA-256). NULL, если пользователь отказался от сбора данных;
    c. fingerprint_updated_at (timestamptz, nullable) — дата последнего обновления отпечатка (обновляется при каждом успешном распознавании пользователя);
    d. free_interpretation_used (bool, default false) — бесплатное толкование успешно сгенерировано хотя бы один раз;
    e. free_interpretation_pending (bool, default false) — бесплатное право зарезервировано под активную задачу;
    f. free_interpretation_pending_since (timestamptz, nullable) — время установки резерва (для cleanup-задачи);
    g. cookie_consent_given (bool, default false);
    h. consent_at (timestamptz, nullable);
    i. first_visit_at (timestamptz);
    j. last_visit_at (timestamptz).

    Инварианты:
    - free_interpretation_used = true ⇒ free_interpretation_pending = false;
    - одновременное used=false и pending=false означает «бесплатное право доступно»;
    - pending=true может стоять только при наличии активной бесплатной задачи в generated_spreads либо не дольше 2 часов (после чего сбрасывается cleanup-задачей). Зависшие задачи (в статусе QUEUED > 60 мин или PROCESSING > 10 мин) принудительно переводятся в FAILED, чтобы разблокировать резерв.

    Поле id в этой таблице не используется. Все внешние ключи и API-контракты ссылаются на visitor_id.
6. decks (Колоды):
    a. id (PK, UUID),
    b. name (string, NOT NULL),
    c. author, type (enum),
    d. max_cards (int),
    e. allow_reversed (bool),
    f. suits (JSONB, nullable),
    g. description (text),
    h. is_deleted (bool, default false),
    i. created_at, updated_at.

    Для поля name создается частичный уникальный индекс: CREATE UNIQUE INDEX idx_decks_name_active ON decks(name) WHERE is_deleted = false.
7. cards (Карты):
    a. id (PK, UUID),
    b. deck_id (FK),

c. name, suit (nullable),
d. short_description (text),
e. full_description (text),
f. number (int, nullable),
g. reversed_short_description (text, nullable),
h. reversed_full_description (text, nullable),
i. image_url_thumb,
j. image_url_medium,
k. image_url_original (строки с URL S3),
l. is_deleted (bool, default false),
m. created_at,
n. updated_at.

Ограничения (частичные уникальные индексы):
CREATE UNIQUE INDEX idx_cards_number_active ON cards(deck_id, number) WHERE is_deleted = false AND number IS NOT NULL;
CREATE UNIQUE INDEX idx_cards_name_active ON cards(deck_id, name) WHERE is_deleted = false;

8. spreads (Шаблоны раскладов):
    a. id (PK, UUID),
    b. name (string, NOT NULL),
    c. description (text),
    d. amount (int),
    e. positions (JSONB),
    f. info (text, nullable),
    g. tags (JSONB, NOT NULL) — массив объектов вида [{"normalized_name": "строка", "display_name": "строка"}],
    h. image_url (text, NOT NULL),
    i. favorite_deck_ids (JSONB, nullable),
    j. is_deleted (bool, default false),
    k. created_at,
    l. updated_at.

    Для поля name создается частичный уникальный индекс: CREATE UNIQUE INDEX idx_spreads_name_active ON spreads(name) WHERE is_deleted = false.
    Для поля tags создается GIN-индекс для быстрого поиска по normalized_name.
9. generated_spreads (Сгенерированные расклады / Снимки):
    a. id (PK, UUIDv4) — в API и внешних ключах используется как generated_spread_id,
    b. visitor_id (FK, UUID, NOT NULL) — ссылается на anonymous_visitors.visitor_id, ON DELETE RESTRICT,
    c. spread_id (FK),
    d. deck_id (FK),
    e. cards_snapshot (JSONB),
    f. generation_status (enum: AWAITING_PAYMENT, PAYMENT_PENDING, QUEUED, PROCESSING, RETRY_WAIT, COMPLETED, FAILED, FAILED_VALIDATION, CANCELED),
    g. is_free_request (bool, default false),
    h. email (string, nullable),
    i. email_consent_at (timestamp, nullable),

    j. user_context (text, nullable),
    k. interpretation_requested_at (timestamp, nullable),
    l. interpretation_text (text, nullable),
    m. tokens_input (int, nullable) — количество затраченных входных токенов,
    n. tokens_output (int, nullable) — количество затраченных выходных токенов,
    o. error_code (string, nullable) — код ошибки, если задача упала (например, TIMEOUT_STALE, LLM_CONTEXT_OVERFLOW),
    p. is_viewed (bool, default false) — флаг просмотра результата пользователем,
    q. email_delivered (bool, default false) — флаг успешной отправки письма,
    r. email_delete_after (timestamp, nullable) — дата анонимизации email,
    s. created_at (timestamp, NOT NULL DEFAULT NOW()),
    t. updated_at (timestamp, NOT NULL DEFAULT NOW()),
    u. cancel_reason (string, nullable) — причина отмены расклада. Возможные значения: TTL_EXPIRED (истекло 24 часа), ADMIN_MANUAL (отменено администратором). Поле заполняется только если generation_status = CANCELED.
10. payments (Платежи):
    a. id (PK, UUIDv4);
    b. generated_spread_id (FK, UUID, NOT NULL) — ссылается на generated_spreads.id, ON DELETE RESTRICT;
    c. visitor_id (FK, UUID, NOT NULL) — ссылается на anonymous_visitors.visitor_id, ON DELETE RESTRICT;
    d. yookassa_id (unique);
    e. amount (numeric);
    f. status (enum: pending, succeeded, canceled, refunded); created_at (timestamp);
    g. email (string, NOT NULL) — email пользователя, указанный при оплате. Хранится бессрочно/дольше, чем в generated_spreads, для соблюдения требований 422-ФЗ.
    h. offer_agreed_at (timestamptz) — дата и время подтверждения согласия пользователя с офертой при создании платежа;
    i. offer_version (string, NOT NULL) — версия оферты (например, '1.0'), которая была актуальна в момент создания платежа;
    j. possible_duplicate_payment (bool, default false) — выставляется в true, если успешная оплата пришла по раскладу, для которого уже был бесплатный запрос или уже сгенерирован текст. Является прямым указанием администратору для оформления возврата средств (деньги списаны, но услуга не будет оказана повторно). Автоматический возврат через API запрещен, требуется ручное действие в ЛК ЮKassa.

    payments.visitor_id заполняется при создании платежа из generated_spreads.visitor_id и далее не изменяется.
    Поле spread_id в платежах не используется; используется только generated_spread_id.
11. receipt_jobs (Чеки ФНС):
    a. id (PK, UUID),
    b. payment_id (FK, UNIQUE — защита от дублей чеков),

    c. provider (string),
    d. status (enum: created, failed, annulled, not_required),
    e. external_receipt_id (string),
    f. email (string, NOT NULL) — копируется из payments.email в момент создания чека для гарантированной отправки фискального документа.
    g. attempts (int),
    h. last_error (text).
12. interpretation_prices (История цен):
    a. id (PK, UUID),
    b. amount (numeric),
    c. admin_id (FK — кто изменил цену),
    d. is_active (bool),
    e. created_at (timestamptz).
13. prompt_versions (Версии промптов):
    a. id (PK, UUID),
    b. text (text),
    c. admin_id (FK),
    d. created_at.
14. about_page_content (Контент "О проекте" и FAQ):
    a. id (PK, UUID),
    b. content_blocks (JSONB),
    c. faq (JSONB).
15. support_tickets (Обращения):
    a. id (PK, UUID);
    b. visitor_id (nullable, FK);
    c. name (nullable);
    d. email (string, nullable);
    e. contact_channel (string);
    f. telegram_username (string, nullable);
    g. subject (string, nullable);
    h. message (text);
    i. status (enum: NEW, IN_PROGRESS, RESOLVED);
    j. is_deleted (boolean, default false);
    k. deleted_at (timestamptz, nullable);
    l. deleted_by (FK, UUID, nullable) — ID администратора, удалившего обращение;
    m. created_at, updated_at;
    n. email_delete_after (timestamptz, nullable) — дата, после которой email подлежит анонимизации. Вычисляется как now() + interval '12 months' при каждом сохранении или обновлении email.

    Если пользователь выбирает канал связи Telegram (contact_channel = 'telegram'), поле email может быть пустым, но поле telegram_username становится обязательным для заполнения (на уровне валидации бэкенда: CHECK (email IS NOT NULL OR telegram_username IS NOT NULL)).
16. logs (Логи и аудит):
    a. id (UUID),
    b. timestamp (timestamptz),
    c. level (enum),

    d. event_type (string),
    e. actor_id (UUID, nullable) — идентификатор актора (admin_users.id для администраторов или anonymous_visitors.visitor_id для пользователей),
    f. source_ip (string),
    g. success (bool),
    h. entity_type (string, nullable),
    i. entity_id (UUID, nullable),
    j. description (text),
    k. payload (JSONB).

   (Ограничение: PRIMARY KEY (id, timestamp) — требуется для партиционирования).
17. outbox_events (События Outbox):
    a. id (PK, UUIDv4);
    b. event_type (string);
    c. payload (JSONB);
    d. status (enum: PENDING, PROCESSED, FAILED) — FAILED означает, что исчерпаны попытки автоматической отправки, и требуется ручное вмешательство администратора через админ-панель;
    e. retry_count (int, NOT NULL, default 0) — количество неудачных попыток отправки в Redis;
    f. next_retry_at (timestamp, nullable) — время, когда диспетчер должен предпринять следующую попытку отправки (используется для экспоненциальной задержки);
    g. last_error (text, nullable) — текст последней ошибки при отправке;
    h. celery_task_id (string, nullable) — ID задачи, возвращенный Celery для отслеживания;
    i. created_at (timestamptz);
    j. processed_at (timestamptz, nullable).
18. support_ticket_comments (История комментариев к обращениям):
    a. id (PK, UUID),
    b. ticket_id (FK),
    c. admin_id (FK — кто оставил комментарий),
    d. text (text),
    e. created_at (timestamptz).
19. data_deletion_requests (Запросы на удаление ПДн):
    a. id (PK, UUIDv4);
    b. visitor_id (FK, UUID, nullable) — ссылается на anonymous_visitors.visitor_id, ON DELETE SET NULL;
    c. email (string, nullable) — оригинальный email. После обработки заявки заменяется на NULL или заглушку; email_hash (string, NOT NULL) — криптографический хэш (SHA-256) от оригинального email. Хранится бессрочно для аудита и предотвращения повторных злоупотреблений;
    d. status (enum: PENDING, COMPLETED, REJECTED);
    e. rejection_reason (string, nullable) — причина отказа (например, «422-ФЗ»);
    f. created_at (timestamptz);
    g. processed_at (timestamptz, nullable).
20. analytics_events (События для статистики):
    a. id (PK, UUIDv4);

b. visitor_id (FK, UUID, nullable) — ссылается на anonymous_visitors.visitor_id, ON DELETE SET NULL;
c. event_type (string, NOT NULL) — тип события (visit, card_draw, spread_generate, interpretation_request, payment_success, result_view);
d. deck_id (FK, UUID, nullable) — если событие связано с колодой;
e. spread_id (FK, UUID, nullable) — если событие связано с шаблоном расклада;
f. generated_spread_id (FK, UUID, nullable) — если событие связано с конкретным снимком;
g. amount (numeric, nullable) — для финансовых событий;
h. metadata (JSONB, nullable) — дополнительные данные (например, {"is_free": true});
i. created_at (timestamptz, NOT NULL, default now()).

21. legal_documents (Юридические документы):
    a. id (PK, UUIDv4);
    b. slug (string) — системный идентификатор (например, offer, refund-policy, user-agreement).
    c. title (string) — заголовок документа;
    d. content (text) — HTML/Markdown текст документа;
    e. version (string) — версия документа (например, «1.0», «1.1»);
    f. is_active (bool, default true) — флаг актуальной версии (только одна версия для каждого slug может быть активной);
    g. updated_at (timestamptz).

    (Примечание: тексты документов кэшируются в Redis на 24 часа, инвалидация кэша происходит при сохранении новой версии через админ-панель).
    Индекс для таблицы: CREATE UNIQUE INDEX idx_legal_docs_active_slug ON legal_documents(slug) WHERE is_active = true;

Примечание: Все таблицы содержат системные поля created_at и updated_at (timestamptz, UTC), за исключением таблицы logs, которая использует поле timestamp для корректного партиционирования.

Таблицы payments и receipt_jobs являются критически важными для финансового аудита. Для них должен быть гарантирован RPO ≤ 15 минут за счёт механизма PITR. Рекомендуется вынести эти таблицы в отдельную файловую группу (tablespace) на диске с повышенной надежностью (например, RAID 10 или NVMe SSD с репликацией), если это позволяет инфраструктура.

Таблица logs должна быть защищена от изменения и удаления на уровне прав пользователя БД (роль приложения имеет права только на INSERT и SELECT для этой таблицы). При записи логов генерации (llm.request) система должна сохранять количество подсчитанных входных токенов (до и после обрезки), чтобы администратор мог видеть, на каком этапе сработал лимит.

Таблица analytics_events является append-only (запрещены UPDATE и DELETE на уровне прав БД). Таблица партиционируется по месяцам (как и логи), чтобы не деградировать при росте. Создаются индексы: (event_type, created_at), (visitor_id, created_at), (deck_id), (spread_id).

Для обеспечения заявленных метрик производительности (RPS) в базе данных должны быть созданы следующие индексы:

- decks: индекс на is_deleted (для быстрой выборки активных колод).
- cards: составной индекс на (deck_id, is_deleted) и индекс на (deck_id, suit).
- spreads: GIN-индекс на поле tags (JSONB) для быстрого поиска раскладов по тегам.
- payments: индексы на status и created_at (для быстрого формирования истории платежей и статистики); idx_payments_visitor_id по полю visitor_id (для фильтрации и поиска в админ-панели).
- logs: индексы на timestamp (партиционирование), level, event_type. Дополнительный составной индекс на (actor_id, timestamp) для быстрого поиска истории действий конкретного администратора (аудит).

Спецификация API

Общие правила API:

1. Используется архитектурный стиль REST. Формат обмена данными — application/json. Все эндпоинты, кроме вебхуков и публичных GET-запросов, требуют авторизации (через сессионные cookies для админа или visitor_id для пользователя).
2. Для всех публичных эндпоинтов visitor_id не принимается в теле запроса или query-параметрах. Бэкенд извлекает visitor_id только из подписанного visitor_token, переданного в HttpOnly cookie.
3. В административных эндпоинтах visitor_id может использоваться как query-параметр точного поиска. В ответах публичного API поле visitor_id не возвращается. В ответах административного API платежей поле visitor_id возвращается.
4. Сортировка и поиск: Все эндпоинты, возвращающие списки (особенно в /api/admin/*), должны поддерживать параметры:
    a. ?search=<строка> — полнотекстовый поиск по определенным полям (указаны в каждом эндпоинте).
    b. ?sort_by=<поле> — поле для сортировки. Допустимые значения строго ограничены белым списком в каждом эндпоинте (для предотвращения SQL-инъекций).
    c. ?sort_order=<asc|desc> — направление сортировки (по умолчанию desc для дат, asc для названий).

    Если параметр sort_by передан с недопустимым значением, API возвращает 400 VALIDATION_ERROR.

CSRF-защита:

1. Все запросы к административному API (/api/admin/*), кроме GET и эндпоинтов авторизации, должны содержать заголовок X-CSRF-Token, значение которого было получено при логине.
2. Чтобы обеспечить возможность первого POST-запроса (например, POST /api/consent), бэкенд обязан устанавливать cookie csrf_token при обработке

любого начального публичного GET-запроса (например, при запросе главной страницы, GET /api/config/price или GET /api/decks). Фронтенд считывает эту куку при загрузке страницы и использует её для всех последующих POST-запросов.

3. Все публичные POST-запросы (/api/spreads/generate, /api/cards/draw, /api/support/tickets и т.д.) должны содержать заголовок X-CSRF-Token, значение которого совпадает с cookie csrf_token.
4. При ошибке валидации CSRF-токена API возвращает HTTP 403 Forbidden с error.code = CSRF_TOKEN_INVALID.

Стандарты API-контракта (OpenAPI 3.1):

Разработчик обязан вести актуальную OpenAPI-спецификацию. Документация Swagger UI должна быть доступна по адресу /api/docs (в production закрыта Basic Auth).

1. Обертка ответов: Все успешные ответы (HTTP 2xx) оборачиваются в объект {"data": ...}. Для списков используется объект с пагинацией.
2. Пагинация: Все эндпоинты, возвращающие списки, принимают query-параметры ?page=1&limit=20 (по умолчанию page=1, limit=20, максимальный limit=100). Ответ имеет вид json:

```
{
  "data": [ ... ],
  "meta": {
    "total_items": 150,
    "total_pages": 8,
    "current_page": 1,
    "limit": 20
  }
}
```

3. Строгая валидация (Mass Assignment Protection): Во всех схемах запросов (Pydantic / DTO) должен быть установлен запрет на передачу лишних полей (extra = "forbid"). Если клиент передает поле, не описанное в схеме (например, пытается изменить is_deleted или id), API возвращает 400 VALIDATION_ERROR.

Перечень основных эндпоинтов:

1. Публичное API (Пользовательская часть)
Конфигурация и контент:

- GET /api/config/price — получить актуальную стоимость AI-толкования. При вызове этого эндпоинта (если кука отсутствует) бэкенд устанавливает cookie csrf_token.
- GET /api/about — получить контент страницы «О проекте» и блок FAQ. В ответе также передаются реквизиты самозанятого (для отображения в футере):

```
json

{
  "data": {
```

```
    "content_blocks": [...],
    "faq": [...],
    "taxpayer_info": {
      "name": "Иванов Иван Иванович",
      "inn": "123456789012",
      "status": "Плательщик налога на профессиональный доход"
    }
  }
}
```

- GET /api/legal/{slug} — получить текст актуальной версии юридического документа (например, GET /api/legal/offer). Возвращает { "data": { "slug": "offer", "title": "Публичная оферта", "content": "<html>...</html>", "version": "1.0", "updated_at": "..." } }
- GET /api/health — проверка состояния сервиса.
- Возвращает 200 OK со статусом {"status": "ok"} и 503 Service Unavailable при проблемах с БД или критическими зависимостями. Используется системами мониторинга. Не требует авторизации.

Идентификация и согласия:

- POST /api/consent — принимает { "consent": true/false, "fingerprint_hash": "строка | null" }.
    - Если consent: true и передан fingerprint_hash, бэкенд обновляет cookie_consent_given = true, consent_at = now(), сохраняет хэш отпечатка (с солью) и (если это первый запрос) активирует возможность бесплатного толкования.
    - Если consent: false, бэкенд обновляет cookie_consent_given = false, а поле fingerprint обнуляет (устанавливает в NULL). Ранее сгенерированный fingerprint_hash больше не используется для идентификации.
- GET /api/visitors/me — получить статус текущего анонимного пользователя. Возвращает{ "visitor_id": "...", "free_interpretation_used": false, "free_interpretation_pending": true, "active_free_generated_spread_id": "uuid-... | null", "cookie_consent_given": true }.
  active_free_generated_spread_id возвращается, если free_interpretation_pending = true и есть активная бесплатная задача. Используется фронтендом для отображения кнопки «Перейти к активному запросу».

Колоды и Карты:

- GET /api/decks — получить список активных колод (с автором и кратким описанием, без карт).
- GET /api/decks/{deck_id}/cards — получить список карт конкретной колоды (для просмотра значений). Поддерживает фильтр ?suit=....
- GET /api/cards/{card_id} — получить подробную информацию о конкретной карте.
- POST /api/cards/draw — вытянуть случайную карту. Принимает { "deck_id": "uuid", "suit": "string | null" }. Возвращает объект выпавшей карты.

Теги и Расклады:

- GET /api/tags — получить список всех уникальных тегов раскладов.
  Бэкенд выполняет запрос к таблице spreads, распаковывает JSONB-массив tags и возвращает уникальный список (уникальность проверяется по normalized_name):

```
json
{
  "data": [
    {"normalized_name": "отношения", "display_name": "Отношения"},
    {"normalized_name": "финансы", "display_name": "Финансы"},
    {"normalized_name": "любовь", "display_name": "Любовь"}
  ]
}
```

- GET /api/spreads — получить список доступных раскладов. Поддерживает query-параметры ?tag=... (фильтр по тегу). Параметр tag принимает normalized_name (например, ?tag=отношения). Бэкенд выполняет поиск по JSONB-массиву: tags @> '[{"normalized_name": "отношения"}]'::jsonb.
- POST /api/spreads/generate — создать новый сгенерированный расклад (вытянуть карты). Принимает: spread_id, deck_id. Возвращает: generated_spread_id, список выпавших карт с позициями (снимок).
- POST /api/spreads/{generated_spread_id}/interpretation — запросить AI-толкование. Принимает: email, user_context, personal_data_consent.

  Валидация: Если длина поля user_context превышает 1500 символов, API должно вернуть ошибку VALIDATION_ERROR (400). Если поле email не соответствует формату RFC 5322, возвращается 422 VALIDATION_ERROR.

  Логика работы:

  1. Валидация согласия на обработку ПДн:
     Бэкенд проверяет поле personal_data_consent в теле запроса. Если оно равно false или отсутствует, система возвращает HTTP 422 (ошибка VALIDATION_ERROR) с сообщением: «Для получения толкования необходимо согласие на обработку персональных данных (email)». В лог пишется событие уровня WARN с event_type = security.pd_consent_missing и visitor_id нарушителя. Обработка запроса на этом прекращается.
  2. SELECT ... FROM generated_spreads WHERE id = $1 FOR UPDATE. При несовпадении visitor_id (взятого из токена) с generated_spreads.visitor_id — возвращается 404 NOT_FOUND.
  3. Идемпотентные проверки (статус задачи):
     a. Если interpretation_text уже есть → 200 { "data": { "status": "COMPLETED" } }.
     b. Если generation_status IN ('QUEUED', 'PROCESSING', 'RETRY_WAIT') → 200 { "data": { "status": "<current_status>" } }.
  4. Проверка наличия успешной оплаты: Бэкенд проверяет таблицу payments. Если для данного generated_spread_id существует запись со статусом succeeded, система пропускает все проверки бесплатного

лимита и переходит сразу к шагу 7 (постановка в очередь QUEUED), так как услуга уже оплачена.

5. SELECT ... FROM anonymous_visitors WHERE visitor_id = $visitor_id FOR UPDATE.
6. Если free_interpretation_used = true → Бэкенд переводит generated_spreads.generation_status в PAYMENT_PENDING и возвращает HTTP 402 Payment Required со строго стандартизированной структурой ошибки:

```
json

{
  "error": {
    "code": "PAYMENT_REQUIRED",
    "message": "Для получения толкования требуется оплата",
    "details": {
      "status": "PAYMENT_PENDING",
      "price": "159.00"
    }
  }
}
```

(Фронтенд берет цену из error.details.price для отображения на экране оплаты).

7. Если free_interpretation_pending = true:
    a. Если есть активная бесплатная задача для другого generated_spread_id → возвращается HTTP 409 Conflict с ошибкой FREE_INTERPRETATION_PENDING.

```
json
{
  "error": {
    "code": "FREE_INTERPRETATION_PENDING",
    "message": "У вас уже есть активный запрос на бесплатное толкование",
    "details": { "active_free_generated_spread_id": "..." }
  }
}
```

    b. Если активной задачи нет (резерв завис) → сбросить резерв в БД (pending=false, pending_since=NULL), записать WARN в лог и продолжить как свежий запрос.
8. Постановка в очередь: Выставить free_interpretation_pending=true (если запрос бесплатный), free_interpretation_pending_since=now(), перевести generated_spreads.generation_status='QUEUED', установить is_free_request=true (если запрос бесплатный), сохранить email, email_consent_at, user_context, interpretation_requested_at; вставить запись в outbox_events.
9. Выполнить COMMIT и вернуть HTTP 200 OK:

```
json
{
  "data": {
    "status": "QUEUED",
```

```
    "requires_payment": false,
    "price": null
  }
}
```

Коды ответов:

- 200 — успех или идемпотентный повтор (возвращает статус COMPLETED, QUEUED, PROCESSING или RETRY_WAIT).
- 402 — PAYMENT_REQUIRED (требуется оплата, бэкенд перевел расклад в PAYMENT_PENDING).
- 404 — NOT_FOUND (расклад не найден или принадлежит другому пользователю).
- 409 — FREE_INTERPRETATION_PENDING (бесплатное право уже зарезервировано под другой расклад).
- 422 — VALIDATION_ERROR (некорректный email, отсутствует согласие на обработку персональных данных или превышен лимит user_context).
- 503 — QUEUE_OVERFLOW (только для бесплатных запросов при переполненной очереди).
- 500 — INTERNAL_ERROR (непредвиденная ошибка сервера).

- GET /api/spreads/{generated_spread_id}/status — проверить статус генерации AI-толкования (для polling). Возвращает:

```
json

{
  "data": {
    "status": "FAILED_VALIDATION",
    "queue_position": null,
    "estimated_wait_seconds": null,
    "error_code": "LLM_PROHIBITED_CONTENT",
    "cancel_reason": null
  }
}
```

(Поле is_high_load: true также передается внутри объекта data, если очередь переполнена).

Описание полей:

- status — текущий статус задачи (QUEUED, PROCESSING, COMPLETED, FAILED, FAILED_VALIDATION).
- queue_position — позиция задачи в очереди (целое число). Возвращается null, если задача уже находится в статусе PROCESSING или COMPLETED.
- estimated_wait_seconds — ожидаемое время ожидания в секундах. Рассчитывается динамически по формуле: (Количество задач в очереди перед пользователем) * 30 сек. Возвращается null, если задача уже в работе (PROCESSING) или выполнена.
- is_high_load — флаг перегрузки очереди (возвращается true, если очередь превышает MAX_HARD_QUEUE_LIMIT).

    - error_code — код ошибки (возвращается только если статус FAILED или FAILED_VALIDATION). Позволяет фронтенду определить причину сбоя (например, TIMEOUT_STALE, LLM_CONTEXT_OVERFLOW) и предложить пользователю соответствующий сценарий (повторить запрос или запросить возврат средств).
    - cancel_reason — причина отмены (возвращается только если статус CANCELED).

  Текст толкования этот эндпоинт не возвращает.
- GET /api/spreads/{generated_spread_id}/view?token={result_access_token} — получить текст толкования. Защищен: принимает JWT-токен result_access_token в query-параметре. Бэкенд проверяет:
    - подпись токена (HMAC/JWT с секретом JWT_SECRET_KEY);
    - срок действия (exp);
    - соответствие generated_spread_id внутри токена и в URL.

  Проверка visitor_id из cookie для этого эндпоинта не требуется, что позволяет открывать ссылку на любом устройстве.
  Действие на сервере: При успешном возврате данных (HTTP 200) этот эндпоинт атомарно обновляет поле is_viewed = true для указанного generated_spread_id, если оно еще не было установлено. Возвращает { "data": { "interpretation_text": "...", "created_at": "..." } }.
  Важно: Параметр token является конфиденциальным. При логировании запросов на уровне бэкенда (FastAPI) значение токена должно маскироваться (заменяться на [REDACTED]).
- GET /api/spreads/latest-unviewed — запросить баннер на главной с датой генерации и masked_email.

  Логика:
  1. Бэкенд ищет в таблице generated_spreads запись, которая удовлетворяет условиям:
     a. visitor_id совпадает с текущим.
     b. generation_status = 'COMPLETED'.
     c. is_viewed = false.
  2. Возвращается самый последний по created_at. Если таких записей нет, возвращается 200 { "data": null }.

  Возвращает:

```
json

{
  "data": {
    "generated_spread_id": "uuid-...",
    "created_at": "2026-10-28T12:00:00Z",
    "masked_email": "a***@mail.ru"
  }
}
```

- POST /api/spreads/{id}/verify-email — подтвердить email для просмотра непросмотренного расклада. Принимает { "email": "..." }. Если email совпадает,

бэкенд генерирует JWT-токен (срок жизни 1 час) и возвращает его вместе с текстом толкования. Также ставит is_viewed = true.

Коды ответов:

- 200 OK — email совпал, возвращается токен и текст.
- 400 VALIDATION_ERROR — некорректный формат email.
- 403 FORBIDDEN — email не совпадает (или лимит в 3 неверные попытки исчерпан).
- 429 RATE_LIMITED — превышен лимит попыток (5 на расклад в час / 10 с IP в час). В заголовке Retry-After передается количество секунд до сброса лимита.

Платежи и Обратная связь:

- POST /api/payments/create — создать платеж в ЮKassa. Принимает: generated_spread_id (UUID, обязательный); email (string, обязательный); expected_amount (numeric, обязательный) — сумма в рублях, которую пользователь видел на экране; offer_agreed (boolean, обязательный) — должно быть true. Если передано false или поле отсутствует, возвращается ошибка VALIDATION_ERROR (400) с сообщением: «Необходимо согласие с условиями оферты».
  Логика работы:
  1. Бэкенд извлекает актуальную цену из таблицы interpretation_prices (где is_active = true), применяя блокировку SELECT FOR SHARE.
  2. Если expected_amount не совпадает с актуальной ценой БД, возвращается 409 CONFLICT с { "error": { "code": "PRICE_CHANGED", "message": "Цена изменилась", "details": { "new_amount": 199.00 } } }.
  3. Если суммы совпадают, создается платеж в ЮKassa на сумму expected_amount и возвращается confirmation_url.
  4. При успешном создании платежа бэкенд сохраняет в таблицу payments email (переданный в запросе), отметку offer_agreed_at = now() и текущую версию оферты offer_version (из таблицы legal_documents где slug = 'offer'), чтобы зафиксировать, на каких условиях был заключен договор.

  Коды ответов:
  - 200 — успех.
  - 400 — VALIDATION_ERROR (некорректный формат email, отсутствует согласие с офертой или переданы лишние поля).
  - 409 — PRICE_CHANGED.
  - 503 — QUEUE_OVERFLOW.
- POST /api/support/tickets — отправить обращение через форму «Свяжитесь с нами». Принимает: email, contact_channel, subject, message.
  Коды ответов:
  - 200 — успех.
  - 400 — VALIDATION_ERROR (некорректные данные формы).
  - 429 — RATE_LIMITED (превышен лимит отправки обращений).
  - 500 — INTERNAL_ERROR.
- POST /api/payments/verify — принудительно проверить актуальный статус платежа в ЮKassa (вызывается фронтендом при возврате пользователя по

return_url). Принимает: { "payment_id": "uuid" } (локальный ID платежа или yookassa_id).
Важно: Этот эндпоинт также не выполняет запрос к нейросети и не сохраняет текст. Он только гарантирует, что при возврате пользователя задача гарантированно попадет в очередь.

Логика работы:

1. Бэкенд проверяет текущий статус платежа в локальной БД. Если он уже succeeded, эндпоинт сразу возвращает актуальный статус generated_spreads (например, QUEUED или PROCESSING).
2. Если статус pending, бэкенд отправляет GET-запрос к API ЮKassa (/v3/payments/{yookassa_id}) для получения актуального статуса.
3. Если ЮKassa возвращает succeeded, бэкенд применяет ту же самую логику, что и при обработке вебхука: открывает транзакцию, блокирует строку (SELECT FOR UPDATE), обновляет статус платежа на succeeded, меняет статус расклада на QUEUED и создает запись в outbox_events.
4. Возвращает фронтенду:

```
json

{
  "data": {
    "payment_status": "succeeded",
    "generation_status": "PAYMENT_PENDING"
  }
}
```

5. Если payment_status = canceled, фронтенд должен разблокировать кнопку «Оплатить» и показать сообщение: «Оплата не была завершена. Вы можете попробовать снова».

Ответы:

- 200 — успех (статус проверен).
- 404 — платеж не найден.
- 502 — ошибка при обращении к API ЮKassa (фронтенд должен показать сообщение «Не удалось проверить статус, обновите страницу»).

2. Вебхуки (Webhooks)

- POST /api/webhooks/yookassa — эндпоинт для приема уведомлений от ЮKassa (payment.succeeded, payment.canceled). Валидация по IP и повторная проверка через API ЮKassa.

3. Административное API (Защищенное, доступно по адресу /api/admin)
Ко всем эндпоинтам, возвращающим списки, применяется общая пагинация: ?page=1&limit=20

Авторизация и безопасность:

- POST /api/admin/auth/login — вход в админ-панель (возвращает HTTP-only cookie).
  При успешном входе (HTTP 200) возвращает:

```
json

{
  "data": {
    "admin_id": "...",
    "email": "...",
    "csrf_token": "строка-токена"
  }
}
```

Фронтенд сохраняет csrf_token в памяти и использует его в заголовке X-CSRF-Token для всех последующих изменяющих запросов. Сессионная cookie устанавливается автоматически с флагами HttpOnly, Secure, SameSite=Strict. Примечание для фронтенда: Полученный csrf_token сохраняется в памяти (например, в состоянии React/Vue). При перезагрузке страницы (F5) фронтенд должен выполнить запрос GET /api/admin/auth/csrf для восстановления токена, так как сессионная cookie является HttpOnly и недоступна из JS.

- POST /api/admin/auth/logout — выход из системы.
- POST /api/admin/auth/reset-password — запрос на сброс пароля. Принимает: { "email": "string" }.
  Логика работы:
  1. Бэкенд проверяет наличие email в базе данных.
  2. Если найден — генерирует токен, сохраняет его (срок жизни 1 час) и отправляет письмо.
  3. Если не найден — не делает ничего (кроме логирования).
  4. В обоих случаях возвращается ответ 200 OK со строго одинаковым телом:

```
json

{
  "data": {
    "message": "Если аккаунт с таким email существует, мы отправили инструкцию по восстановлению пароля."
  }
}
```

Коды ответов:

- 200 OK — запрос обработан (независимо от существования email). Возвращается также при превышении лимитов (для защиты от перебора).
- 429 RATE_LIMITED — возвращается только в случае явного подозрения на DDoS (например, при превышении жесткого глобального лимита по IP). В большинстве случаев при превышении лимитов на email система имитирует успешный ответ 200 OK.
- 500 INTERNAL_ERROR — ошибка сервера или сервиса отправки почты.

- POST /api/admin/auth/reset-password/confirm — подтверждение сброса пароля (принимает токен и новый пароль).
  Коды ответов:
  - 200 OK — пароль успешно изменён.

    - 400 VALIDATION_ERROR — пароль не соответствует политике безопасности.
    - 403 FORBIDDEN (INVALID_TOKEN) — токен недействителен или истёк. (Формулировка ошибки должна быть общей, чтобы не раскрывать, был ли токен использован ранее или его вообще не существовало).

- POST /api/admin/auth/2fa/setup — сгенерировать QR-код и секретный ключ для 2FA.
- POST /api/admin/auth/2fa/verify — проверить 6-значный код и активировать 2FA (возвращает резервные коды).
- POST /api/admin/auth/2fa/disable — отключить 2FA (требует ввода пароля и кода).
- GET /api/admin/auth/csrf — получить актуальный CSRF-токен для текущей сессии.
  Используется фронтендом при перезагрузке страницы (F5), чтобы восстановить токен в памяти. Бэкенд проверяет валидность сессионной cookie (admin_session) и возвращает:

```
json
{ "data": { "csrf_token": "строка-токена" } }
```

  Если сессия невалидна — возвращается 401 UNAUTHORIZED.

Управление колодами и картами:

- GET /api/admin/decks — список колод. Сортировка: sort_by (name, created_at), поиск: search (по name, author). Фильтры: ?is_deleted=false (по умолчанию), ?search=... (поиск по названию).
- POST /api/admin/decks — создание колоды.
- PUT / DELETE /api/admin/decks/{id} — редактирование / мягкое удаление колоды.
- Коды ответов:
    - 200 — успех.
    - 400 VALIDATION_ERROR — некорректные данные.
    - 409 CONFLICT (SUIT_IN_USE) — при попытке удалить масть, которая используется в активных картах колоды.
- POST /api/admin/decks/{id}/restore — восстановить удаленную колоду.
- GET /api/admin/cards — список карт. Фильтры: ?deck_id=... (обязательный), ?is_deleted=false, ?suit=..., ?search=....
- POST /api/admin/cards — создание карты.
  Коды ответов:
    - 200 / 201 — успех.
    - 400 VALIDATION_ERROR — некорректные данные (включая несоответствие масти deck.suits).
    - 404 NOT_FOUND — колода не найдена.
- PUT / DELETE /api/admin/cards/{id} — редактирование / мягкое удаление карты.
- Коды ответов для обновления карты:
    - 200 / 201 — успех.
    - 400 VALIDATION_ERROR — некорректные данные (включая несоответствие масти deck.suits).

    - 404 NOT_FOUND — колода не найдена.
- POST /api/admin/cards/{id}/restore — восстановить удаленную карту.
- POST /api/admin/cards/upload — массовая загрузка карт. Принимает multipart/form-data с файлом .zip (для CSV+картинки) или файлом .json.

Управление раскладами, промптами и контентом:

- GET /api/admin/spreads — список раскладов. Фильтры: ?is_deleted=false, ?tag=..., ?search=....
- POST /api/admin/spreads — создание расклада.
- PUT / DELETE /api/admin/spreads/{id} — редактирование / мягкое удаление расклада.
- POST /api/admin/spreads/{id}/restore — восстановить удаленный расклад.
- GET /api/admin/prompts — получить текущий мастер-промпт.
- PUT /api/admin/prompts — сохранить новую версию мастер-промпта.
- GET /api/admin/prompts/versions — получить список всех сохраненных версий.
- POST /api/admin/prompts/{id}/rollback — откатить промпт к указанной версии.
- GET / PUT /api/admin/about — получить / обновить основной текст и блок FAQ страницы «О проекте».
- GET /api/admin/legal-docs — получить список всех юридических документов с текущими версиями и текстами.
- PUT /api/admin/legal-docs/{slug} — обновить текст документа (создать новую версию). Принимает title и content (Markdown). Бэкенд деактивирует старую версию и создает новую. Не позволяет изменить slug или удалить документ.
- GET /api/admin/legal-docs/{slug}/history — получить историю версий документа. Возвращает массив объектов, отсортированный по убыванию updated_at:

```
json

{ "data": [
  { "id": "uuid", "version": "1.2", "is_active": true, "updated_at": "...", "content": "..." },
  { "id": "uuid", "version": "1.1", "is_active": false, "updated_at": "...", "content": "..." }
] }
```

- POST /api/admin/generated-spreads/{id}/cancel — ручная отмена «зависшего» снимка расклада. Доступно только для снимков в статусах AWAITING_PAYMENT или PAYMENT_PENDING. Бэкенд переводит generation_status в CANCELED, заполняет поле cancel_reason = 'ADMIN_MANUAL', пишет событие generated_spread.manual_cancel в лог аудита (с указанием ID админа) и возвращает 200 OK.

Финансы и операции:

- GET /api/admin/prices — получить текущую активную цену.
- POST /api/admin/prices — установить новую цену.
- GET /api/admin/prices/history — получить историю изменения цен (кто и когда менял).
- GET /api/admin/payments — история платежей (с фильтрами и пагинацией). сортировка: sort_by (created_at, amount, status), поиск: search (по yookassa_id, email, visitor_id). Фильтры (query):
    - status (succeeded, canceled, refunded) — точное совпадение.
    - date_from, date_to — диапазон дат (включительно).

    - search — строка для поиска. Бэкенд должен выполнять поиск по полям: yookassa_id (точное), visitor_id (точное), generated_spread_id (точное), email (частичное, без учета регистра), amount (точное).
    - ?possible_duplicate=true — фильтрация только платежей, требующих возврата (флаг possible_duplicate_payment = true).
- POST /api/admin/receipts/{id}/retry-annul — принудительно повторить попытку аннулирования чека в «Мой Налог» (вызывается администратором только если автоматический процесс через yookassa-to-mynalog завершился ошибкой). Принимает ID чека (receipt_jobs.id). Не инициирует сам возврат денег, а только повторяет отмену чека.
- GET /api/admin/deletion-requests — список запросов на удаление ПДн.
- POST /api/admin/deletion-requests/{id}/process — обработать запрос на удаление ПДн. Принимает: status (enum: COMPLETED, REJECTED, обязательный); rejection_reason (string, обязательный, если status = REJECTED).

  Логика:
  1. Бэкенд обновляет статус заявки и processed_at = now().
  2. Если status = REJECTED, сохраняет rejection_reason.
  3. Если status = COMPLETED, бэкенд запускает фоновую задачу anonymize_user_data (см. раздел «Воркеры»).
  4. Бэкенд заменяет значение поля email в этой заявке на NULL (анонимизация самой заявки).
  5. Возвращает 200 OK с обновленной карточкой заявки.

Поддержка и обратная связь:

- POST /api/support/tickets — (Публичный API) отправить обращение. Принимает: email, name (опционально), contact_channel, telegram_username (опционально), subject (опционально), message.
- GET /api/admin/support/tickets — список обращений. Сортировка: sort_by (created_at, status), поиск: search (по email, name, message). Фильтры: ?status=NEW, ?date_from=..., ?date_to=..., ?search=..., ?is_deleted=false (по умолчанию false — показываются только активные). Для просмотра архива передается ?is_deleted=true.
- GET /api/admin/support/tickets/new-count — вернуть количество новых обращений (для бейджа).
- GET /api/admin/support/tickets/{id} — получить полную карточку обращения, включая историю комментариев.
- PUT /api/admin/support/tickets/{id} — изменить статус обращения.
- POST /api/admin/support/tickets/{id}/comments — добавить внутренний комментарий к обращению. Принимает: { "text": "..." }.
- DELETE /api/admin/support/tickets/{id} — мягкое удаление обращения. Бэкенд не удаляет запись физически, а устанавливает is_deleted = true, deleted_at = now(), deleted_by = admin_id. Возвращает 200 OK (или 204 No Content). Записывает событие support_ticket.deleted в лог.
- POST /api/admin/support/tickets/{id}/restore — восстановить удаленное обращение. Снимает флаг is_deleted, очищает deleted_at и deleted_by. Записывает событие support_ticket.restored в лог.

Мониторинг и статистика:

- GET /api/admin/stats — получение агрегированных данных для дашборда статистики. Query-параметры:
    - period — временной диапазон (1, 3, 7, 30 дней).
    - deck_type / spread_id — фильтры для контентных метрик.
    - payment_status — фильтр для финансовых метрик.

  Источники данных для бэкенда:
    - Бизнес-метрики (воронка, платежи, токены) — из PostgreSQL.
    - Инфраструктурные метрики (RPS, длина очереди) — из Redis (для очереди llm_tasks) или из логов Nginx.

  Бэкенд использует заранее подготовленные Materialized Views (MV):
    - mv_daily_active_users — для DAU/WAU/MAU.
    - mv_conversion_funnel — для воронки.
    - mv_deck_popularity — для популярности колод.
    - mv_spread_popularity — для популярности раскладов.
    - mv_financial_metrics — для маржинальности и ARPPU (на основе payments).

  При обновлении MV (раз в 5 минут) используется синтаксис REFRESH MATERIALIZED VIEW CONCURRENTLY, чтобы не блокировать чтение.

- GET /api/admin/stats/export — выгрузка статистики в формате CSV/XLSX.
- GET /api/admin/logs — просмотр логов. Сортировка: sort_by (timestamp, level), поиск: search (по description, entity_id). Query-параметры:
    - ?date_from=..., ?date_to=...
    - ?level=ERROR
    - ?event_type=...
    - ?actor_id=<uuid> — фильтрация по ID администратора (или посетителя).
    - ?is_audit=true — фильтрация только событий аудита (изменение данных).
    - ?search=... (по description или entity_id).
- GET /api/admin/dlq — список задач в Dead Letter Queue.
- POST /api/admin/dlq/{task_id}/retry — перезапустить упавшую задачу.
- DELETE /api/admin/dlq/{task_id} — удалить задачу из DLQ без перезапуска.
- GET /api/admin/outbox — список событий из таблицы outbox_events (фильтры: ?status=FAILED).
- POST /api/admin/outbox/{id}/retry — сбросить статус события на PENDING и обнулить retry_count.

Диаграмма состояний задач (State Machine)

Каждая задача (снимок расклада), требующая обращения к нейросети, имеет строгий жизненный цикл, который хранится в поле generation_status таблицы generated_spreads.

1. Начальные состояния:

- AWAITING_PAYMENT (Ожидает оплаты): Расклад создан (карты вытянуты), но пользователь еще не запросил толкование.

2. Переходы при запросе толкования (Бесплатный сценарий):
   - AWAITING_PAYMENT → QUEUED: Пользователь отправил форму, бесплатное право доступно. Бэкенд резервирует право (free_interpretation_pending = true) и пишет задачу в outbox_events.

3. Переходы при запросе толкования (Платный сценарий):
   - AWAITING_PAYMENT → PAYMENT_PENDING: Бесплатное право использовано (или ранее была ошибка генерации). Бэкенд возвращает PAYMENT_REQUIRED и переводит расклад в ожидание оплаты.
   - PAYMENT_PENDING → QUEUED: Пришел вебхук payment.succeeded. Платёж обновлен, задача добавлена в outbox_events.
   - AWAITING_PAYMENT → CANCELED: TTL (24 часа). Бэкенд выставляет cancel_reason = 'TTL_EXPIRED'.

4. Состояния выполнения:
   - QUEUED (В очереди): Задача находится в Redis.
   - QUEUED → PROCESSING: Воркер забрал задачу в работу.
   - PROCESSING → RETRY_WAIT: Ошибка API 429/503 (ожидание exponential backoff).
   - RETRY_WAIT → PROCESSING: Пауза прошла, воркер повторяет запрос.
   - PROCESSING → COMPLETED: Успешный ответ сохранен в БД.
   - PROCESSING → FAILED: Исчерпаны все попытки (3–5 раз) при ошибках API, таймауте или получении 4xx.
   - PROCESSING → FAILED_VALIDATION: Исчерпаны попытки (3 раза) при проверке ответа нейросети ИЛИ Ошибка переполнения контекста (LLM_CONTEXT_OVERFLOW, 0 попыток).

5. Восстановление после сбоев:
   - PROCESSING / QUEUED → QUEUED: Если воркер потерял задачу (Docker перезапустил контейнер), благодаря task_acks_late = True задача возвращается в очередь.
   - QUEUED / PROCESSING → FAILED: Если задача «зависла» (не меняла статус длительное время: >60 мин для QUEUED или >10 мин для PROCESSING), фоновая задача cleanup_stale_free_reservations принудительно завершает её с ошибкой TIMEOUT_STALE, чтобы разблокировать бесплатный резерв.
   - FAILED / FAILED_VALIDATION / CANCELED → QUEUED: Перевод происходит при нажатии пользователем кнопки «Повторить запрос» (без новой оплаты) или администратором из DLQ.
     Важно: переход CANCELED → QUEUED разрешен только для задач, отмененных по TTL (24 часа) или вручную администратором. Если платеж был отменен пользователем в ЮKassa, снимок остается в статусе PAYMENT_PENDING, и для повторной попытки создается новый объект платежа.

6. TTL (срок жизни) и очистка:
   - Задачи в статусе AWAITING_PAYMENT переводятся в CANCELED, если с момента их создания прошло более 24 часов.

- Задачи в статусе PAYMENT_PENDING переводятся в CANCELED фоновой задачей cancel_stale_payments, если с момента создания прошло более 24 часов, а платеж так и не был завершен.

Резерв бесплатного права.
При переходах QUEUED/PROCESSING/RETRY_WAIT → COMPLETED (для бесплатного запроса) выставляется free_interpretation_used = true, free_interpretation_pending = false.

При переходах QUEUED/PROCESSING/RETRY_WAIT → FAILED, → FAILED_VALIDATION, → CANCELED (для бесплатного запроса) выставляется free_interpretation_pending = false. Флаг free_interpretation_used не меняется.

Если резерв не был снят по этим переходам (например, воркер потерял задачу), он снимается фоновой задачей cleanup_stale_free_reservations при условии отсутствия активной бесплатной задачи и истечения 2 часов с pending_since.

Визуализация жизненного цикла разбита на три диаграммы: общая концепция (Приложение 6), обработка оплаты и очередей (Приложение 7), работа воркера и ретраи (Приложение 8).

Аутентификация и модель ролей

Роли (RBAC): На этапе MVP в системе существует только одна роль — Admin (полный доступ ко всем разделам). Внедрение ролей с ограниченными правами (например, Content_Manager, Support) вынесено в бэклог.

Механизм аутентификации: Используются сессии на основе HTTP-only Cookies. Токен сессии (JWT или случайная строка) передается в заголовке Set-Cookie с флагами HttpOnly, Secure и SameSite=Strict. Это защищает от кражи токена через XSS. Все активные сессии администраторов, токены для сброса пароля (срок жизни 1 час) и резервные коды 2FA должны храниться в соответствующих таблицах базы данных (см. раздел "Модель данных"). В БД хранятся только хеши токенов и кодов.

Защита от CSRF: Для всех небезопасных методов (POST, PUT, DELETE) в админ-панели должен использоваться CSRF-токен.

Парольная политика: Минимальная длина пароля администратора — 10 символов, обязательное наличие букв разного регистра и цифр. Данное правило применяется как при первичной установке пароля, так и при его сбросе через форму восстановления. Пароли хешируются алгоритмом Argon2id.

Формат ошибок (Единый стандарт)

Все ответы API с HTTP-кодами 4xx и 5xx должны возвращать JSON строго определенного формата:

```
{
```

```
  "error": {
    "code": "STRING_CODE",
    "message": "Понятное сообщение для пользователя на русском языке",
    "details": null
  }
}
```

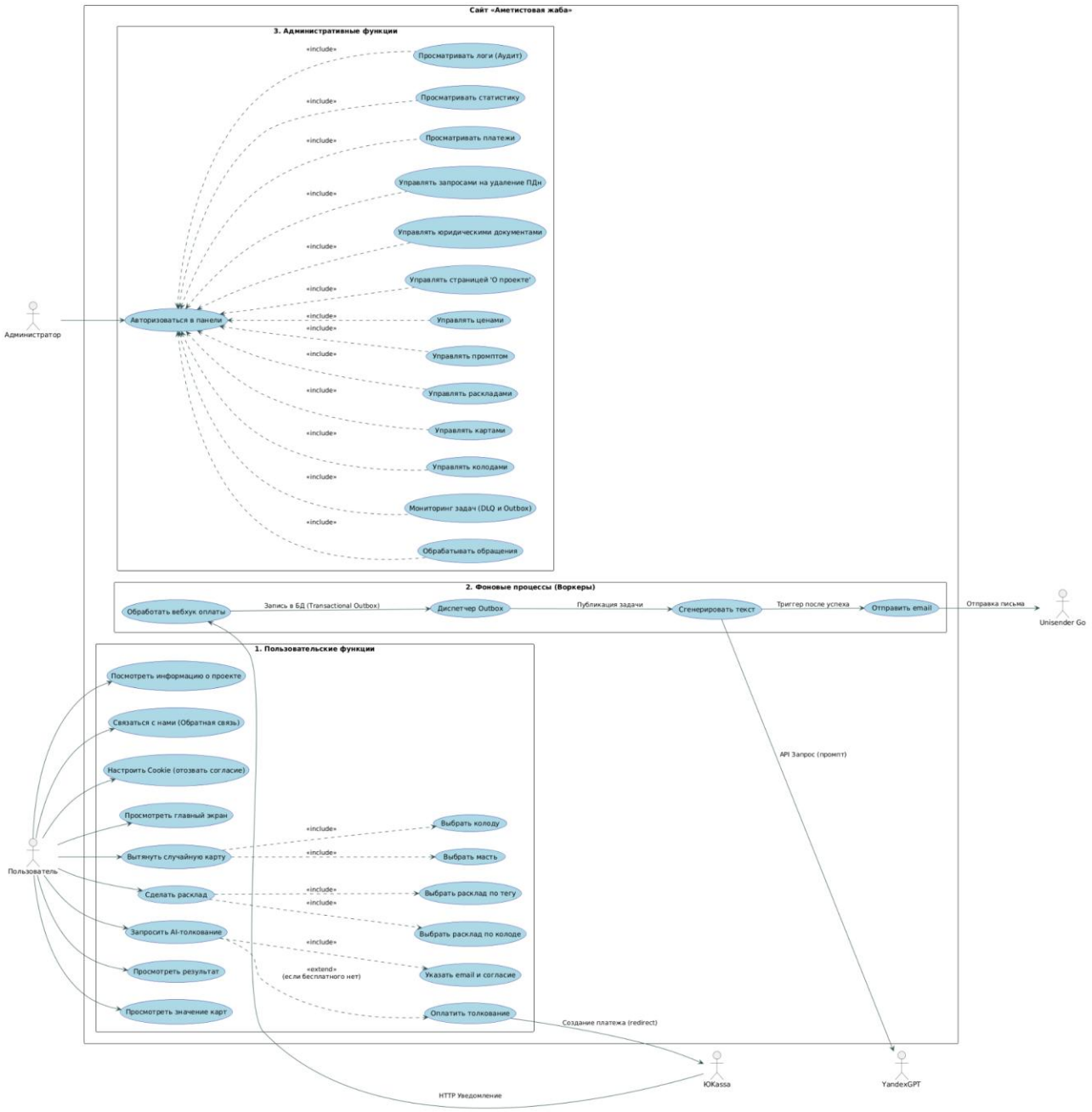
Справочник кодов ошибок (Code):

- VALIDATION_ERROR (400) — неверный формат данных или переданы лишние поля (фронтенд подсвечивает конкретные поля).
- UNAUTHORIZED (401) — сессия истекла или отсутствует валидный токен (фронтенд перенаправляет на логин).
- FORBIDDEN (403) — нет прав на выполнение операции (например, попытка входа в админку без роли).
- CSRF_TOKEN_INVALID (403) — отсутствует или недействителен CSRF-токен. Фронтенд должен перезапросить токен (или перелогинить пользователя) и повторить запрос.
- CONSENT_REQUIRED (403) — пользователь пытается получить бесплатное толкование, но ранее отказался от сбора данных (fingerprint). Фронтенд должен показать модальное окно с предложением дать согласие (и получить бесплатно) или перейти к оплате.
- NOT_FOUND (404) — запрашиваемая сущность не найдена (несуществующий ID).
- CONFLICT (409) — базовый конфликт состояния (например, данные были изменены другим процессом). Фронтенд показывает общее сообщение «Данные устарели, обновите страницу».
- ENTITY_DELETED (409) — запрашиваемая сущность (колода, карта или расклад) была удалена в процессе выполнения операции. Фронтенд показывает сообщение «Выбранная колода была удалена. Пожалуйста, обновите страницу и выберите другую» и предлагает перезагрузить список.
- FREE_INTERPRETATION_PENDING (409) — бесплатное толкование уже зарезервировано под другой расклад. В details.active_free_generated_spread_id возвращается идентификатор активного расклада. Фронтенд показывает кнопку «Перейти к активному запросу».
- NAME_ALREADY_TAKEN (409) — попытка создать, переименовать или восстановить сущность (колоду, карту, расклад) с названием, которое уже занято активной записью. Фронтенд подсвечивает поле ввода названия и показывает сообщение.
- PRICE_CHANGED (409) — актуальная цена в базе данных отличается от суммы, которую запросил фронтенд (expected_amount). В details.new_amount возвращается новая стоимость.
  Действие фронтенда: Показать пользователю модальное окно с подтверждением новой цены. При согласии — сгенерировать новый Idempotence-Key и повторить запрос с новой суммой.
- SUIT_IN_USE (409) — попытка удалить масть из настроек колоды, которая используется в активных картах. Фронтенд показывает сообщение с указанием масти и предлагает сначала переназначить карты.

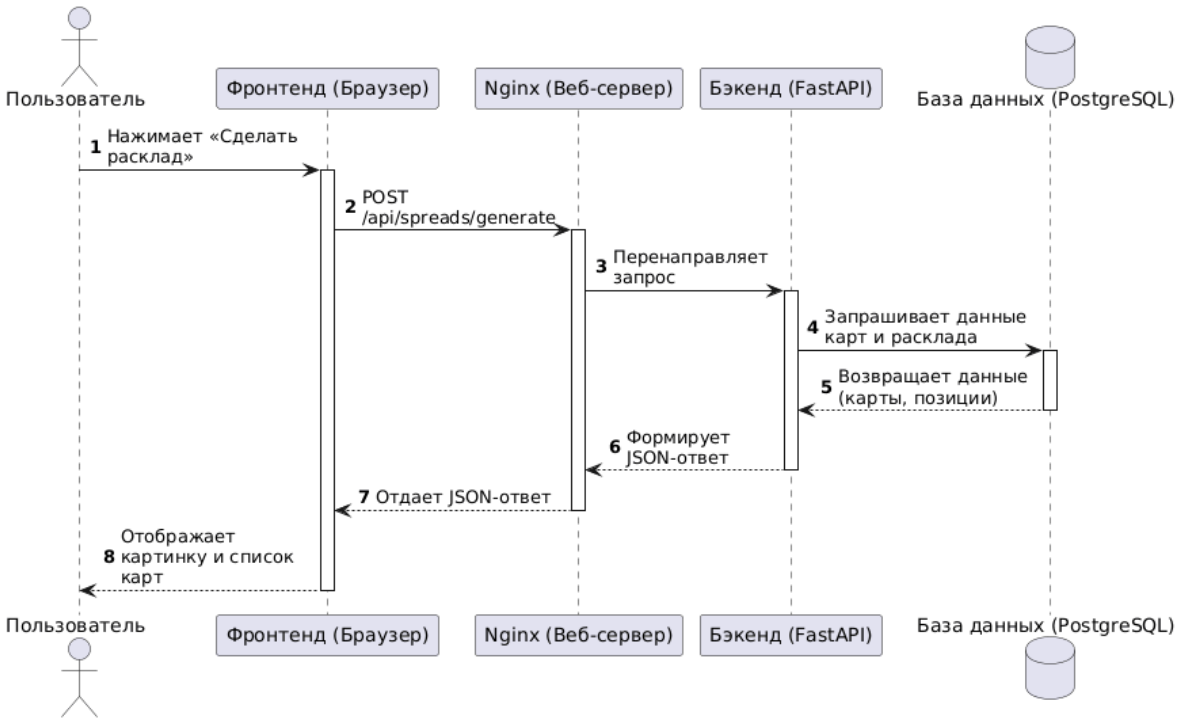
- PAYMENT_REQUIRED (402) — для получения услуги требуется оплата. Возвращается эндпоинтом /interpretation, если бесплатное толкование уже использовано. В теле ответа (в блоке data) передается текущий статус расклада (PAYMENT_PENDING) и актуальная цена.
- LLM_VALIDATION_FAILED (422) — нейросеть выдала невалидный текст (не прошел проверку на длину/язык/стоп-слова).
- LLM_CONTEXT_OVERFLOW (422) — объем переданных данных (контекст пользователя + снимок расклада) превышает лимиты модели даже после применения механизмов обрезки. Задача не может быть выполнена без сокращения входных данных.
- RATE_LIMITED (429) — превышен лимит запросов (фронтенд показывает сообщение «Слишком много запросов, попробуйте позже» и блокирует кнопку на время из заголовка Retry-After).
- INTERNAL_ERROR (500) — непредвиденная ошибка на сервере (технические детали не передаются, ошибка логируется).
- LLM_TIMEOUT (504) — нейросеть не ответила за отведенное время (таймаут 60 секунд).
- QUEUE_OVERFLOW (503) — сервер перегружен, очередь заполнена (фронтенд отображает модальное окно с текстом: «Сейчас очень много желающих получить толкование. Пожалуйста, попробуйте через 10–15 минут. Деньги не списаны»).
- DATABASE_UNAVAILABLE (503) — временная недоступность базы данных. Фронтенд показывает экран ожидания или предлагает повторить попытку позже.
- SERVICE_UNAVAILABLE (503) — базовый код для общей недоступности сервиса (например, при падении Nginx или других критических компонентов).
- LLM_PROHIBITED_CONTENT — сгенерированный нейросетью текст не прошел пост-модерацию на соответствие возрастной маркировке 16+ (содержит сцены насилия, призывы к противоправным действиям и т.д.). Задача переходит в статус FAILED_VALIDATION.

Приложения

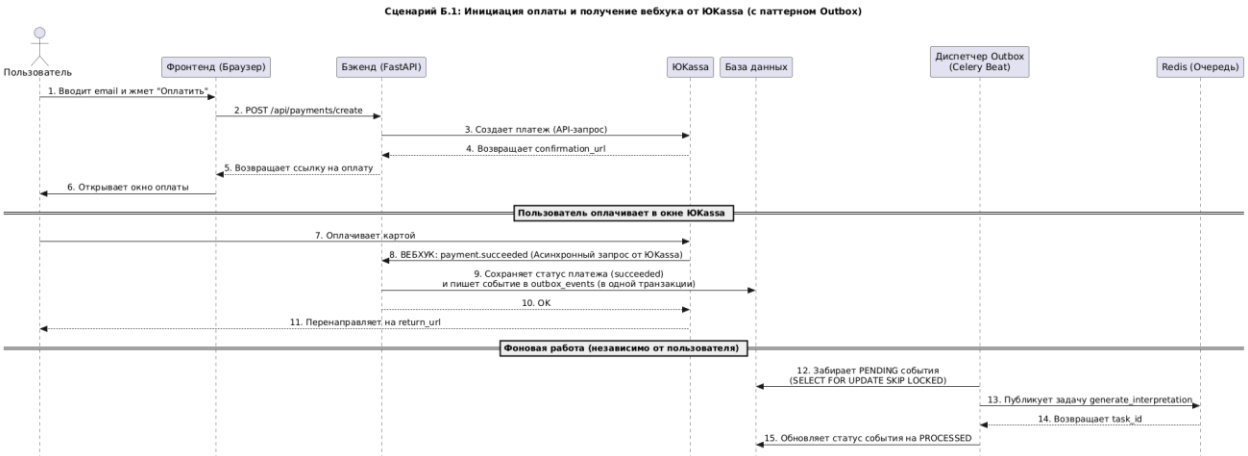
Приложение 1 - UseCase диаграмма



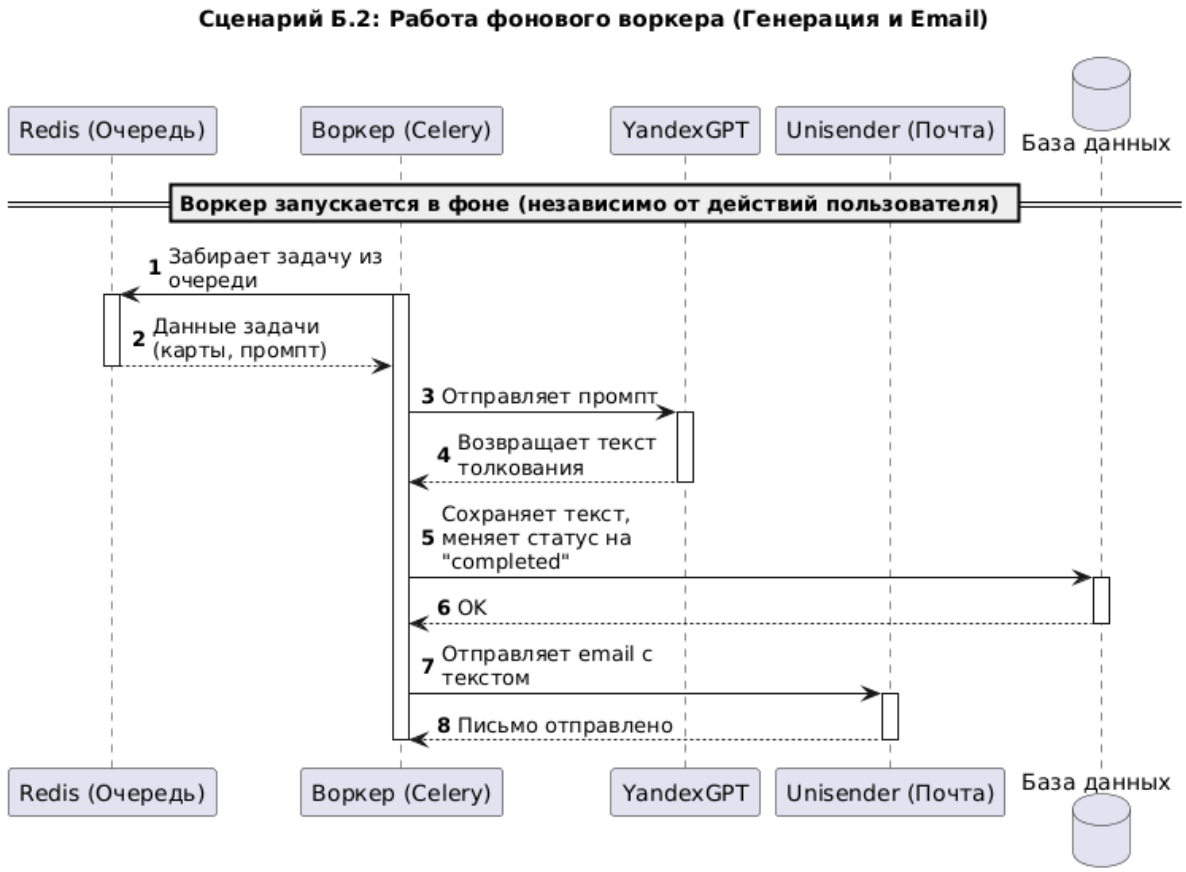
Приложение 2 - UML-диаграмма последовательности: Генерация расклада



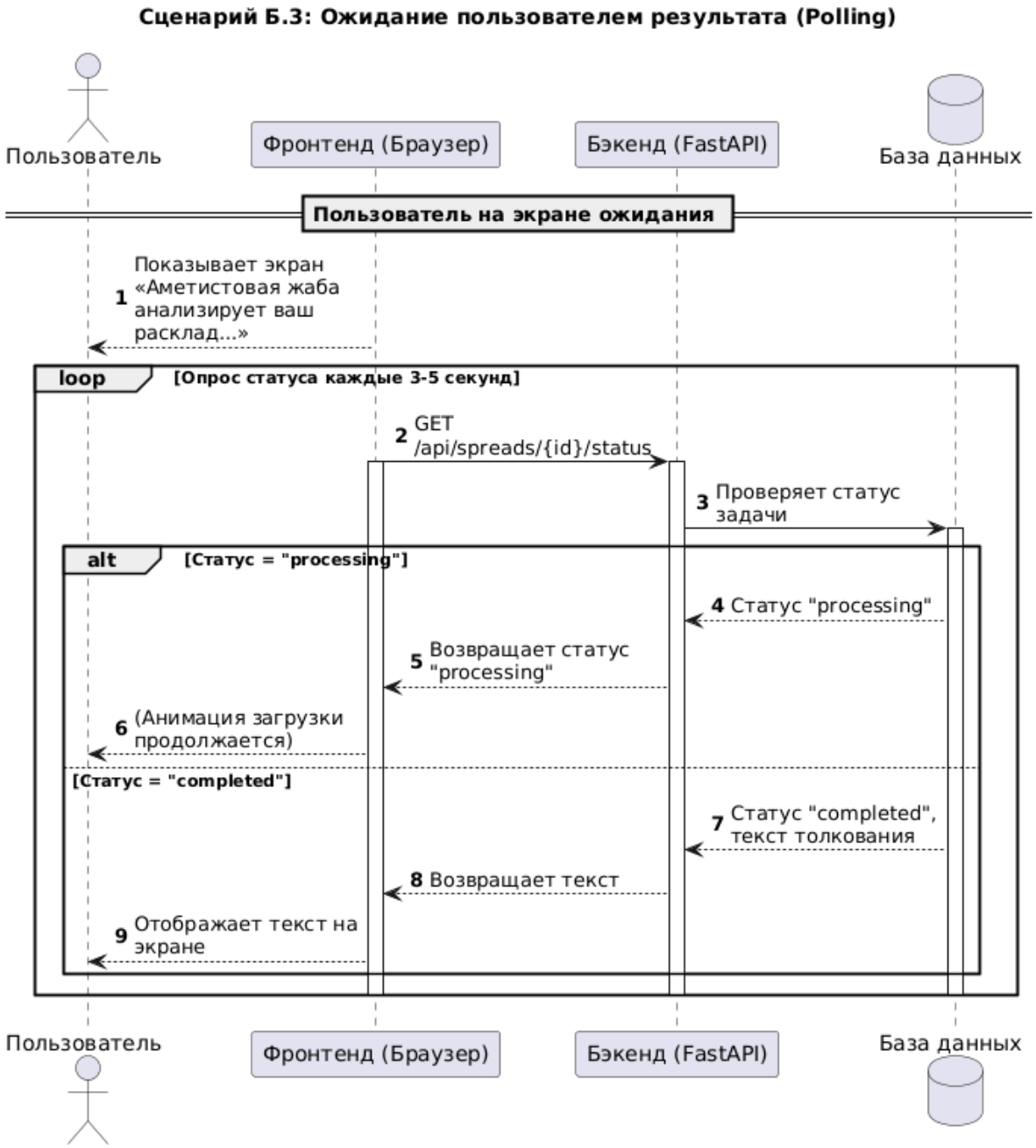
Приложение 3 - UML-диаграмма последовательности: Инициация оплаты и получение вебхука



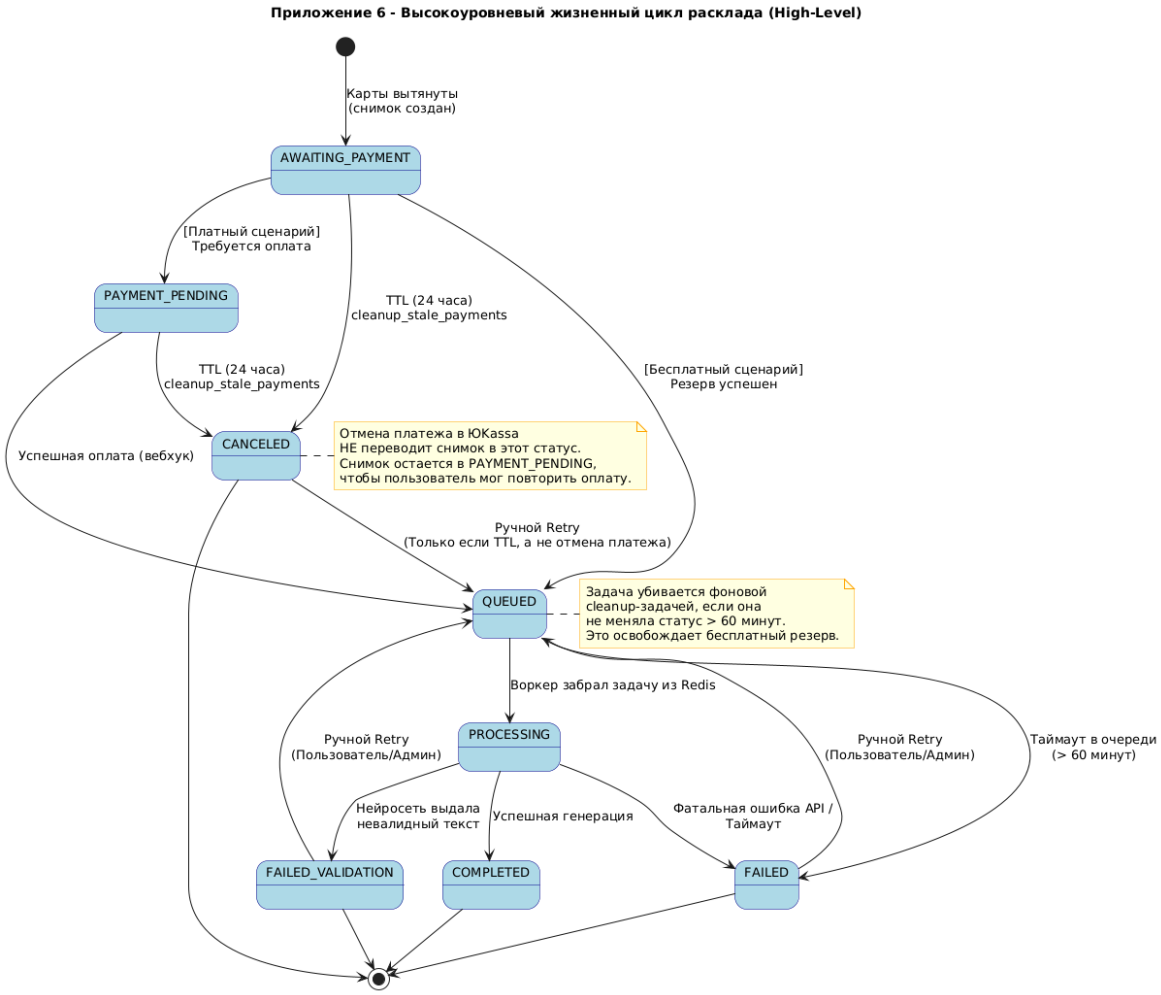
Приложение 4 - UML-диаграмма последовательности: Работа воркера (нейросеть и почта)



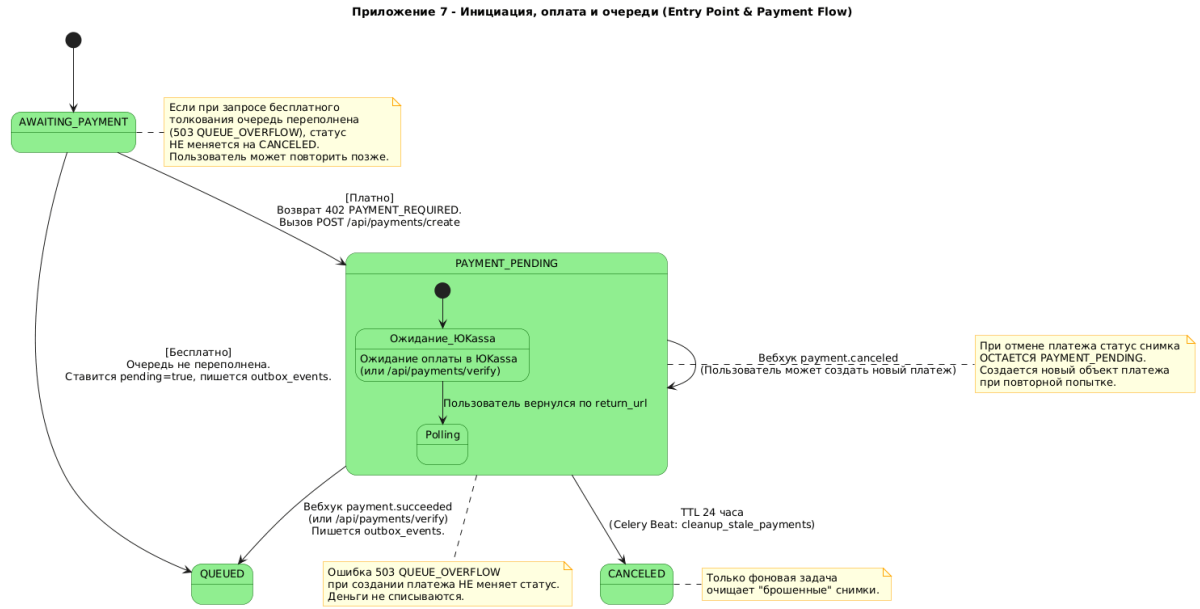
Приложение 5 - UML-диаграмма последовательности: Ожидание и получение результата (Polling)



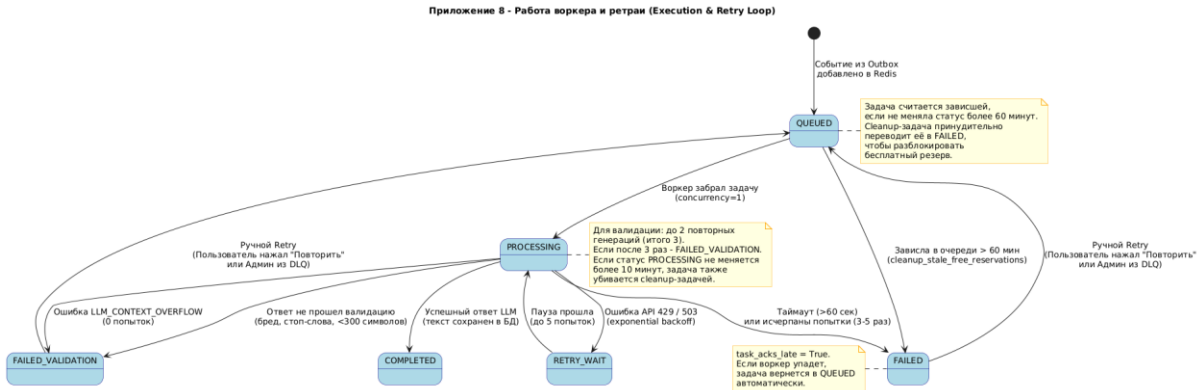
Приложение 6 - Высокоуровневый жизненный цикл расклада (High-Level)



Приложение 7 - Инициация, оплата и очереди (Entry Point & Payment Flow)



Приложение 8 - Работа воркера и ретраи (Execution & Retry Loop)



Приложение 9 - Итоговая схема базы данных (DDL фрагмент)

(Примечание: представлен фрагмент, включающий таблицы, измененные в ходе аудита. Полная DDL-схема должна поддерживаться через систему миграций Alembic).
SQL

```
-- Расширения
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. ENUM типы
CREATE TYPE generation_status_enum AS ENUM (
```

```
    'AWAITING_PAYMENT', 'PAYMENT_PENDING',
    'QUEUED', 'PROCESSING', 'RETRY_WAIT',
    'COMPLETED', 'FAILED', 'FAILED_VALIDATION', 'CANCELED'
);
CREATE TYPE payment_status_enum AS ENUM ('pending', 'succeeded', 'canceled',
'refunded');
CREATE TYPE receipt_status_enum AS ENUM ('created', 'failed', 'annulled',
'not_required');
CREATE TYPE ticket_status_enum AS ENUM ('NEW', 'IN_PROGRESS', 'RESOLVED');
CREATE TYPE deletion_status_enum AS ENUM ('PENDING', 'COMPLETED',
'REJECTED');

-- ========================================
-- 2. АДМИНИСТРАТОРЫ И БЕЗОПАСНОСТЬ
-- ========================================

CREATE TABLE admin_users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    totp_secret VARCHAR(255),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE admin_sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    admin_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE CASCADE,
    token_hash VARCHAR(255) NOT NULL,
    csrf_token VARCHAR(255) NOT NULL, -- M-09
    expires_at TIMESTAMPTZ NOT NULL,
    ip_address VARCHAR(45),
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE password_reset_tokens (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    admin_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE CASCADE,
    token_hash VARCHAR(255) NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    is_used BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE 2fa_backup_codes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    admin_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE CASCADE,
    code_hash VARCHAR(255) NOT NULL,
    is_used BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-- ========================================
-- 3. ПОЛЬЗОВАТЕЛИ И КОНТЕНТ
-- ========================================

CREATE TABLE anonymous_visitors (
    visitor_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    fingerprint VARCHAR(255),
    fingerprint_updated_at TIMESTAMPTZ, -- M-06
    free_interpretation_used BOOLEAN NOT NULL DEFAULT FALSE,
    free_interpretation_pending BOOLEAN NOT NULL DEFAULT FALSE, -- C-02
    free_interpretation_pending_since TIMESTAMPTZ, -- C-02
    cookie_consent_given BOOLEAN NOT NULL DEFAULT FALSE,
    consent_at TIMESTAMPTZ,
    first_visit_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_visit_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE decks (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL,
    max_cards INT NOT NULL DEFAULT 78,
    allow_reversed BOOLEAN NOT NULL DEFAULT FALSE,
    suits JSONB,
    description TEXT NOT NULL,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE UNIQUE INDEX idx_decks_name_active ON decks(name) WHERE is_deleted =
false; -- C-07

CREATE TABLE cards (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    deck_id UUID NOT NULL REFERENCES decks(id) ON DELETE RESTRICT,
    name VARCHAR(255) NOT NULL,
    suit VARCHAR(255),
    short_description TEXT NOT NULL,
    full_description TEXT NOT NULL,
    number INT,
    reversed_short_description TEXT,
    reversed_full_description TEXT,
    image_url_thumb VARCHAR(500),
    image_url_medium VARCHAR(500),
    image_url_original VARCHAR(500),
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Частичные уникальные индексы для карт (C-07)
CREATE UNIQUE INDEX idx_cards_name_active ON cards(deck_id, name) WHERE
is_deleted = false;
CREATE UNIQUE INDEX idx_cards_number_active ON cards(deck_id, number) WHERE
```

```
is_deleted = false AND number IS NOT NULL;

CREATE TABLE spreads (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    amount INT NOT NULL,
    positions JSONB NOT NULL,
    info TEXT,
    tags JSONB NOT NULL, -- M-03 (массив объектов normalized/display)
    image_url VARCHAR(500) NOT NULL,
    favorite_deck_ids JSONB,
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE UNIQUE INDEX idx_spreads_name_active ON spreads(name) WHERE
is_deleted = false; -- C-07
CREATE INDEX idx_spreads_tags_gin ON spreads USING GIN (tags); -- M-03

-- ========================================
-- 4. ГЕНЕРАЦИЯ И ПЛАТЕЖИ
-- ========================================

CREATE TABLE generated_spreads (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    visitor_id UUID NOT NULL REFERENCES anonymous_visitors(visitor_id) ON DELETE
RESTRICT,
    spread_id UUID NOT NULL REFERENCES spreads(id) ON DELETE RESTRICT,
    deck_id UUID NOT NULL REFERENCES decks(id) ON DELETE RESTRICT,
    cards_snapshot JSONB NOT NULL,
    generation_status generation_status_enum NOT NULL DEFAULT
'AWAITING_PAYMENT',
    is_free_request BOOLEAN NOT NULL DEFAULT FALSE,
    email VARCHAR(255),
    email_consent_at TIMESTAMPTZ,
    user_context TEXT,
    interpretation_requested_at TIMESTAMPTZ,
    interpretation_text TEXT,
    tokens_input INT,
    tokens_output INT,
    error_code VARCHAR(50),
    is_viewed BOOLEAN NOT NULL DEFAULT FALSE,
    email_delivered BOOLEAN NOT NULL DEFAULT FALSE,
    email_delete_after TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    cancel_reason VARCHAR(50) NULL
);
CREATE INDEX idx_generated_spreads_status ON
generated_spreads(generation_status);

CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    generated_spread_id UUID NOT NULL REFERENCES generated_spreads(id) ON
DELETE RESTRICT,
    visitor_id UUID NOT NULL REFERENCES anonymous_visitors(visitor_id) ON DELETE
RESTRICT,
    yookassa_id VARCHAR(255) UNIQUE NOT NULL,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    status payment_status_enum NOT NULL DEFAULT 'pending',
    email VARCHAR(255) NOT NULL, -- Добавлено для 422-ФЗ
    possible_duplicate_payment BOOLEAN NOT NULL DEFAULT FALSE,
    offer_agreed_at TIMESTAMPTZ,
    offer_version VARCHAR(20),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_payments_visitor_id ON payments(visitor_id);
CREATE INDEX idx_payments_status ON payments(status);

CREATE TABLE receipt_jobs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    payment_id UUID NOT NULL UNIQUE REFERENCES payments(id) ON DELETE
RESTRICT,
    provider VARCHAR(50) NOT NULL,
    status receipt_status_enum NOT NULL DEFAULT 'created',
    email VARCHAR(255) NOT NULL, -- Добавлено для гарантированной отправки чека
    external_receipt_id VARCHAR(255),
    attempts INT NOT NULL DEFAULT 0,
    last_error TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE interpretation_prices (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    admin_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE RESTRICT,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ========================================
-- 5. КОНТЕНТ, ПОДДЕРЖКА И ЮРИДИЧЕСКИЕ ДОКУМЕНТЫ
-- ========================================

CREATE TABLE prompt_versions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    text TEXT NOT NULL,
    admin_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE RESTRICT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE about_page_content (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    content_blocks JSONB NOT NULL,
    faq JSONB NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

```
);

CREATE TABLE legal_documents (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    slug VARCHAR(50) NOT NULL,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    version VARCHAR(20) NOT NULL DEFAULT '1.0',
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Частичный уникальный индекс: только одна активная версия на документ
CREATE UNIQUE INDEX idx_legal_docs_active_slug ON legal_documents(slug) WHERE
is_active = TRUE;

CREATE TABLE support_tickets (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    visitor_id UUID REFERENCES anonymous_visitors(visitor_id) ON DELETE SET NULL,
    name VARCHAR(255),
    email VARCHAR(255),
    contact_channel VARCHAR(50) NOT NULL,
    telegram_username VARCHAR(255),
    subject VARCHAR(255),
    message TEXT NOT NULL,
    status ticket_status_enum NOT NULL DEFAULT 'NEW',
    is_deleted BOOLEAN NOT NULL DEFAULT FALSE, -- M-07
    deleted_at TIMESTAMPTZ,                    -- M-07
    deleted_by UUID REFERENCES admin_users(id) ON DELETE SET NULL, -- M-07
    email_delete_after TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE support_ticket_comments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    ticket_id UUID NOT NULL REFERENCES support_tickets(id) ON DELETE CASCADE,
    admin_id UUID NOT NULL REFERENCES admin_users(id) ON DELETE RESTRICT,
    text TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE data_deletion_requests (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    visitor_id UUID REFERENCES anonymous_visitors(visitor_id) ON DELETE SET NULL,
    email VARCHAR(255), -- M-08 (зануляется)
    email_hash VARCHAR(255) NOT NULL, -- M-08 (хранится)
    status deletion_status_enum NOT NULL DEFAULT 'PENDING',
    rejection_reason TEXT, -- M-08
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    processed_at TIMESTAMPTZ
);

-- ========================================
```

```
-- 6. СИСТЕМНЫЕ ТАБЛИЦЫ И АНАЛИТИКА
-- ========================================

CREATE TABLE outbox_events (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    retry_count INT NOT NULL DEFAULT 0,
    last_error TEXT,
    celery_task_id VARCHAR(255),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    next_retry_at TIMESTAMPTZ,
    processed_at TIMESTAMPTZ
);
CREATE INDEX idx_outbox_pending ON outbox_events(created_at) WHERE status =
'PENDING';

CREATE TABLE analytics_events (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    visitor_id UUID REFERENCES anonymous_visitors(visitor_id) ON DELETE SET NULL,
    event_type VARCHAR(50) NOT NULL,
    deck_id UUID REFERENCES decks(id) ON DELETE SET NULL,
    spread_id UUID REFERENCES spreads(id) ON DELETE SET NULL,
    generated_spread_id UUID REFERENCES generated_spreads(id) ON DELETE SET
NULL,
    amount NUMERIC(10, 2),
    metadata JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

CREATE INDEX idx_analytics_event_type ON analytics_events(event_type, created_at);
CREATE INDEX idx_analytics_visitor ON analytics_events(visitor_id, created_at);
CREATE INDEX idx_support_tickets_active ON support_tickets(status, created_at)
WHERE is_deleted = false;
CREATE INDEX idx_outbox_failed ON outbox_events(created_at) WHERE status =
'FAILED';

-- Примечание: Таблица logs создается с партиционированием по timestamp.
-- В рамках данного DDL она опущена для краткости, так как ее структура
-- описана в разделе "Модель данных" и требует ручного создания партиций.
```

Приложение 10 - Фрагмент OpenAPI-схемы (Общие компоненты)

Yaml

```
openapi: 3.1.0
info:
  title: Ametistovaya Zhaba API
  description: |
    API для AI-сервиса раскладов Таро «Аметистовая жаба».
    Все успешные ответы (HTTP 2xx) оборачиваются в объект `data`.
    Все ошибки (HTTP 4xx, 5xx) — в объект `error`.
  version: 1.0.0
  contact:
    name: Ильиных Дарья
    email: support@ametistovaya-zhaba.ru

servers:
  - url: https://api.ametistovaya-zhaba.ru/v1
    description: Production server
  - url: http://localhost:8000/v1
    description: Local development

tags:
  - name: Public - Config
    description: Получение настроек и контента
  - name: Public - Auth
    description: Идентификация анонимных пользователей
  - name: Public - Decks & Cards
    description: Просмотр колод и карт
  - name: Public - Spreads
    description: Создание раскладов и получение толкований
  - name: Public - Payments
    description: Оплата толкований
  - name: Public - Support
    description: Обратная связь
  - name: Webhooks
    description: Эндпоинты для внешних систем
  - name: Admin
    description: Административные методы (требуют сессионной cookie и CSRF-
токена)

security:
  - VisitorTokenAuth: []

paths:
  # ========================================
  # Конфигурация и контент
  # ========================================
  /config/price:
    get:
      tags: [Public - Config]
      summary: Получить актуальную цену AI-толкования
      security: []
      responses:
        '200':
          description: Успешно
          content:
            application/json:
```

```
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
                    amount:
                      type: number
                      format: float
                      example: 159.00
                    currency:
                      type: string
                      example: RUB

/about:
  get:
    tags: [Public - Config]
    summary: Получить контент страницы «О проекте»
    security: []
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
                    content_blocks:
                      type: array
                      items:
                        type: object
                    faq:
                      type: array
                      items:
                        type: object
                    taxpayer_info:
                      type: object
                      properties:
                        name:
                          type: string
                        inn:
                          type: string
                        status:
                          type: string

/legal/{slug}:
  get:
    tags: [Public - Config]
    summary: Получить актуальную версию юридического документа
    security: []
    parameters:
```

```
      - name: slug
        in: path
        required: true
        schema:
          type: string
          enum: [offer, refund-policy, user-agreement]
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
                    slug:
                      type: string
                    title:
                      type: string
                    content:
                      type: string
                    version:
                      type: string
                    updated_at:
                      type: string
                      format: date-time

# ========================================
# Идентификация
# ========================================
/consent:
  post:
    tags: [Public - Auth]
    summary: Сохранить согласие на сбор fingerprint
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ConsentRequest'
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
```

```
                message:
                  type: string

/visitors/me:
  get:
    tags: [Public - Auth]
    summary: Получить статус текущего анонимного пользователя
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  $ref: '#/components/schemas/VisitorStatus'

# ========================================
# Колоды и Карты
# ========================================
/decks:
  get:
    tags: [Public - Decks & Cards]
    summary: Получить список активных колод
    security: []
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/DeckResponse'

/decks/{deck_id}/cards:
  get:
    tags: [Public - Decks & Cards]
    summary: Получить список карт конкретной колоды
    security: []
    parameters:
      - name: deck_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
      - name: suit
        in: query
        required: false
```

```
        schema:
          type: string
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/CardResponse'

/cards/draw:
  post:
    tags: [Public - Decks & Cards]
    summary: Вытянуть случайную карту
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/DrawCardRequest'
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  $ref: '#/components/schemas/CardResponse'
      '404':
        $ref: '#/components/responses/NotFound'

# ========================================
# Расклады и Толкование
# ========================================
/tags:
  get:
    tags: [Public - Spreads]
    summary: Получить список всех уникальных тегов
    security: []
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
```

```
            properties:
              data:
                type: array
                items:
                  type: object
                  properties:
                    normalized_name:
                      type: string
                    display_name:
                      type: string

/spreads:
  get:
    tags: [Public - Spreads]
    summary: Получить список доступных раскладов
    security: []
    parameters:
      - name: tag
        in: query
        required: false
        schema:
          type: string
      - name: deck_id
        in: query
        required: false
        schema:
          type: string
          format: uuid
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/SpreadResponse'

/spreads/generate:
  post:
    tags: [Public - Spreads]
    summary: Создать новый сгенерированный расклад (вытянуть карты)
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/SpreadGenerateRequest'
    responses:
      '200':
        description: Успешно
```

```
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  $ref: '#/components/schemas/GeneratedSpreadResponse'
      '400':
        $ref: '#/components/responses/BadRequest'
      '409':
        $ref: '#/components/responses/Conflict'

  /spreads/{generated_spread_id}/interpretation:
    post:
      tags: [Public - Spreads]
      summary: Запросить AI-толкование (бесплатное или платное)
      parameters:
        - name: generated_spread_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/InterpretationRequest'
      responses:
        '200':
          description: Успешно (идемпотентный повтор или постановка в очередь)
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    $ref: '#/components/schemas/InterpretationResponse'
        '402':
          $ref: '#/components/responses/PaymentRequired'
        '403':
          description: Требуется согласие на сбор данных (fingerprint) для использования
бесплатного лимита
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              example:
                error:
                  code: CONSENT_REQUIRED
                  message: Для получения бесплатного толкования необходимо согласие
на обработку данных браузера
                  details: null
```

```
      '404':
        $ref: '#/components/responses/NotFound'
      '409':
        $ref: '#/components/responses/Conflict'
      '429':
        $ref: '#/components/responses/RateLimited'
      '503':
        $ref: '#/components/responses/ServiceUnavailable'

  /spreads/{generated_spread_id}/status:
    get:
      tags: [Public - Spreads]
      summary: Проверить статус генерации AI-толкования (Polling)
      parameters:
        - name: generated_spread_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Успешно
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    $ref: '#/components/schemas/StatusResponse'

  /spreads/{generated_spread_id}/view:
    get:
      tags: [Public - Spreads]
      summary: Получить текст толкования по токену
      security: []
      parameters:
        - name: generated_spread_id
          in: path
          required: true
          schema:
            type: string
            format: uuid
        - name: token
          in: query
          required: true
          description: JWT result_access_token. Конфиденциальный параметр. Подлежит
маскированию в логах.
          schema:
            type: string
      responses:
        '200':
          description: Успешно
          content:
```

```
          application/json:
            schema:
              type: object
              properties:
                data:
                  $ref: '#/components/schemas/ViewResultResponse'
      '403':
        description: Токен недействителен или истек

/spreads/latest-unviewed:
  get:
    tags: [Public - Spreads]
    summary: Запросить баннер на главной с непросмотренным результатом
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: [object, "null"]
                  properties:
                    generated_spread_id:
                      type: string
                      format: uuid
                    created_at:
                      type: string
                      format: date-time
                    masked_email:
                      type: string

/spreads/{id}/verify-email:
  post:
    tags: [Public - Spreads]
    summary: Подтвердить email для просмотра непросмотренного расклада
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email]
            properties:
              email:
                type: string
```

```
            format: email
    responses:
      '200':
        description: Успешно (возвращает токен и текст)
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
                    result_access_token:
                      type: string
                    interpretation_text:
                      type: string
      '400':
        $ref: '#/components/responses/BadRequest'
      '403':
        $ref: '#/components/responses/Forbidden'
      '429':
        $ref: '#/components/responses/RateLimited'

# ========================================
# Платежи
# ========================================
/payments/create:
  post:
    tags: [Public - Payments]
    summary: Создать платеж в ЮKassa
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PaymentCreateRequest'
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PaymentCreateResponse'
      '400':
        $ref: '#/components/responses/BadRequest'
      '409':
        $ref: '#/components/responses/Conflict'
      '503':
        $ref: '#/components/responses/ServiceUnavailable'

/payments/verify:
  post:
    tags: [Public - Payments]
    summary: Принудительно проверить актуальный статус платежа в ЮKassa
```

```
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [payment_id]
            properties:
              payment_id:
                type: string
                format: uuid
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
                    payment_status:
                      type: string
                      enum: [pending, succeeded, canceled, refunded]
                    generation_status:
                      type: string
                      enum: [AWAITING_PAYMENT, PAYMENT_PENDING, QUEUED,
PROCESSING, RETRY_WAIT, COMPLETED, FAILED, FAILED_VALIDATION,
CANCELED]
      '404':
        $ref: '#/components/responses/NotFound'
      '502':
        description: Ошибка при обращении к API ЮKassa

# ============================================
# Поддержка
# ============================================
/support/tickets:
  post:
    tags: [Public - Support]
    summary: Отправить обращение через форму обратной связи
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email, contact_channel, message]
            properties:
              email:
                type: string
                format: email
```

```
            name:
              type: string
            contact_channel:
              type: string
              enum: [email, telegram]
            telegram_username:
              type: string
            subject:
              type: string
            message:
              type: string
    responses:
      '200':
        description: Успешно
      '400':
        $ref: '#/components/responses/BadRequest'
      '429':
        $ref: '#/components/responses/RateLimited'

# ========================================
# Вебхуки
# ========================================
/webhooks/yookassa:
  post:
    tags: [Webhooks]
    summary: Эндпоинт для приема уведомлений от ЮKassa
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              type:
                type: string
                enum: [payment.succeeded, payment.canceled, refund.succeeded]
              object:
                type: object
                properties:
                  id:
                    type: string
                  status:
                    type: string
                  metadata:
                    type: object
                    properties:
                      generated_spread_id:
                        type: string
                      visitor_id:
                        type: string
    responses:
      '200':
        description: Успешно
```

```
      '403':
        description: Ошибка валидации подписи или IP

# ========================================
# Административное API
# ========================================
/admin/auth/login:
  post:
    tags: [Admin]
    summary: Вход в админ-панель
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email, password]
            properties:
              email:
                type: string
              password:
                type: string
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: object
                  properties:
                    admin_id:
                      type: string
                    email:
                      type: string
                    csrf_token:
                      type: string

/admin/auth/csrf:
  get:
    tags: [Admin]
    summary: Восстановить CSRF-токен для текущей сессии
    security:
      - AdminSessionAuth: []
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
```

```
            properties:
              data:
                type: object
                properties:
                  csrf_token:
                    type: string
      '401':
        description: Сессия истекла или невалидна

/admin/payments:
  get:
    tags: [Admin]
    summary: История платежей
    security:
      - AdminSessionAuth: []
      - CsrfTokenAuth: []
    parameters:
      - name: page
        in: query
        schema:
          type: integer
          default: 1
      - name: limit
        in: query
        schema:
          type: integer
          default: 20
      - name: possible_duplicate
        in: query
        description: Фильтр по дублирующим оплатам
        schema:
          type: boolean
      - name: status
        in: query
        schema:
          type: string
          enum: [pending, succeeded, canceled, refunded]
      - name: date_from
        in: query
        schema:
          type: string
          format: date-time
      - name: date_to
        in: query
        schema:
          type: string
          format: date-time
      - name: search
        in: query
        description: Поиск по yookassa_id, visitor_id, generated_spread_id, email
        schema:
          type: string
    responses:
      '200':
```

```
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/AdminPaymentResponse'
                meta:
                  $ref: '#/components/schemas/PaginationMeta'

  /admin/generated-spreads/{id}/cancel:
    post:
      tags: [Admin]
      summary: Ручная отмена зависшего снимка расклада
      security:
        - AdminSessionAuth: []
        - CsrfTokenAuth: []
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Успешно (статус изменен на CANCELED, cancel_reason =
ADMIN_MANUAL)
        '400':
          description: Снимок находится в статусе, который нельзя отменить
        '404':
          $ref: '#/components/responses/NotFound'

  /admin/logs:
    get:
      tags: [Admin]
      summary: Просмотр логов аудита
      security:
        - AdminSessionAuth: []
        - CsrfTokenAuth: []
      parameters:
        - name: actor_id
          in: query
          schema:
            type: string
            format: uuid
        - name: is_audit
          in: query
          schema:
            type: boolean
        - name: search
```

```
        in: query
        schema:
          type: string
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/LogResponse'

/admin/about:
  put:
    tags: [Admin]
    summary: Обновить контент страницы «О проекте»
    security:
      - AdminSessionAuth: []
      - CsrfTokenAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/AboutContentRequest' # Ссылка на схему
    responses:
      '200':
        description: Успешно обновлено

/admin/outbox:
  get:
    tags: [Admin]
    summary: Список событий Outbox (фильтр по статусу FAILED)
    security:
      - AdminSessionAuth: []
      - CsrfTokenAuth: []
    responses:
      '200':
        description: Успешно
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/OutboxEventResponse'

/admin/outbox/{id}/retry:
```

```
    post:
      tags: [Admin]
      summary: Переотправить Outbox-событие
      security:
        - AdminSessionAuth: []
        - CsrfTokenAuth: []
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: Успешно
        '404':
          $ref: '#/components/responses/NotFound'

  /admin/legal-docs/{slug}:
    put:
      tags: [Admin]
      summary: Обновить юридический документ (создать новую версию)
      security:
        - AdminSessionAuth: []
        - CsrfTokenAuth: []
      parameters:
        - name: slug
          in: path
          required: true
          schema:
            type: string
            enum: [offer, refund-policy, user-agreement]
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [title, content]
              properties:
                title:
                  type: string
                content:
                  type: string
                  description: Markdown-текст документа. Санитизация HTML выполняется
на стороне сервера.
      responses:
        '200':
          description: Успешно (возвращает новую версию)
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/LegalDocumentVersionResponse'
```

```
  /admin/legal-docs/{slug}/history:
    get:
      tags: [Admin]
      summary: Получить историю версий юридического документа
      security:
        - AdminSessionAuth: []
        - CsrfTokenAuth: []
      parameters:
        - name: slug
          in: path
          required: true
          schema:
            type: string
            enum: [offer, refund-policy, user-agreement]
      responses:
        '200':
          description: Успешно
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    type: array
                    items:
                      $ref: '#/components/schemas/LegalDocumentVersionResponse'
        '404':
          $ref: '#/components/responses/NotFound'

components:
  securitySchemes:
    VisitorTokenAuth:
      type: apiKey
      in: cookie
      name: visitor_token
    AdminSessionAuth:
      type: apiKey
      in: cookie
      name: admin_session
    CsrfTokenAuth:
      type: apiKey
      in: header
      name: X-CSRF-Token

  responses:
    BadRequest:
      description: Ошибка валидации данных
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
    PaymentRequired:
      description: Требуется оплата (бесплатный лимит исчерпан)
```

```
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
        example:
          error:
            code: PAYMENT_REQUIRED
            message: Для получения толкования требуется оплата
            details:
              status: PAYMENT_PENDING
              price: "159.00"
  NotFound:
    description: Сущность не найдена
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  Forbidden:
    description: Доступ запрещен
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  Conflict:
    description: Конфликт состояния
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  RateLimited:
    description: Слишком много запросов
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  ServiceUnavailable:
    description: Сервис временно недоступен
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'

schemas:
  ErrorResponse:
    type: object
    required: [error]
    properties:
      error:
        type: object
        required: [code, message]
        properties:
          code:
            type: string
          message:
```

```
          type: string
        details:
          type: object
          additionalProperties: true

PaginationMeta:
  type: object
  properties:
    total_items:
      type: integer
    total_pages:
      type: integer
    current_page:
      type: integer
    limit:
      type: integer

ConsentRequest:
  type: object
  required: [consent]
  properties:
    consent:
      type: boolean
    fingerprint_hash:
      type: [string, "null"]

VisitorStatus:
  type: object
  properties:
    visitor_id:
      type: string
      format: uuid
    free_interpretation_used:
      type: boolean
    free_interpretation_pending:
      type: boolean
    active_free_generated_spread_id:
      type: [string, "null"]
      format: uuid
    cookie_consent_given:
      type: boolean

DeckResponse:
  type: object
  properties:
    id:
      type: string
      format: uuid
    name:
      type: string
    author:
      type: string
    type:
      type: string
```

```
        enum: [tarot, oracle, playing_cards, other]
      max_cards:
        type: integer
      allow_reversed:
        type: boolean
      suits:
        type: [array, "null"]
        items:
          type: string
      description:
        type: string

  CardResponse:
    type: object
    properties:
      id:
        type: string
        format: uuid
      deck_id:
        type: string
        format: uuid
      name:
        type: string
      suit:
        type: [string, "null"]
      short_description:
        type: string
      full_description:
        type: string
      number:
        type: [integer, "null"]
      reversed_short_description:
        type: [string, "null"]
      reversed_full_description:
        type: [string, "null"]
      image_url_thumb:
        type: [string, "null"]
        format: uri
      image_url_medium:
        type: [string, "null"]
        format: uri
      image_url_original:
        type: [string, "null"]
        format: uri

  DrawCardRequest:
    type: object
    required: [deck_id]
    properties:
      deck_id:
        type: string
        format: uuid
      suit:
        type: [string, "null"]
```

```
SpreadResponse:
  type: object
  properties:
    id:
      type: string
      format: uuid
    name:
      type: string
    description:
      type: string
    amount:
      type: integer
    tags:
      type: array
      items:
        type: object
    image_url:
      type: string
    info:
      type: [string, "null"]
    favorite_deck_ids:
      type: [array, "null"]
      items:
        type: string
        format: uuid

SpreadGenerateRequest:
  type: object
  required: [spread_id, deck_id]
  properties:
    spread_id:
      type: string
      format: uuid
    deck_id:
      type: string
      format: uuid
  additionalProperties: false

GeneratedSpreadResponse:
  type: object
  properties:
    generated_spread_id:
      type: string
      format: uuid
    spread_name:
      type: string
    deck_name:
      type: string
    cards:
      type: array
      items:
        type: object
        properties:
```

```
            position_number:
              type: integer
            position_name:
              type: string
            card_name:
              type: string
            suit:
              type: string
            is_reversed:
              type: boolean
            short_description:
              type: string

    InterpretationRequest:
      type: object
      required: [email, personal_data_consent]
      properties:
        email:
          type: string
          format: email
          maxLength: 254
        user_context:
          type: string
          maxLength: 1500
        personal_data_consent:
          type: boolean
          description: Явное согласие на обработку персональных данных (должно быть
true). При false возвращается ошибка VALIDATION_ERROR.
      additionalProperties: false

    InterpretationResponse:
      type: object
      properties:
        status:
          type: string
          enum: [QUEUED, PROCESSING, COMPLETED, RETRY_WAIT, FAILED,
FAILED_VALIDATION]
        requires_payment:
          type: boolean
        price:
          type: [string, "null"]

    StatusResponse:
      type: object
      properties:
        status:
          type: string
          enum: [QUEUED, PROCESSING, COMPLETED, FAILED, FAILED_VALIDATION,
CANCELED]
        queue_position:
          type: [integer, "null"]
          description: Позиция задачи в очереди. Возвращается null, если задача уже
выполняется или завершена.
        estimated_wait_seconds:
```

```
        type: [integer, "null"]
        description: Ожидаемое время ожидания в секундах. Рассчитывается как (количество задач перед пользователем) * 30 сек. Возвращается null, если задача уже выполняется или завершена.
      is_high_load:
        type: [boolean, "null"]
        description: Флаг, указывающий на перегрузку очереди (превышение MAX_HARD_QUEUE_LIMIT).
      error_code:
        type: [string, "null"]
        description: Код ошибки, если задача упала (например, TIMEOUT_STALE, LLM_CONTEXT_OVERFLOW).
      cancel_reason:
        type: [string, "null"]
        description: Причина отмены (TTL_EXPIRED, ADMIN_MANUAL). Заполняется только если status = CANCELED.

  ViewResultResponse:
    type: object
    properties:
      interpretation_text:
        type: string
      created_at:
        type: string
        format: date-time
      spread_name:
        type: string

  PaymentCreateRequest:
    type: object
    required: [generated_spread_id, email, expected_amount, offer_agreed]
    properties:
      generated_spread_id:
        type: string
        format: uuid
      email:
        type: string
        format: email
        maxLength: 254
      expected_amount:
        type: number
        format: float
        example: 159.00
      offer_agreed:
        type: boolean
    additionalProperties: false

  PaymentCreateResponse:
    type: object
    properties:
      data:
        type: object
        properties:
          payment_id:
```

```
          type: string
          format: uuid
        confirmation_url:
          type: string
          format: uri

AdminPaymentResponse:
  type: object
  properties:
    id:
      type: string
      format: uuid
    yookassa_id:
      type: string
    amount:
      type: number
    status:
      type: string
      enum: [pending, succeeded, canceled, refunded]
    created_at:
      type: string
      format: date-time
    generated_spread_id:
      type: string
      format: uuid
    visitor_id:
      type: string
      format: uuid
    email:
      type: string
    offer_version:
      type: string
    possible_duplicate_payment:
      type: boolean
      description: Флаг дублирующей оплаты (требуется ручной возврат)
    generation_status:
      type: string
      description: Текущий статус снимка расклада

LogResponse:
  type: object
  properties:
    id:
      type: string
      format: uuid
    timestamp:
      type: string
      format: date-time
    level:
      type: string
      enum: [INFO, WARN, ERROR]
    event_type:
      type: string
    actor_id:
```

```
        type: [string, "null"]
        format: uuid
      success:
        type: boolean
      description:
        type: string
      payload:
        type: [object, "null"]

  OutboxEventResponse:
    type: object
    properties:
      id:
        type: string
        format: uuid
      event_type:
        type: string
      status:
        type: string
        enum: [PENDING, PROCESSED, FAILED]
      retry_count:
        type: integer
      last_error:
        type: [string, "null"]
      created_at:
        type: string
        format: date-time

  LegalDocumentVersionResponse:
    type: object
    properties:
      id:
        type: string
        format: uuid
      version:
        type: string
        example: "1.2"
      is_active:
        type: boolean
        description: Флаг, указывающий, является ли эта версия актуальной
      content:
        type: string
        description: Markdown-текст документа
      updated_at:
        type: string
        format: date-time

  AboutContentRequest:
    type: object
    properties:
      content_blocks:
        type: array
        description: Контент страницы «О проекте». Серверная санитизация HTML
обязательна.
```

```
    items:
      type: object
  faq:
    type: array
    description: Блок FAQ. Серверная санитизация HTML обязательна.
    items:
      type: object
```

История изменений

Версия 1.0

Дата: 09.11.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация

1. User Stories:
    a. Переформулирована US-30: убрано слово «редактировать», теперь цель — «видеть историю изменения цен».
    b. Добавлена US-35 (включение 2FA и настройка сброса пароля).
    c. Добавлена US-36 (просмотр и перезапуск зависших Outbox-событий).
    d. Обновлено примечание к разделу US (удалено упоминание 2FA, так как оно вынесено в отдельную US).
2. Админ-панель: В сайдбаре пункт меню переименован в «Мониторинг задач (DLQ и Outbox-события)».

Модель данных и база данных

1. Таблица generated_spreads: Добавлено поле cancel_reason (string, nullable) для хранения причины отмены (TTL_EXPIRED, ADMIN_MANUAL).
2. Таблица outbox_events: Добавлено поле next_retry_at (timestamp, nullable) для реализации механизма экспоненциальной задержки при повторных отправках.
3. Таблица legal_documents:
    a. Поле slug больше не является уникальным (убрано ограничение UNIQUE).
    b. Добавлено поле is_active (bool) для хранения истории версий (активной может быть только одна версия для каждого slug).
    c. Добавлен частичный уникальный индекс idx_legal_docs_active_slug.
4. Примечание к модели данных: Добавлено уточнение, что счетчики резервов слотов (queue_reservations:paid) хранятся только в Redis и не персистятся в PostgreSQL.

API-контракт и OpenAPI

1. Управление юридическими документами:
    a. Добавлен эндпоинт GET /api/admin/legal-docs/{slug}/history для получения истории версий документа.
    b. Изменен PUT /api/admin/legal-docs/{slug} (теперь создает новую версию, а не перезаписывает старую).
2. Управление раскладами: Добавлен эндпоинт POST /api/admin/generated-spreads/{id}/cancel для ручной отмены зависшего снимка.

Платежи:

1. Добавлен параметр фильтрации ?possible_duplicate=true для GET /api/admin/payments.
2. В ответ POST /api/payments/verify добавлено возвращение статуса PAYMENT_PENDING (если платеж отменен, но снимок жив).

3. CSRF-защита:
    a. Добавлен эндпоинт GET /api/admin/auth/csrf для восстановления токена при перезагрузке страницы (F5).
    b. В публичных GET-эндпоинтах (например, GET /api/config/price) явно прописана установка cookie csrf_token.
4. Безопасность URL: В спецификации эндпоинта /view добавлено требование маскировать параметр token в логах.

Схемы OpenAPI:

1. В StatusResponse добавлены поля error_code и cancel_reason.
2. В InterpretationRequest добавлено описание обязательной валидации поля personal_data_consent.
3. Добавлена схема AboutContentRequest с описанием обязательной санитизации.

Бизнес-логика (Платежи, очереди, нейросеть)

1. Резервирование слотов очереди:
    a. Перед созданием платежа внедрено атомарное резервирование слота в Redis (INCR queue_reservations:paid).
    b. При успешной оплате, отмене или истечении 15 минут резерв снимается (DECR).
    c. Добавлена фоновая задача cleanup_expired_reservations для очистки просроченных резервов.
2. Защита от Race Condition:
    a. В логику вебхука добавлен шаг проверки статуса generated_spreads перед созданием outbox_events. Если текст уже сгенерирован или задача в работе, дубль не создается, выставляется флаг possible_duplicate_payment.
    b. При получении вебхука refund.succeeded флаг possible_duplicate_payment сбрасывается в false.
    c. Уточнено, что возврат средств не меняет статус generated_spreads.generation_status.
3. Логика ретраев Outbox: Изменен алгоритм. Вместо 10 попыток по 5 секунд, внедрена экспоненциальная задержка (1 мин, 5 мин, 15 мин, 1 час). Перевод в FAILED происходит только после 15 попыток или по истечении 24 часов.
4. Анонимизация данных (152-ФЗ): Внедрена фоновая задача anonymize_user_data, которая вызывается админом при обработке заявки. Она чистит email в generated_spreads и support_tickets, а также обнуляет fingerprint в anonymous_visitors.
5. Валидация ПДн: В алгоритм эндпоинта /interpretation добавлен обязательный шаг проверки флага personal_data_consent. Если он false или отсутствует — возвращается 422.
6. Модерация и PII: Внедрено обязательное маскирование персональных данных (телефоны, email, номера карт) перед записью «плохого» ответа нейросети в логи.

Безопасность и логирование

1. Хеширование паролей: Везде вместо bcrypt зафиксирован алгоритм Argon2id (с добавлением библиотеки argon2-cffi).

2. Защита от Stored XSS: Внедрена обязательная серверная санитизация Markdown/HTML для юридических документов и страницы «О проекте» (с использованием markdown-it-py и bleach).
3. Защита result_access_token:
    a. Nginx должен маскировать параметр token в логах доступа.
    b. Добавлен заголовок Referrer-Policy: no-referrer и мета-тег для страниц результатов.
    c. Фронтенд обязан очищать URL (history.replaceState) после загрузки страницы.
4. CORS: В раздел CSRF добавлено требование передавать credentials: 'include' и настроить заголовок Access-Control-Allow-Credentials: true.
5. Мониторинг: Внедрены обязательные алерты строго на email. Добавлены новые триггеры для уведомлений (проблемы с чеками, Outbox, DLQ, заполнение очереди на 80% и 100%).
6. Логирование: Раздел «Категории событий» полностью переструктурирован. Добавлены события: security.pd_consent_missing, payment.duplicate_detected, queue.reservation_expired, queue.overshoot_payment, privacy.user_anonymized и другие.

Пользовательский интерфейс (UI/UX)

1. Раздел «Просмотр платежей»: Добавлены колонки со статусом снимка расклада, подсказка «Ожидает повторной оплаты» (если платеж отменен, но расклад жив) и красная иконка для дублирующих оплат.
2. Раздел «Просмотр статистики»: Добавлен блок «Требуют внимания» со счетчиками (дублирующие оплаты, новые обращения, задачи в DLQ/Outbox).
3. Раздел «Управление ценами»: Явно указано, что история цен доступна только в режиме просмотра.

Приложения (схемы и DDL)

1. Приложение 6 и 7 (State Machine): Перерисованы схемы. Убран переход в CANCELED при переполнении очереди. При отмене платежа снимок остается в PAYMENT_PENDING. Переход CANCELED -> QUEUED разрешен только для TTL/ручной отмены.
2. Приложение 9 (DDL): Обновлен в соответствии с изменениями в generated_spreads (cancel_reason), outbox_events (next_retry_at) и legal_documents (версионирование). Убрана опечатка pow().
3. Приложение 10 (OpenAPI):
    a. Добавлены все новые пути и схемы.
    b. Все JSON-примеры приведены к стилю snake_case (ранее местами был camelCase).
    c. Исправлена вложенность additionalProperties: false.

Версия 0.7

Дата: 29.10.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация

1. User Stories: Добавлено примечание после US-34, разъясняющее, что технические сценарии (2FA, health-check, алерты) не выносятся в отдельные US, а покрываются НФТ и ФТ.

Модель данных и база данных

1. Платежи и чеки (422-ФЗ):
    a. В таблицу payments добавлено поле email (обязательное, не подлежит анонимизации).
    b. В таблицу receipt_jobs добавлено поле email (копируется из payments для отправки чеков).
2. В разделе «Просмотр платежей» уточнено, что email подтягивается напрямую из payments, а не через JOIN с generated_spreads.
3. Инварианты anonymous_visitors: Добавлено правило принудительного завершения «зависших» задач (статус QUEUED > 60 мин, PROCESSING > 10 мин) для разблокировки бесплатного резерва.
4. Outbox-события: В таблице outbox_events уточнено, что статус FAILED требует ручного вмешательства администратора.

API-контракт и OpenAPI

1. Согласие на обработку данных:
    a. Добавлена ошибка 403 CONSENT_REQUIRED для эндпоинта запроса толкования (если пользователь отказался от передачи fingerprint).
    b. В спецификацию POST /api/consent добавлена логика обнуления поля fingerprint в БД при отзыве согласия.
2. Восстановление Outbox: Добавлены эндпоинты GET /api/admin/outbox и POST /api/admin/outbox/{id}/retry для просмотра и перезапуска зависших событий.
3. Создание платежа: В POST /api/payments/create добавлено требование сохранять email в таблицу платежей.
4. Валидация Email: Во всех схемах добавлена строгая валидация по RFC 5322 и ограничение длины maxLength: 254.
5. Исправление OpenAPI: Исправлена ошибка размещения additionalProperties: false в схеме InterpretationRequest. В ответ InterpretationResponse добавлены статусы FAILED и FAILED_VALIDATION.

Платежи, нейросеть и юридические аспекты

1. Логика модерации 16+: Если задача упала с ошибкой FAILED_VALIDATION по причине срабатывания модерации, пользователю отображается кнопка «Запросить возврат средств», а администратор обязан оформить возврат.
2. Обработка ошибок контекста: При ошибке LLM_CONTEXT_OVERFLOW задача сразу переводится в FAILED_VALIDATION (0 попыток ретрая).
3. Ограничения очередей: Обновлены текстовые описания лимитов (50 минут ожидания при аварийном пределе в 100 задач). Уточнен формат отображения очереди в админ-панели.
4. Валидация конфигурации чеков: Сервис yookassa-to-mynalog должен аварийно завершать работу (fail-fast), если не заданы обязательные переменные окружения.

5. Логика отправки Email: Воркер перед отправкой письма обязан проверять флаг email_delivered. Ошибка почтового сервиса больше не влияет на общий статус COMPLETED генерации толкования.

Пользовательский интерфейс (UI/UX)

1. Защита от отзыва Cookie: В футер добавлена ссылка «Настройки Cookie», открывающая модальное окно для изменения согласия (152-ФЗ).
2. Обработка отказа от fingerprint: Добавлено модальное окно с кнопкой «Согласиться и получить бесплатно» для пользователей, ранее отклонивших сбор данных.
3. Время ожидания: Если estimated_wait_seconds равно null, интерфейс отображает «Ожидаемое время: менее минуты». При аварийной нагрузке выводится честное предупреждение о возможном ожидании до 50 минут.
4. Режим Read-Only: При недоступности Redis фронтенд блокирует кнопки действий и показывает глобальный баннер о временной недоступности создания раскладов и оплаты.

Безопасность и НФТ

1. Rate Limiting: Для сброса пароля внедрен двухуровневый лимит (по IP и по конкретному email).
2. Безопасность инфраструктуры: Обновлена политика Content Security Policy (CSP) для поддержки Sentry, S3 и API. Добавлена классификация сбоев для RTO (критичные и некритичные).

Приложения

1. Приложение 1 (UseCase): Обновлена диаграмма, добавлены кейсы настройки Cookie, управления юридическими документами и мониторинга Outbox.
2. Приложение 3 (UML-последовательность): Добавлен диспетчер Outbox, перерисована схема постановки задачи в очередь.
3. Приложения 6 и 8 (State Machine): Добавлены переходы QUEUED -> FAILED (таймаут в очереди), PROCESSING -> FAILED_VALIDATION (ошибка контекста) и обратные переходы для ручного перезапуска.

Версия 0.6

Дата: 29.10.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация

1. User Stories: Переформулирована US-06. Теперь она четко разделяет базовый функционал (без сбора ПДн) и запрос AI-толкования (требует email для доставки результата и формирования чека по 422-ФЗ). В раздел «Общие требования» внесено соответствующее уточнение.

Архитектура, безопасность и API

1. API-контракт:

    a. Внедрен строгий формат обертки data во всех примерах ответов (включая /interpretation, /status, /payments/verify).
    b. В Приложение 10 добавлен полный YAML-файл OpenAPI 3.1 со всеми путями (Paths), компонентами и схемами (схемы для колод, карт, раскладов, платежей).
2. Идемпотентность: Детализирована работа с Idempotence-Key при оплате. Указано, что при получении ошибки PRICE_CHANGED фронтенд обязан сгенерировать новый ключ.

Безопасность (CSP и Rate Limiting):

1. В заголовок Content-Security-Policy добавлена директива frame-ancestors 'none'; для защиты от кликджекинга.
2. Разделены лимиты для публичных GET (120 запросов/мин) и POST (30 запросов/мин) запросов.
3. Оптимизация поиска: В админ-панели добавлено требование использовать задержку ввода (debounce) не менее 300 мс для полей поиска.

Модель данных и база данных

1. DDL (Приложение 9): Удален фантомный статус AWAITING_REQUEST из ENUM generation_status_enum.

Платежи, нейросеть и юридические аспекты

1. Логика повторных запросов: В алгоритм эндпоинта /interpretation добавлен шаг №3: проверка таблицы payments. Если для расклада уже есть успешный платеж, система пропускает проверку бесплатного лимита и сразу ставит задачу в QUEUED. Фронтенд при нажатии «Повторить» больше не вызывает платежный шлюз.
    a. Валидация ответа LLM: Внедрены четкие критерии «брака» на стороне воркера: Длина (300–4000 символов);
    b. Язык (кириллица > 70%);
    c. Стоп-слова (утечка промпта);
    d. Семантическая релевантность (упоминание хотя бы 2 карт из снимка);
    e. Модерация 16+.
2. Очереди и лимиты: Уменьшены дефолтные значения лимитов очередей для предотвращения длительного ожидания: MAX_FREE_QUEUE_LENGTH (10), MAX_PAID_QUEUE_LENGTH (20), MAX_HARD_QUEUE_LIMIT (100). Обновлена формула расчета ETA (учитывает общее количество задач перед пользователем).
3. Логика вебхуков: Убрано требование сохранения текста LLM из вебхука (шаг 11). Вебхук только фиксирует факт оплаты и пишет в Outbox.
4. Юридические требования: В подвал сайта добавлены обязательные реквизиты TAXPAYER_NAME и TAXPAYER_INN. В разделе управления юр. документами добавлен третий документ user-agreement.

Пользовательский интерфейс (UI/UX)

1. Защита от потери результата: Внедрен механизм GET /api/spreads/latest-unviewed (описана логика работы баннера и выдачи result_access_token).

Уточнено, что beforeunload срабатывает только если пользователь уже отправил форму и задача находится в QUEUED или PROCESSING.
2. Инструкция по оплате: Добавлено предупреждение о том, что при закрытии вкладки результат и чек придут на указанный email.

Инфраструктура и отказоустойчивость
1. RPO (Точка восстановления): Разделены понятия целевого RPO (15 минут при штатной работе WAL) и аварийного RPO (до 24 часов при потере WAL). Настроен обязательный алерт при сбоях команды archive_command.
2. Сценарии деградации: Описана частичная деградация при падении Redis. GET-запросы продолжают работать (напрямую из БД), блокируются только POST-запросы (fail-closed для финансовых операций и rate limiting).
3. Мониторинг: В разделе «Мониторинг» акцентировано требование настройки алертов при ошибках архивации WAL.
4. Приложения: Диаграммы состояний (Приложения 6 и 7) обновлены — из них удален статус AWAITING_REQUEST.

Версия 0.5

Дата: 28.10.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация
1. User Stories: Добавлены US-33 (управление юридическими документами), US-34 (ознакомление с офертой), US-35 (2FA и сброс пароля для админа). Переформулирована US-15 (чистая выручка и возвраты).
2. Новые разделы: Добавлены разделы «Управление юридическими документами», «Управление запросами на удаление ПДн», «Сценарии деградации при сбоях инфраструктуры», «Стандарты API-контракта (OpenAPI 3.1)».
3. Приложения:
    a. Добавлены Приложения 6, 7, 8 с детальными UML-диаграммами состояний (High-Level, Оплата, Работа воркера).
    b. Добавлено Приложение 9 с итоговым DDL-скриптом для PostgreSQL.
    c. Добавлено Приложение 10 с фрагментом OpenAPI-схемы.
4. Исправление опечаток: Исправлены favoriteDecIdIs на favoriteDeckIds, yookassa на yookassa, убраны круглые скобки в URL эндпоинтов API на фигурные.

Архитектура, безопасность и API
1. CSRF-защита: Требование о CSRF полностью переписано. Внедрен паттерн Synchronizer Token для админки (заголовок X-CSRF-Token) и Double Submit Cookie для публичной части. Добавлена проверка Origin/Referer.
2. Сбор Fingerprint: Сбор данных перенесен на сторону клиента (Canvas, WebGL). Бэкенд принимает хэш через POST /api/consent, добавляет соль и сохраняет.
3. Защита от Email Enumeration: Эндпоинт сброса пароля теперь всегда возвращает 200 OK с одинаковым текстом (независимо от наличия email в БД), добавлена защита от Timing Attacks.

4. Rate Limiting: Добавлены лимиты для формы обратной связи (3 запроса / 10 минут) и эндпоинта verify-email (5 попыток на расклад / 10 с IP в час). Внедрена стратегия fail-closed при недоступности Redis.
5. Безопасность инфраструктуры: Добавлены заголовки HSTS и конкретные домены в CSP. Внедрена защита от Mass Assignment (Pydantic extra="forbid"). Добавлен эндпоинт GET /api/health.
6. OpenAPI: Внедрены стандарты пагинации, сортировки (sort_by, sort_order, search) и строгие форматы ответов/ошибок.

Модель данных и база данных

1. Новые таблицы:
    a. analytics_events (для событийной аналитики).
    b. legal_documents (для хранения оферты и правил возврата).
    c. outbox_events (для паттерна Transactional Outbox).
2. Soft Delete (Мягкое удаление): Для колод, карт и обращений внедрены частичные уникальные индексы в PostgreSQL (WHERE is_deleted = false), чтобы избежать конфликтов имен при удалении.
3. Доработка таблиц:
    a. anonymous_visitors: добавлены free_interpretation_pending и fingerprint_updated_at.
    b. generated_spreads: обновлен enum статусов (добавлен PAYMENT_PENDING и др.), добавлена связь visitor_id.
    c. payments: добавлены visitor_id, possible_duplicate_payment, offer_agreed_at, offer_version.
    d. support_tickets: добавлены поля для мягкого удаления (is_deleted, deleted_at, deleted_by).
    e. data_deletion_requests: добавлены email_hash и rejection_reason.
    f. spreads: формат поля tags изменен с массива строк на массив объектов [{"normalized_name": "...", "display_name": "..."}].

Платежи, нейросеть и юридические аспекты

1. Гонка цен: Внедрен механизм expected_amount при создании платежа для предотвращения рассинхронизации цен (ошибка PRICE_CHANGED).
2. Логика бесплатного толкования: Внедрен флаг free_interpretation_pending. Теперь право считается использованным только после успешного COMPLETED. При ошибках FAILED право сохраняется.
3. Очереди и бюджетирование токенов: Внедрены раздельные лимиты MAX_FREE_QUEUE_LENGTH, MAX_PAID_QUEUE_LENGTH и MAX_HARD_QUEUE_LIMIT. Добавлен механизм обрезки контекста (Token Budgeting) и ошибка LLM_CONTEXT_OVERFLOW.
4. Надежность (Outbox): Внедрен паттерн Transactional Outbox. Прямая отправка задач в Redis запрещена. Добавлена блокировка SKIP LOCKED и механизм ретраев для диспетчера.
5. Возвраты и чеки: Автоматический возврат через API запрещен (только через ЛК ЮKassa). Аннулированием чеков занимается исключительно yookassa-to-mynalog. Сервису чеков добавлен Docker Volume и бэкап SQLite.

6. Конфигурация налогов: Добавлены обязательные переменные окружения TAXPAYER_NAME и TAXPAYER_INN. Сервис чеков завершает работу (fail-fast), если они не заданы.
7. Модерация 16+: Внедрена пост-модерация текста LLM на стороне воркера (стоп-слова и запрещенные темы).

Пользовательский интерфейс (UI/UX)

1. Защита от потери результата: Внедрен токен result_access_token (JWT), который позволяет открывать результат по ссылке из письма с любого устройства. Предупреждение beforeunload при закрытии вкладки во время ожидания.
2. Экран ожидания: Текст ожидания стал динамическим (учитывает ETA и перегрузку очереди).
3. Валидация карт: В админ-панели масть карты теперь выбирается из выпадающего списка (dropdown), сгенерированного из deck.suits. Запрещено удалять масть из колоды, если с ней есть карты (ошибка SUIT_IN_USE).
4. Юридические согласия: На экран оплаты добавлен обязательный чекбокс согласия с офертой и правилами возврата (offer_agreed). Добавлен дисклеймер ИИ и 16+ под результатом.
5. Ссылки на возврат: В таблице платежей добавлена ссылка для перехода в ЛК ЮKassa (вместо кнопки прямого возврата в нашей админке).

Инфраструктура и отказоустойчивость

1. Отказоустойчивость: Описаны сценарии деградации при падении PostgreSQL (ошибка DATABASE_UNAVAILABLE), S3 (заглушки картинок) и Nginx (мониторинг).
2. Фоновые задачи (Celery Beat):
    a. cleanup_stale_free_reservations (очистка зависших бесплатных резервов).
    b. cancel_stale_payments (отмена брошенных платежей в статусе PAYMENT_PENDING старше 24 часов).
    c. reconcile_receipts (мониторинг ошибок генерации чеков).
3. Мониторинг DLQ: Добавлен счетчик «Зависшие Outbox-события» на дашборд.
4. Docker: Настроены Volume для yookassa-to-mynalog, добавлены политики restart: always и healthcheck для всех сервисов.
5. Логирование: Внедрена очистка логов только через удаление старых партиций (retention policy 30 дней), ручная очистка через интерфейс админки удалена.

Версия 0.4

Дата: 27.10.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация

1. User Stories: Добавлены новые US-31 (отзыв согласия на обработку ПДн) и US-32 (обработка запросов на удаление данных). Уточнена US-08 (бесплатное толкование не сгорает при ошибках). Расширена US-22 (аудит новых действий).

2. Исправление опечаток и терминологии: Исправлены опечатки favoriteDeckIds, beforehand, символ избранной колоды заменен с «×» на «★». Удален дублирующийся текст в разделах про колоды и карты.
3. Удаление устаревшего функционала: Из административной части окончательно убрано требование «управлять выплатами» (переведено в автоматический режим на стороне ЮKassa).

Архитектура, безопасность и API

1. Безопасная идентификация: Отказались от генерации visitor_id на клиенте. Внедрен серверный подписанный JWT-токен (visitor_token), который хранится в HttpOnly-cookie (с флагами Secure, SameSite=Strict).
2. Разделение cookie: Внедрено разделение на строго необходимые (Essential) и аналитические (Fingerprint). Добавлен эндпоинт POST /api/consent для сохранения согласия пользователя.
3. Защита доступа к результатам: Эндпоинт /status больше не возвращает текст толкования (только метаданные). Текст отдается через /view с проверкой JWT-токена. Флаг is_viewed меняется только при успешной отдаче текста.
4. API-контракт: Добавлен полный перечень недостающих эндпоинтов (получение колод, тегов, случайной карты, api/payments/verify, api/spreads/{id}/interpretation). Внедрены query-параметры фильтрации для админских списков.
5. Дополнение справочника ошибок: Добавлены коды 409 (Conflict), 429 (Rate Limited), 500 (Internal Error), 402 (Payment Required).
6. Очереди Celery: Зафиксирован полный список из 4 очередей (llm_tasks, email_tasks, payment_tasks, dlq). Прописана маршрутизация задач и разделение настроек воркеров (concurrency, приоритеты).

Модель данных и база данных

1. Расширение схемы БД: Количество таблиц увеличено с 16 до 19. Добавлены таблицы outbox_events (для паттерна Transactional Outbox), support_ticket_comments (история комментариев) и data_deletion_requests (запросы на удаление ПДн).
2. Доработка существующих таблиц:
    a. Добавлены поля для контроля времени: consent_at, email_delete_after, is_free_request, contact_channel, subject.
    b. В таблицу receipt_jobs добавлено ограничение UNIQUE(payment_id) для защиты от дублей чеков.
    c. В таблицу cards добавлено ограничение UNIQUE(deck_id, number) для защиты от дублирования номеров карт.
    d. В таблицу logs добавлен составной первичный ключ (id, timestamp) для корректного партиционирования.
3. Оптимизация: Прописаны обязательные индексы для PostgreSQL (B-tree, GIN) для обеспечения заявленных 50 RPS. Для тяжелой статистики добавлены Materialized Views.

Платежи, нейросеть и юридические аспекты

1. Паттерн Outbox: Внедрен механизм гарантированной постановки задач в очередь. Обновление статуса платежа и создание задачи в Redis теперь происходят через транзакцию в БД и таблицу outbox_events.
2. Логика бесплатного толкования: Флаг free_interpretation_used теперь меняется на true только после успешного завершения генерации (статус COMPLETED). При ошибках (FAILED) бесплатное право сохраняется.
3. Приоритизация и ETA: Внедрены приоритеты задач (платные запросы идут с priority=9, бесплатные — с priority=1). Добавлен расчет ожидаемого времени (ETA) на основе длины очереди.
4. Обработка ошибок нейросети: Внедрена единая матрица retry. Лимиты токенов синхронизированы (maxTokens 2000, валидация длины от 300 до 4000 символов). Добавлена защита от переполнения контекстного окна (обрезка пользовательского контекста до 1500 символов).
5. Безопасность вебхуков: Внедрена обязательная проверка IP-адресов ЮKassa и MD5-подписи. Добавлена обязательная перепроверка статуса через API ЮKassa. Реализован механизм верификации платежа через api/payments/verify с транзакционной блокировкой.
6. 152-ФЗ и Retention: Внедрена ежедневная фоновая задача anonymize_old_emails для анонимизации email старше 12 месяцев. Добавлен механизм отзыва согласия через форму поддержки.
7. Безопасность загрузки: Внедрена поддержка загрузки ZIP-архивов (CSV + изображения). Добавлена защита от Zip Slip, MIME-type spoofing и «бомб декомпрессии».

Пользовательский интерфейс и UX

1. Экран ожидания: Текст ожидания стал динамическим (учитывает ETA и перегрузку очереди). Добавлен перехват закрытия вкладки (beforeunload) с предупреждением.
2. Просмотр карт: Добавлен поиск, фильтр по мастям и просмотр перевернутых значений.
3. Отображение картинок: Используется loading="lazy", формат WebP с fallback на JPEG (через тег <picture>). Размеры оптимизированы (thumb, medium, original) и хранятся в S3.
4. Создание расклада: Добавлена блокировка карт (SELECT FOR UPDATE), чтобы админ не мог удалить колоду в момент генерации. При конфликте возвращается ошибка 409.
5. Обработка блокировщиков окон: Если браузер блокирует открытие нового окна при оплате, происходит редирект на страницу ЮKassa в текущей вкладке (с сохранением generated_spread_id в return_url).

Инфраструктура и отказоустойчивость

1. СУБД: Настроено непрерывное архивирование WAL и PITR для обеспечения RPO ≤ 15 минут для всей базы данных. Внедрено тестирование восстановления из бэкапов (Restore Tests) раз в месяц.
2. Кэширование: Внедрено кэширование публичных GET-запросов (списки колод, карт, тегов) в Redis на 15 минут для снижения нагрузки на БД.

3. Docker: Для всех сервисов (включая MinIO) настроены политики restart: always и healthcheck. Добавлено обязательное использование S3-хранилища. Увеличен лимит client_max_body_size до 50 МБ для загрузки карт.
4. Безопасность инфраструктуры: Внедрены ограничения размера JSON, защита от Mass Assignment и SSRF. Добавлено шифрование данных at rest (БД и бэкапы).

Приложения

1. Обновлена UseCase диаграмма (Приложение 1): добавлены новые кейсы (управление картами, ценами, промптом, обработка обращений, DLQ), убран кейс «Вывести деньги».
2. Обновлена UML-диаграмма состояний (Приложение 6): внедрены новые статусы (AWAITING_PAYMENT, AWAITING_REQUEST, RETRY_WAIT) для разделения этапов оплаты и генерации.

Версия 0.3

Дата: 23.10.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация

1. User Stories: Добавлены новые US (US-25, US-28, US-29, US-30) для закрытия «висячих» требований (форма обратной связи, управление картами, DLQ, история цен). Переформулирована US-15 (выплаты теперь происходят на стороне ЮKassa). Расширена US-22 (аудит всех действий).
2. Новые разделы: Добавлены разделы «Модель данных», «Спецификация API», «Диаграмма состояний задач (State Machine)», «Технические спецификации и архитектура».
3. Удалены/Изменены: Убран раздел «Нагрузочные характеристики» (объединен с «Производительностью»). Убрано требование «управлять выплатами» из административной части (заменено на «просматривать историю платежей»).

Архитектура и модель данных

1. Модель данных: Спроектировано 16 таблиц БД с типами данных (UUIDv4, JSONB, enum). Добавлена таблица generated_spreads для хранения снимков (snapshots) раскладов на момент генерации. Добавлены таблицы для сессий, 2FA, чеков, промптов, FAQ и обращений.
2. API-контракт: Добавлен полный перечень эндпоинтов для публичного API, вебхуков и админ-панели. Внедрен единый формат ошибок и успешных ответов.
3. Очереди Celery: Разделены на 4 независимые очереди: llm_tasks, email_tasks, payment_tasks и dlq (Dead Letter Queue). Настроена маршрутизация задач.
4. Инфраструктура: Обязательным стало использование S3-хранилища (MinIO/Yandex Object Storage) для картинок. Внедрены healthcheck и политики перезапуска restart: always для Docker. Настроен Nginx для проксирования S3 и ограничения размера JSON (client_max_body_size).

5. Отказоустойчивость БД: Для обеспечения RPO ≤ 15 минут по платежам внедрено непрерывное архивирование WAL-журналов PostgreSQL и настройка Point-in-Time Recovery (PITR).

Платежи и юридические аспекты

1. Безопасность вебхуков: Внедрена обязательная проверка IP-адресов ЮKassa и MD5-подписи. Добавлена обязательная перепроверка статуса платежа через API ЮKassa. Транзакционная блокировка (SELECT FOR UPDATE) защищает от двойной постановки задач в очередь.
2. Очередь и оплата: Если очередь переполнена (MAX_QUEUE_LENGTH = 20), новые платежи блокируются. Однако уже оплаченные задачи гарантированно ставятся в очередь.
3. Возвраты (422-ФЗ): При возврате средств система обязана аннулировать фискальный чек в «Мой Налог» (через yookassa-to-mynalog). Если AI-толкование уже сгенерировано (статус COMPLETED), администратор вправе отказать в возврате.
4. 152-ФЗ: Внедрен Cookie-баннер для сбора отпечатка (fingerprint) с кнопками «Принять/Отклонить». Чекбокс согласия на обработку email дополнен целями (чеки, связь, рассылка). Прописан порядок отзыва согласия через форму поддержки.
5. Чеки: Детализированы обязательные реквизиты чека. Добавлен статус фискального чека в карточку платежа (created, failed, annulled).

Интеграция с YandexGPT и валидация

1. Ограничения токенов: Установлен лимит maxTokens: 2000. Если суммарный контекст превышает 6000 токенов, вопрос пользователя обрезается до 1500 символов. Валидация ответа установлена в диапазоне от 300 до 4000 символов.
2. Матрица Retry: Создана единая политика повторных попыток: 429/503 — до 5 раз, Timeout — 1 раз, ошибки 400/401/403 — 0 раз (сразу FAILED), ошибки валидации — до 2 раз.
3. Потеря результата: Внедрены уникальные Signed URL (срок жизни 30 дней), которые отправляются на email. Если email не доставлен, пользователь может увидеть баннер на главной странице и подтвердить личность через ввод email.
4. Флаг is_viewed: Меняется на true только при успешном отображении результата бэкендом. Воркер этот флаг не трогает.

Пользовательский интерфейс и UX

1. Ожидание генерации: Текст на экране ожидания стал динамическим (зависит от длины очереди). Внедрено предупреждение beforeunload при попытке закрыть вкладку.
2. Просмотр карт: Добавлен поиск, фильтр по мастям и просмотр перевернутых значений.
3. Отображение картинок: Используется loading="lazy", формат WebP с fallback на JPEG (через тег <picture>). Размеры оптимизированы (thumb, medium, original).
4. Создание расклада: Добавлена блокировка карт (SELECT FOR UPDATE), чтобы админ не мог удалить колоду в момент генерации. При конфликте возвращается ошибка 409.

Админ-панель

1. Мягкое удаление (Soft Delete): Для колод и карт добавлен флаг is_deleted. Удаленные сущности скрываются из пользовательской части, но остаются в статистике. Названия удаленных колод игнорируются при проверке уникальности.
2. Загрузка карт (CSV): Внедрена защита от CSV-инъекций (символы =, +, -, @). Формат CSV детально описан. Добавлена проверка изображений на MIME-тип, размер и «бомбы декомпрессии».
3. Управление ценами: Добавлена история изменения цен (кто и когда менял). Изменение цены применяется динамически.
4. Мониторинг (DLQ): Добавлен интерфейс для просмотра упавших задач, чтения stack trace и их ручного перезапуска.

Безопасность и НФТ

1. RPS: Унифицированы показатели нагрузки: 50 RPS на чтение и 5 RPS на запись.
2. Пароли: Длина пароля администратора увеличена до 10 символов. Внедрена защита от Mass Assignment и SSRF. Настроена CSP и шифрование БД.
3. Rate Limiting: Прописаны конкретные лимиты для авторизации, API и сброса пароля. При падении Redis применяется стратегия fail-closed.
4. Время: Унифицировано хранение в UTC в БД и отображение в UTC+5 в интерфейсе. Исправлены опечатки (например, favoriteDeckIds вместо favoriteDecIdls).

Версия 0.2

Дата: 22.10.2026
Автор изменений: Ильиных Дарья, системный аналитик

Глобальные изменения и документация

1. Добавлены новые User Stories (US-19 – US-27): закрыты «висячие» требования из версии 0.1 (управление промптом, страницей «О проекте», ценами, обращениями, логами аудита, автоматический вывод средств, чеки, email-копии, rate limiting).
2. Добавлены новые разделы: «Просмотр значения карт», «Управление ценами», «Мониторинг задач (DLQ)», «Безопасность», «Модель данных», «Спецификация API», «Диаграмма состояний задач».
3. Обновлены приложения: Добавлена UML-диаграмма состояний задачи (Приложение 6).
4. Разрешены противоречия:
    a. Часовой пояс унифицирован и зафиксирован как UTC+5 (в версии 0.1 были разночтения между МСК и часовым поясом администратора).
    b. Таймауты ожидания нейросети унифицированы до 60 секунд.
    c. Цена толкования приведена к единому значению — 159 ₽ (в версии 0.1 в JSON-примере было 199.00).

Архитектура и модель данных

1. Добавлена структура БД: Описаны основные сущности (таблицы), их поля и типы (UUIDv4, timestamps). Внедрена таблица generated_spreads для хранения снимков раскладов.
2. Добавлена спецификация API: Внедрен единый формат ответа {"data": ...}, стандарт пагинации (?page=1&limit=20), закрытый Swagger UI (/api/docs).
3. Внедрен State Machine (Машина состояний): Описан строгий жизненный цикл задачи (PENDING → QUEUED → PROCESSING → COMPLETED / FAILED / FAILED_VALIDATION / CANCELED).
4. Очереди Celery: Разделены на llm_tasks (высокий приоритет) и email_tasks. Внедрена настройка task_acks_late=True и task_reject_on_worker_lost=True для защиты от потери задач.
5. Отказ от избыточности для MVP: Prometheus, Grafana и Loki убраны из обязательных требований (перенесены в этап масштабирования). Оставлены Sentry и Flower.

Платежи и юридические аспекты

1. Динамическая цена: Цена теперь хранится в БД (interpretation_prices) и подтягивается на фронтенд и в API ЮKassa динамически.
2. Защита от перегрузки: Перед созданием платежа система проверяет длину очереди (MAX_QUEUE_LENGTH). При переполнении деньги не списываются.
3. Сверка платежей (Reconciliation): Добавлена задача Celery Beat для проверки «зависших» платежей (статус pending > 10 минут) через API ЮKassa.
4. Чеки (422-ФЗ): Детализированы требования к чекам (ФИО, ИНН, точное наименование услуги «Толкование расклада Таро»). Добавлена логика повторных попыток при сбоях ФНС.
5. 152-ФЗ: Добавлен обязательный чекбокс согласия на обработку email перед оплатой.

Интеграция с YandexGPT и валидация

1. Защита от галлюцинаций: Внедрена программная валидация ответа LLM (длина от 300 до 5000 символов, проверка на русский язык, фильтрация стоп-слов типа «Как языковая модель»).
2. Идемпотентность: Внедрена распределенная блокировка Redis (SET NX), чтобы исключить дублирование запросов для одного расклада.
3. Снимки раскладов (Snapshot): Нейросеть теперь получает данные из снимка на момент вытягивания карт, а не из актуального шаблона в админке.
4. Независимость от браузера: Воркер обязан завершить задачу и сохранить результат, даже если пользователь закрыл вкладку.

Пользовательский интерфейс и UX

1. Просмотр карт: Добавлен отдельный функционал со списком колод, поиском, фильтрами по мастям и возможностью просмотра прямого/перевернутого значения.
2. Защита от потери результата: При закрытии вкладки во время генерации вызывается beforeunload с предупреждением.
3. Кэширование: Если пользователь вернулся на страницу, система отдает уже готовое толкование без повторной оплаты и генерации.

4. Таймаут оплаты: Если оплата не прошла за 15 минут, опрос статуса прекращается.

Админ-панель

1. Soft Delete (Мягкое удаление): Для колод и карт внедрен флаг is_deleted. Добавлен «Архив» и возможность восстановления.
2. Загрузка карт: Формат массовой загрузки изменен с TXT на CSV (с детальным описанием разделителей и кодировки). Внедрена автоматическая обработка изображений (WebP, ресайз, удаление оригиналов).
3. Безопасность: Добавлен раздел 2FA, сброс пароля по email, парольная политика (от 10 символов), CSRF-защита.
4. DLQ (Dead Letter Queue): Добавлен интерфейс для просмотра «упавших» задач, чтения stack trace и повторного запуска.

Безопасность и НФТ

1. Rate Limiting: Прописаны конкретные лимиты (5 попыток входа, 60 запросов к API, 3 сброса пароля).
2. CSP и Шифрование: Добавлены требования к Content Security Policy и шифрованию БД (Encryption at rest).
3. RPS: Увеличены показатели нагрузки (50 RPS на чтение, 5 RPS на запись).
4. RPO: Для финансовых данных ужесточен RPO (не более 1 часа).
5. Доступность: Добавлен стандарт WCAG 2.1 AA и минимальный размер интерактивных элементов (44x44px).